


欢迎选修《数据结构》

- 课程编号：30240184
- 授课教师：邓俊辉、尹 霞、陈键飞
- 开课单位：计算机系

《数据结构与算法》DSA学什么？

线性结构

- 01.Introduction
  用尺子丈量
- 02.Vector
  向量的排序
- 03.List
  列表的排序
- 04.Stack+Quene
  各类应用

半线性结构

- 05.Binary_Tree
  树的遍历
- 06.Binary_Search_Tree
- 07.BST_Application
- 08.Advanced_BST
  各种树及中序遍历
- 09.Dictionary
  小容器放下大空间

非线性结构

- 10.Graph
- 11.Graph_Application
  图的遍历
- 12.PQ
  只关注极值的“树”
- 13.String
  最常见的串匹配
- 14.Sorting
  永恒的排序算法

独特又普适的树结构：
兼顾向量和列表优势的半线性结构

❖树结构在算法理论和实际应用中始终都扮演着**最为关键**的角色，

并且有不计其数的**变种**。

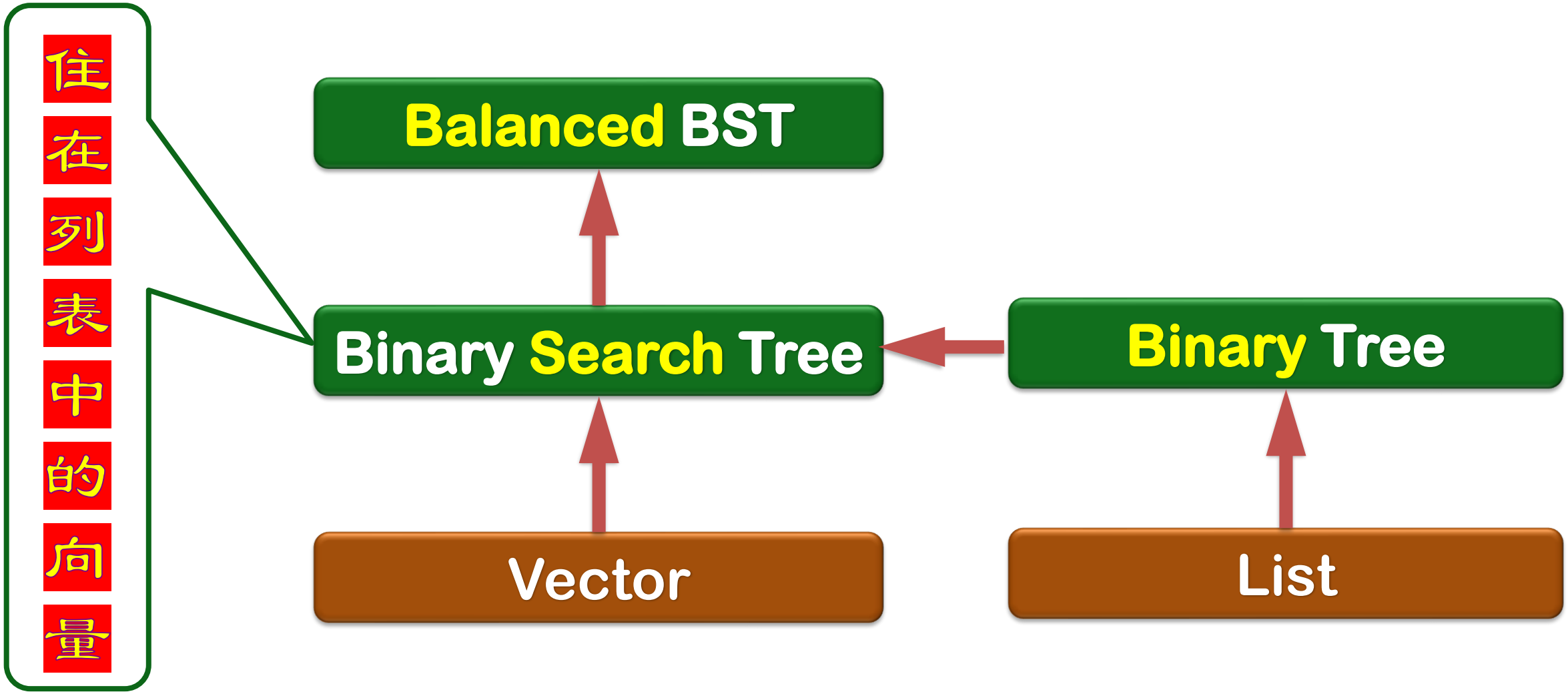
❖树是一种自组织的分层结构，这种结构几乎蕴含于所有事物及其关系中：

- 信息处理系统的文件系统、数据库系统
- 互联网域名系统
- 地理生态系统
- 人类社会系统

❖二叉树是树的特例，无论是逻辑结构还是算法功能，

任何有序多叉树都可以等价地转化为二叉树。

高效地兼顾静态查找与动态修改：求平衡



05. Binary Tree

A. 树（术语和ADT）

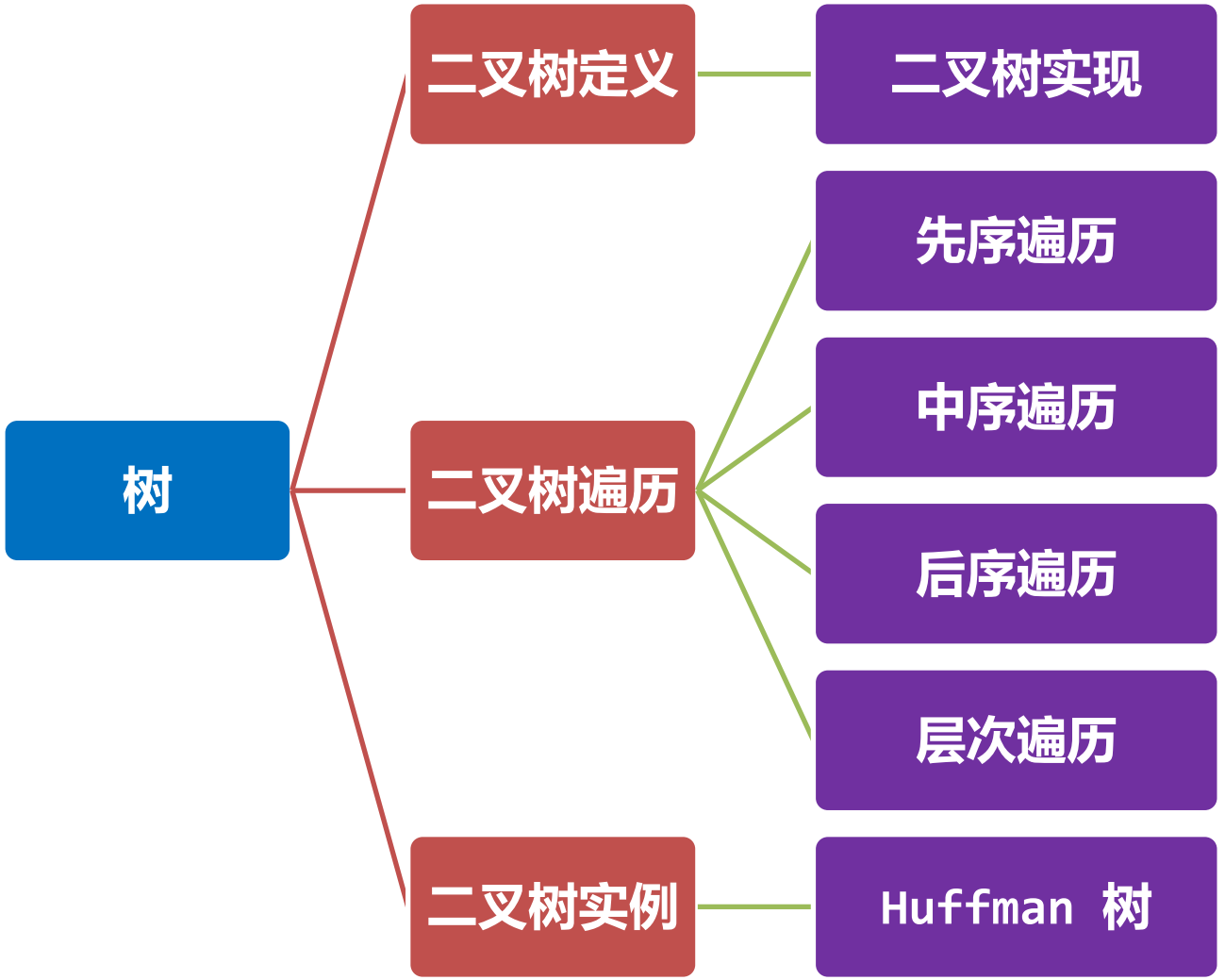
B. 二叉树（转换与实现）

C. 二叉树遍历

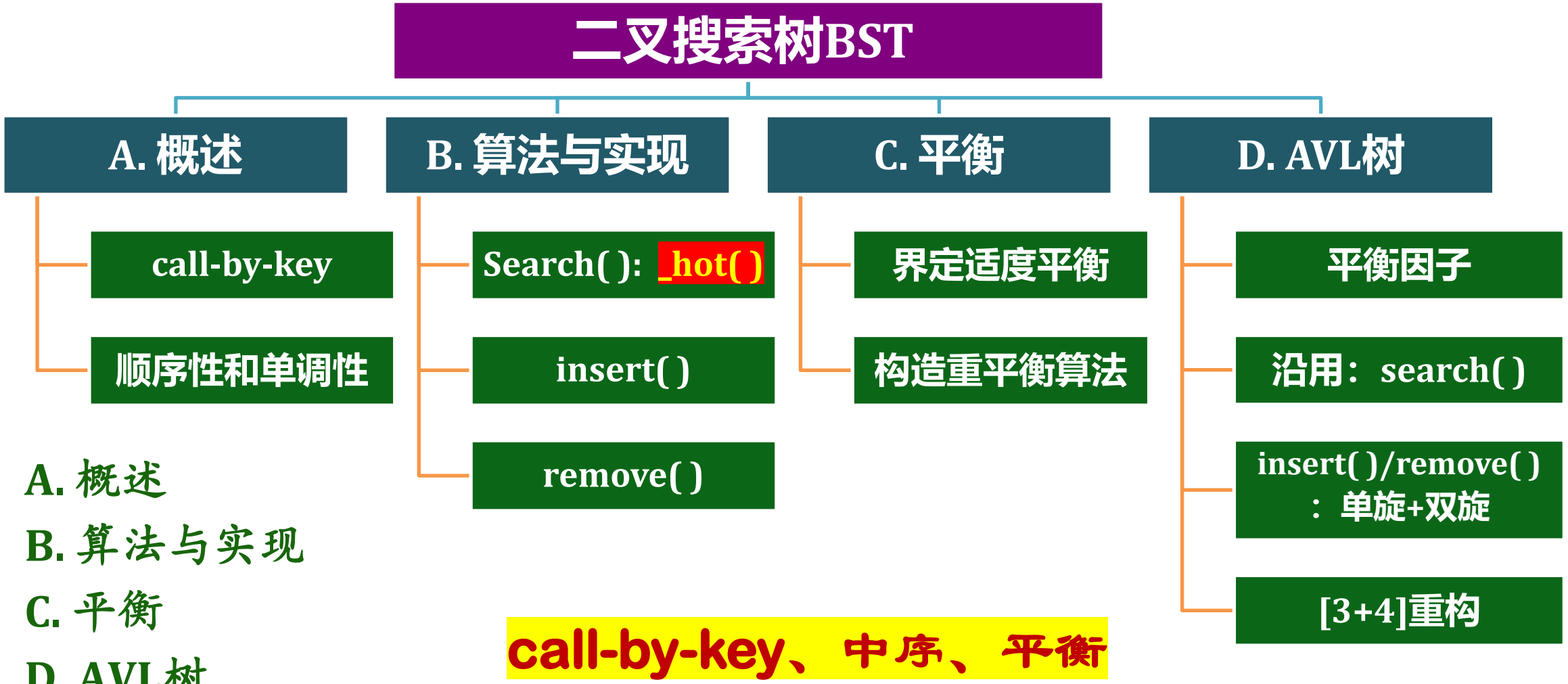
- 先序、中序、后序
- 层次、重构

D. Huffman 树

E. 下界与最优



06. Balanced Binary Search Tree



A. 概述

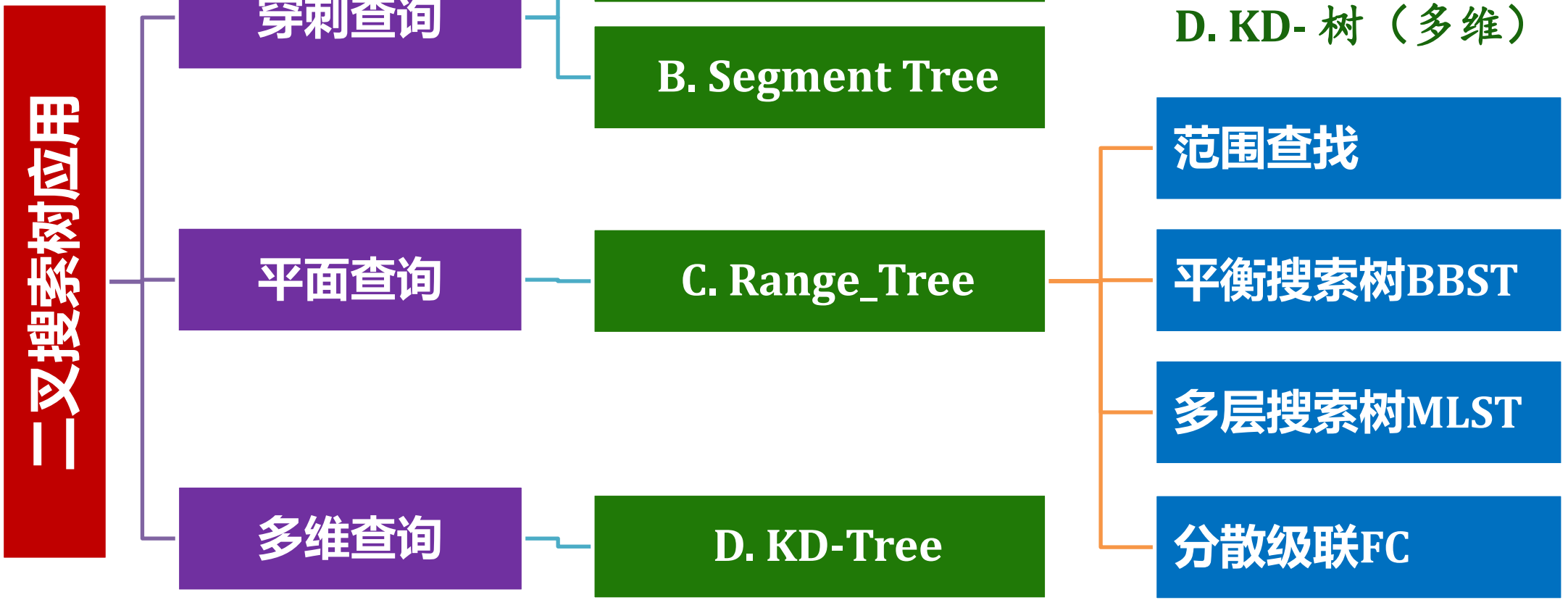
B. 算法与实现

C. 平衡

D. AVL树

call-by-key、中序、平衡

07. BST Application



A. 区间树（一维）
B. 线段树（一维）
C. 范围树（二维）
D. KD- 树（多维）

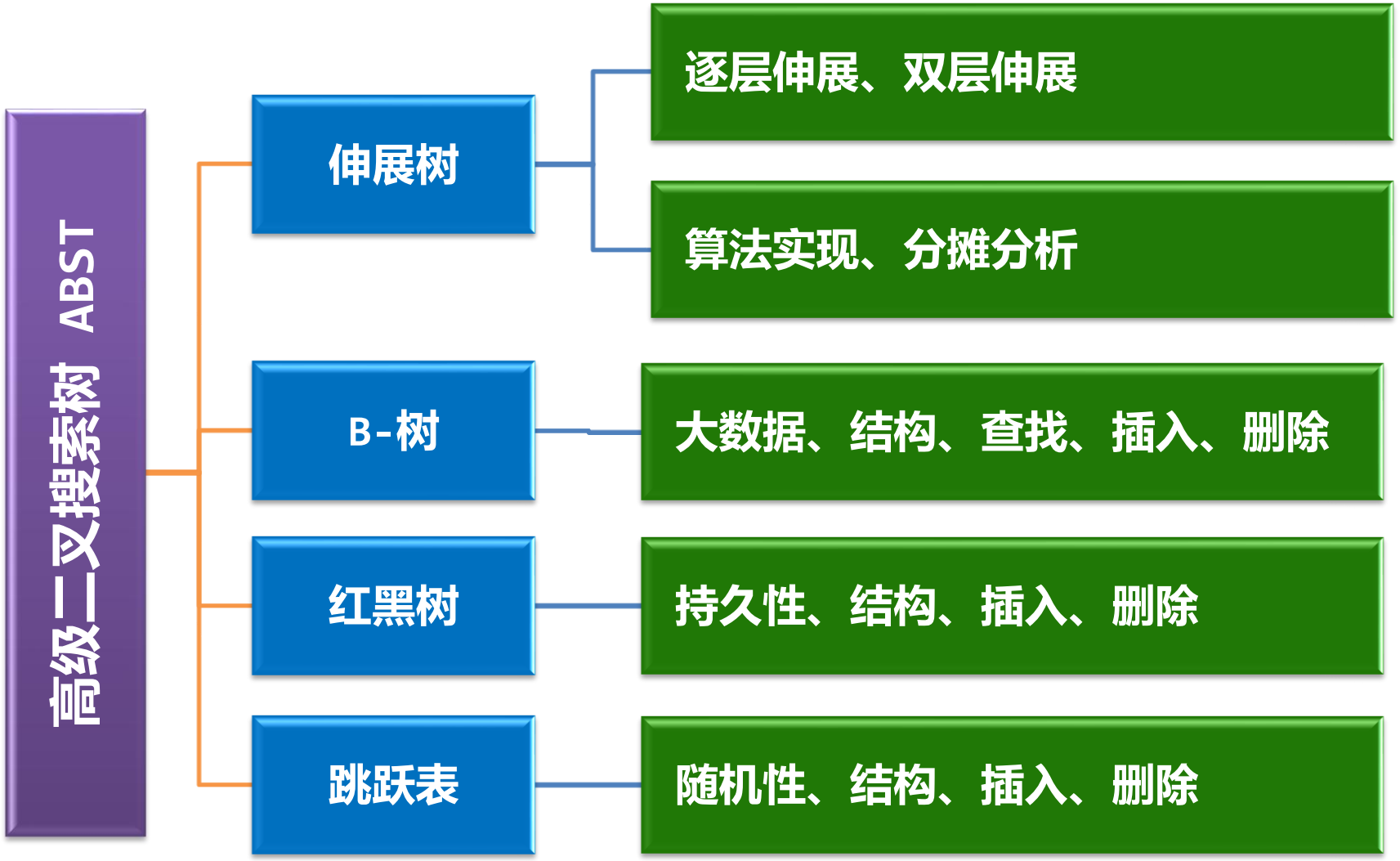
08. 高级二叉搜索树ABST

A. 伸展树
- 逐层伸展
- 双层伸展

B. B-树
- 大数据

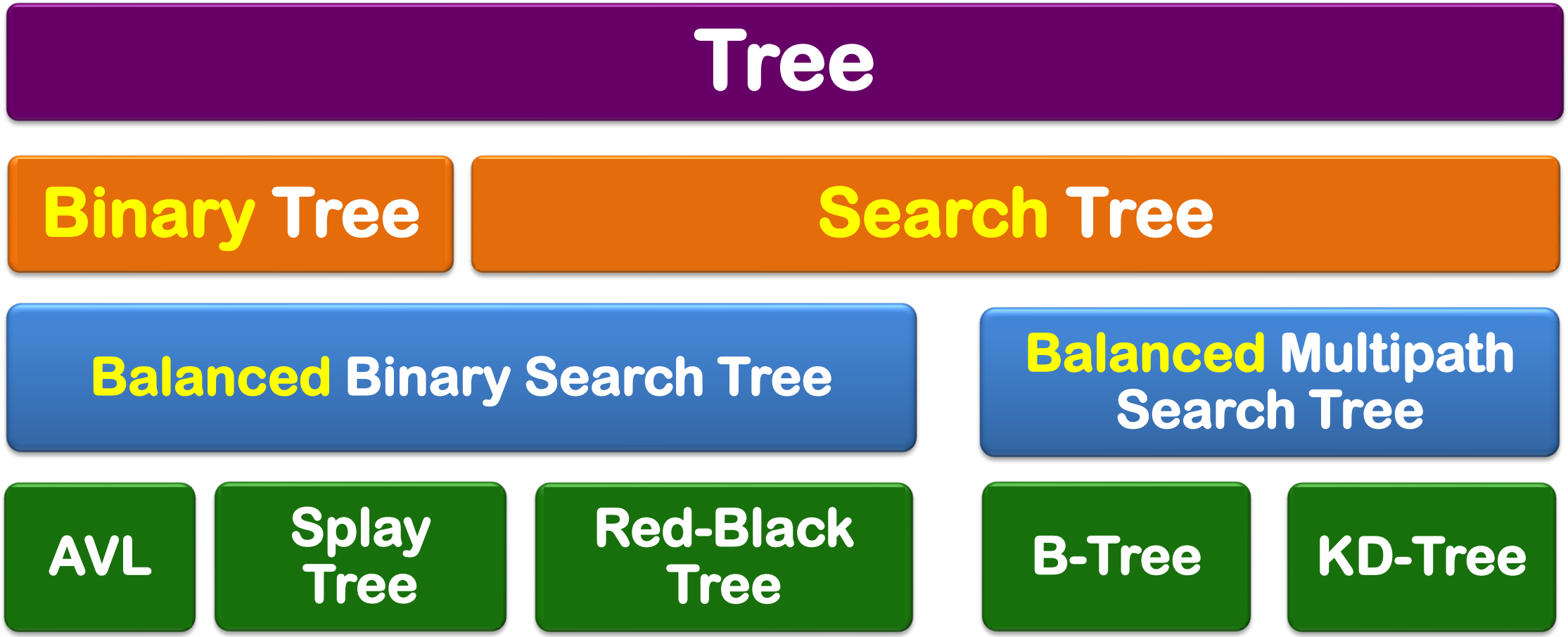
C. 红黑树
- 持久性

D. 跳跃表
- 随机性



（1）如何界定一种适度的平衡标准

（2）一整套重平衡的技巧和算法



05. Binary Tree

A. 树（术语和ADT）

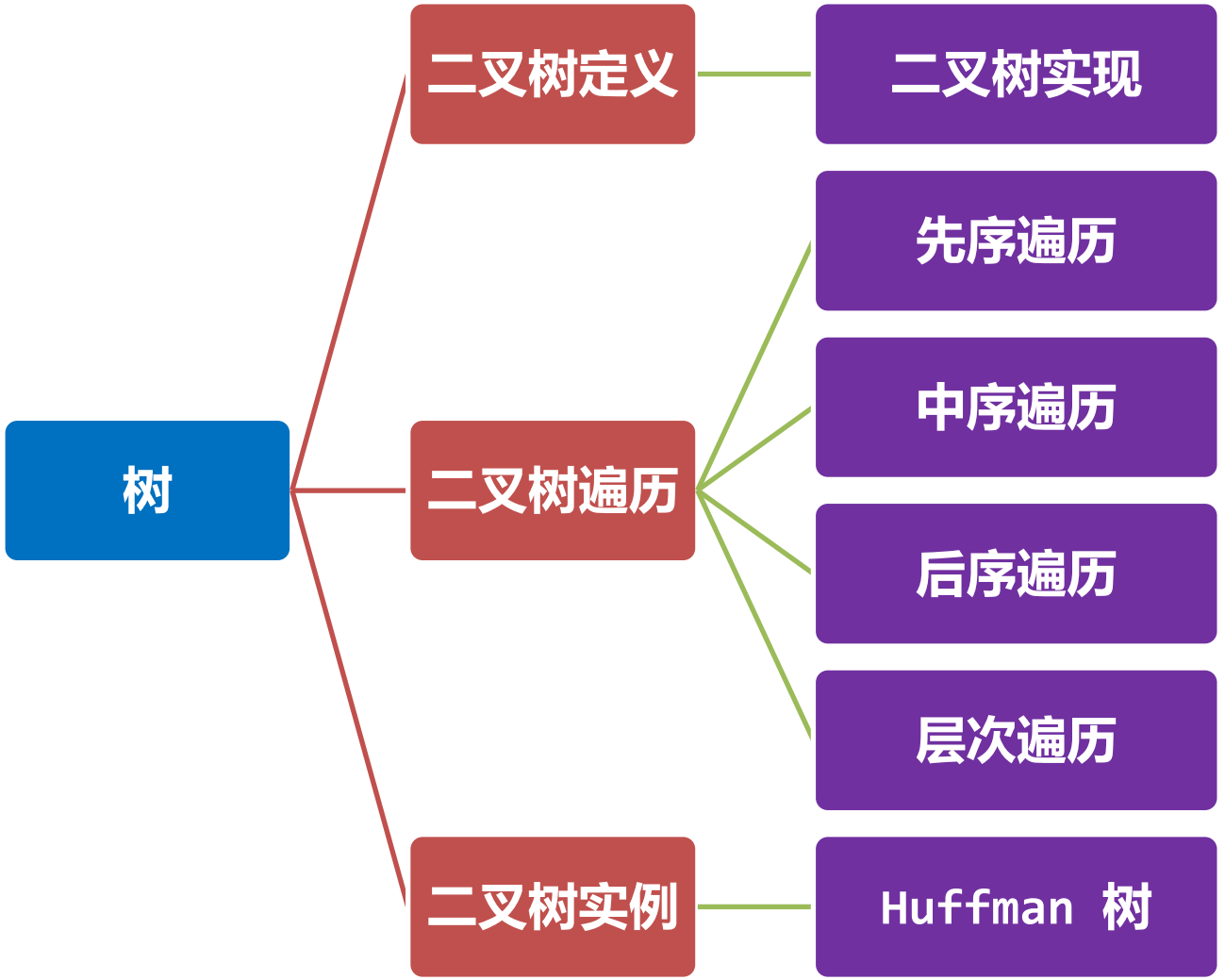
B. 二叉树（转换与实现）

C. 二叉树遍历

- 先序、中序、后序
- 层次、重构

D. Huffman 树

E. 下界与最优



05.Binary Tree

A-树

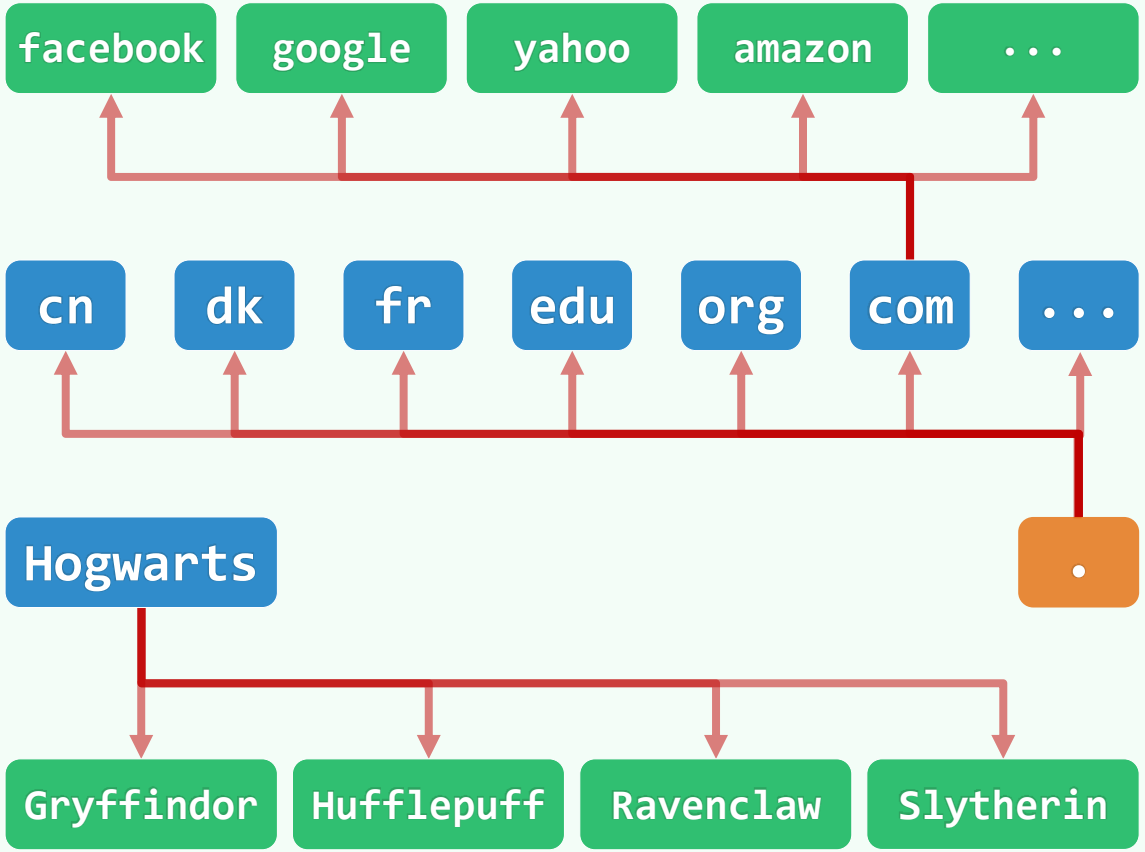
二叉树

树：术语

邓俊辉
deng@tsinghua.edu.cn

动机

- 【应用】层次结构的表示
  - 表达式
  - 文件系统
  - URL ...
- 【数据结构】综合性
  - 兼具Vector和List的优点
  - 兼顾高效的查找、插入、删除
- 【半线性】
  - 不再是简单的线性结构，但
  - 在确定某种次序之后，具有线性特征



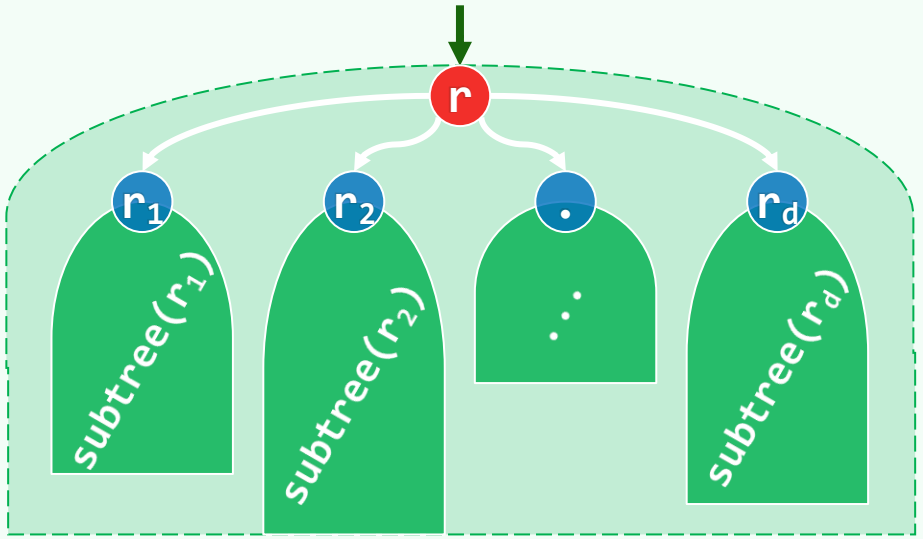
有根有序树

- 树也是图，由顶点（vertex）和联边（edge）构成

  顶点对应的数据结构为节点，不妨混称

- 指定某一节点为根后，即是有根树

  tree rooted at r = tree(r)



- 任给若干棵有根子树（subtree）$\{r_1, r_2, \ldots, r_d\}$

  引入新节点 r，并向 r_i 分别发出一条有向边，便可得到一棵（更大、更高的）有根树

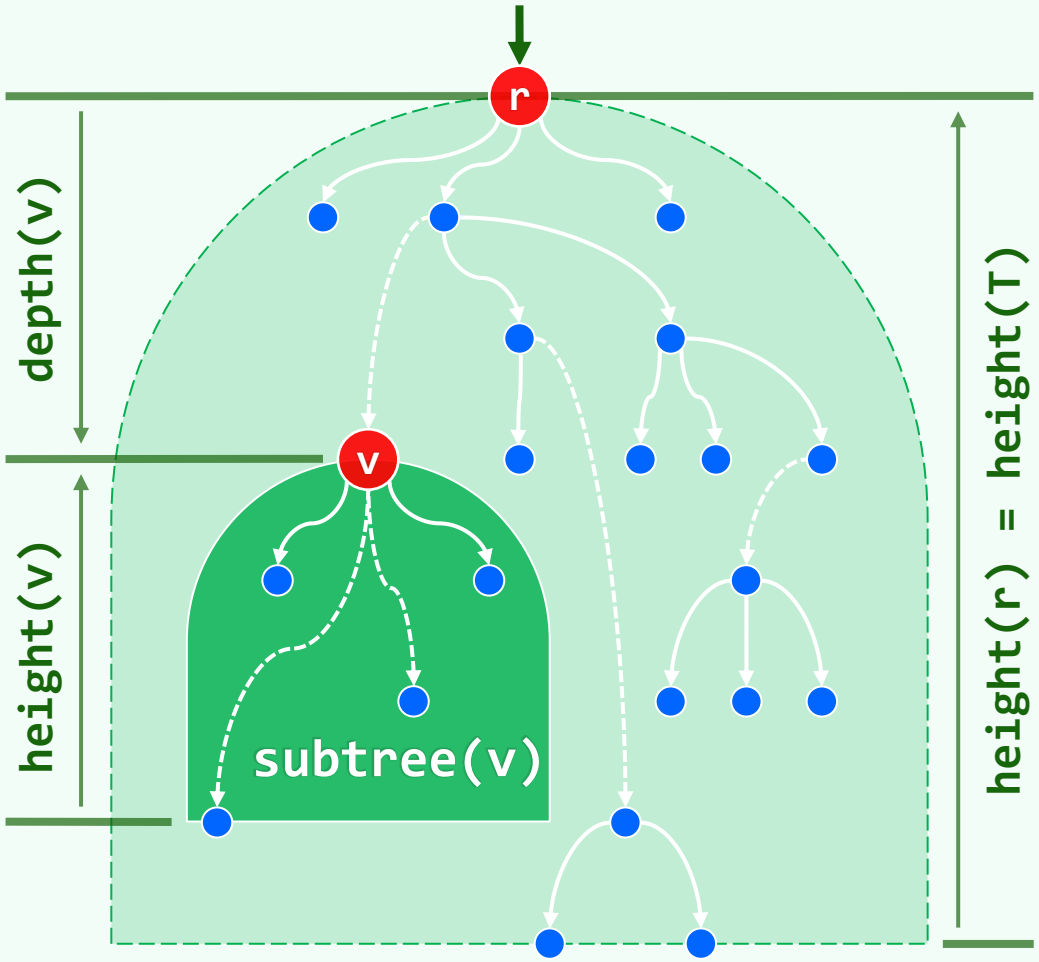
- r 和 r_i 互称父（parent）、子（child）；边数 d 为 r 的度（degree）

  零度节点称叶子（leaf），其余为内部节点（internal node）

- 兄弟（sibling）之间若依 $\{r_1, r_2, \ldots, r_d\}$ 排序，则是有序树（ordered tree）

深度 + 层次

- 从任一**祖先**（ancestor）r 出发，沿边逐代下溯
  可以抵达各级**后代**（descendant）v
- 沿途各边串接成一条**路径**，**边数**即路径的**长度**
- 若 r 为全树之根，则路径长度即 v 的**深度**（depth）
- 依照深度，可将节点划分为 $h+1$ 类；根自成一类
- 所有节点的**最大深度** h ，即树的**高度**（height）
  特别地，**空树**的高度为**-1**
- 节点的**高度** = 后代的最大深度 - 自己的深度
- $\forall v$ ，$depth(v) + height(v) \leq height(T) = height(r)$



二叉树

树：ADT及实现

邓俊辉
deng@tsinghua.edu.cn

Tree ADT

操作接口	功能
size()|empty()	报告树的规模|判空
root()	树根节点
x.parent()	x的父节点
x.firstChild()	x的长子
x.nextSibling()	x（按齿序最大的弟弟）
x.insert(i,e)	将e作为x的第i个孩子插入
x.remove(i)	删除x的第i个孩子（及其后代）
traverse(visit())	遍历全树，统一按visit()处理所有节点

父节点

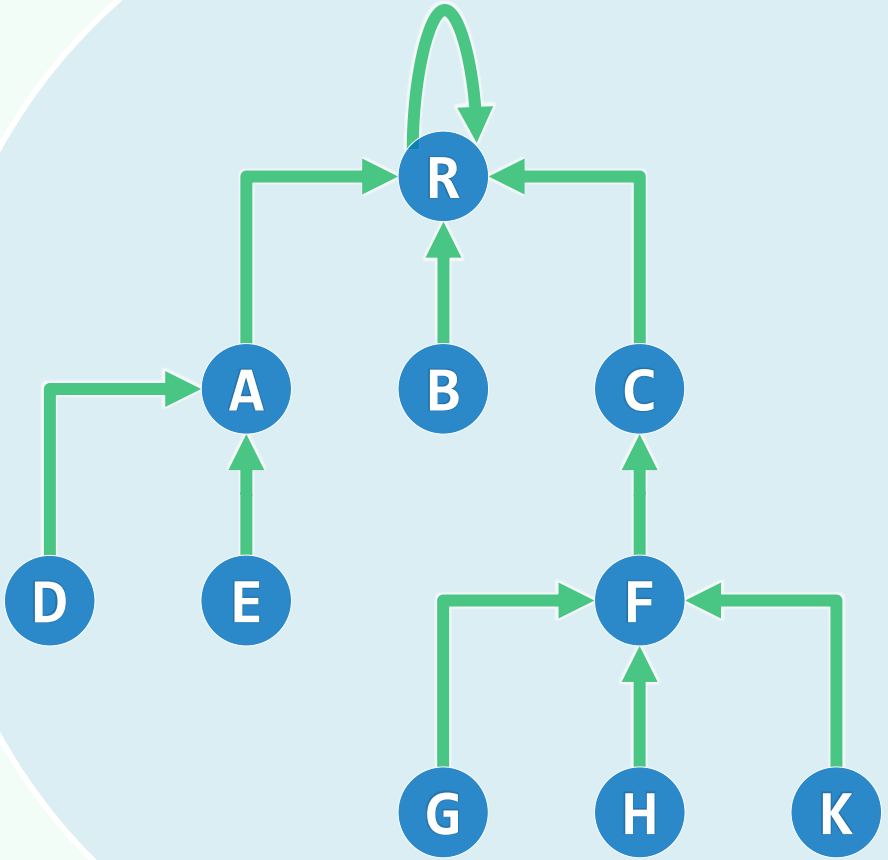
rank	parent	data
0	2	H
1	7	E
2	9	F
3	4	B
4	4	R
5	2	K
6	7	D
7	4	A
8	2	G
9	4	C

- 除根外，任一节点有且仅有一个父节点
- 节点组织为一个序列，各自记录：

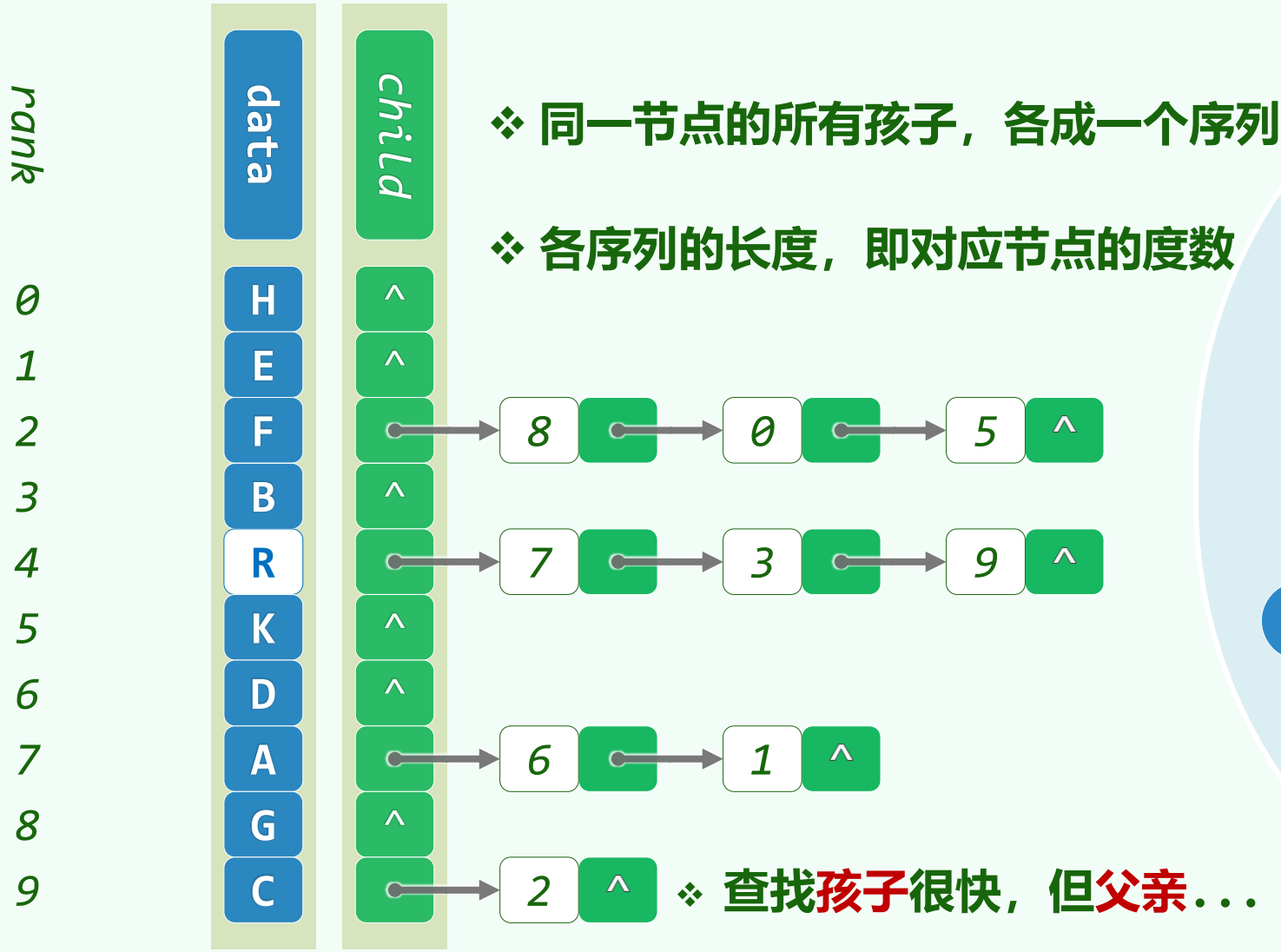
  data 本身信息

  parent 父节点的秩或位置

- 树根：R ~ parent(4) = 4
- 查找父亲很快，但孩子...



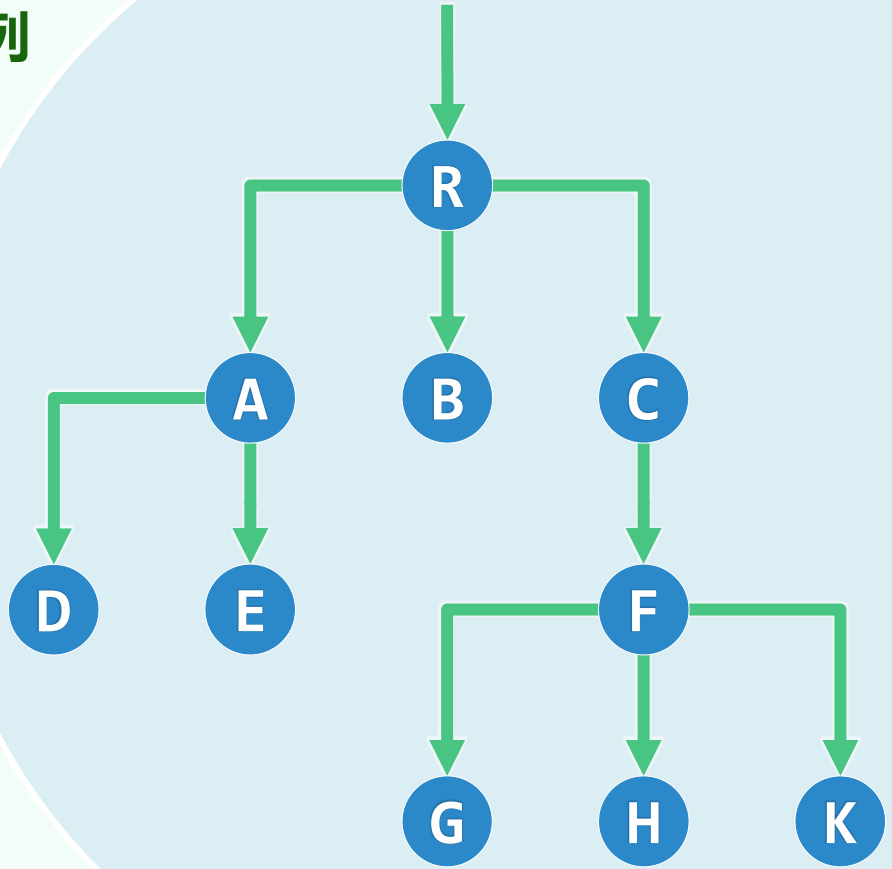
孩子节点



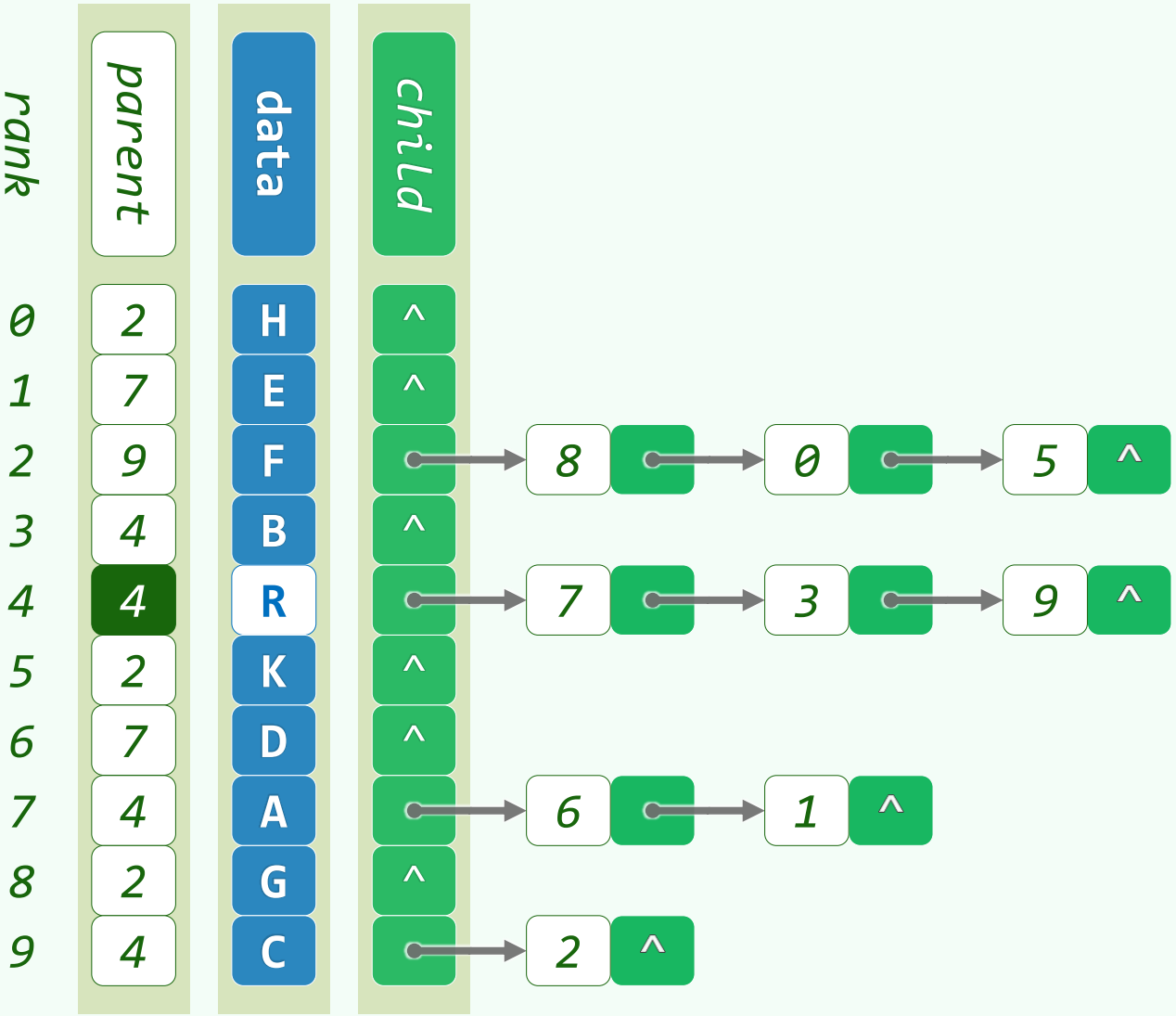
❖ 同一节点的所有孩子，各成一个序列

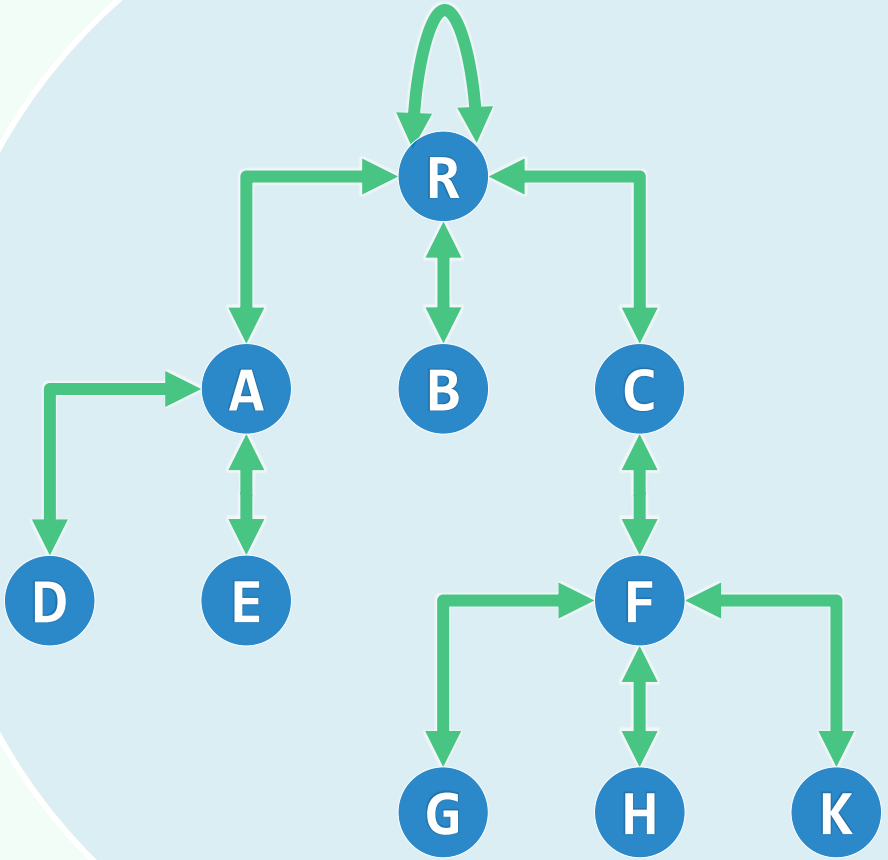
❖ 各序列的长度，即对应节点的度数

❖ 查找孩子很快，但父亲...



父节点 + 孩子节点


rank
parent
data
child
0 2 H ^
1 7 E ^
2 9 F 8 0 5 ^
3 4 B ^
4 4 R 7 3 9 ^
5 2 K ^
6 7 D ^
7 4 A 6 1 ^
8 2 G ^
9 4 C 2 ^


R
A
B
C
D
E
F
G
H
K

05. Binary Tree

A. 树（术语和ADT）

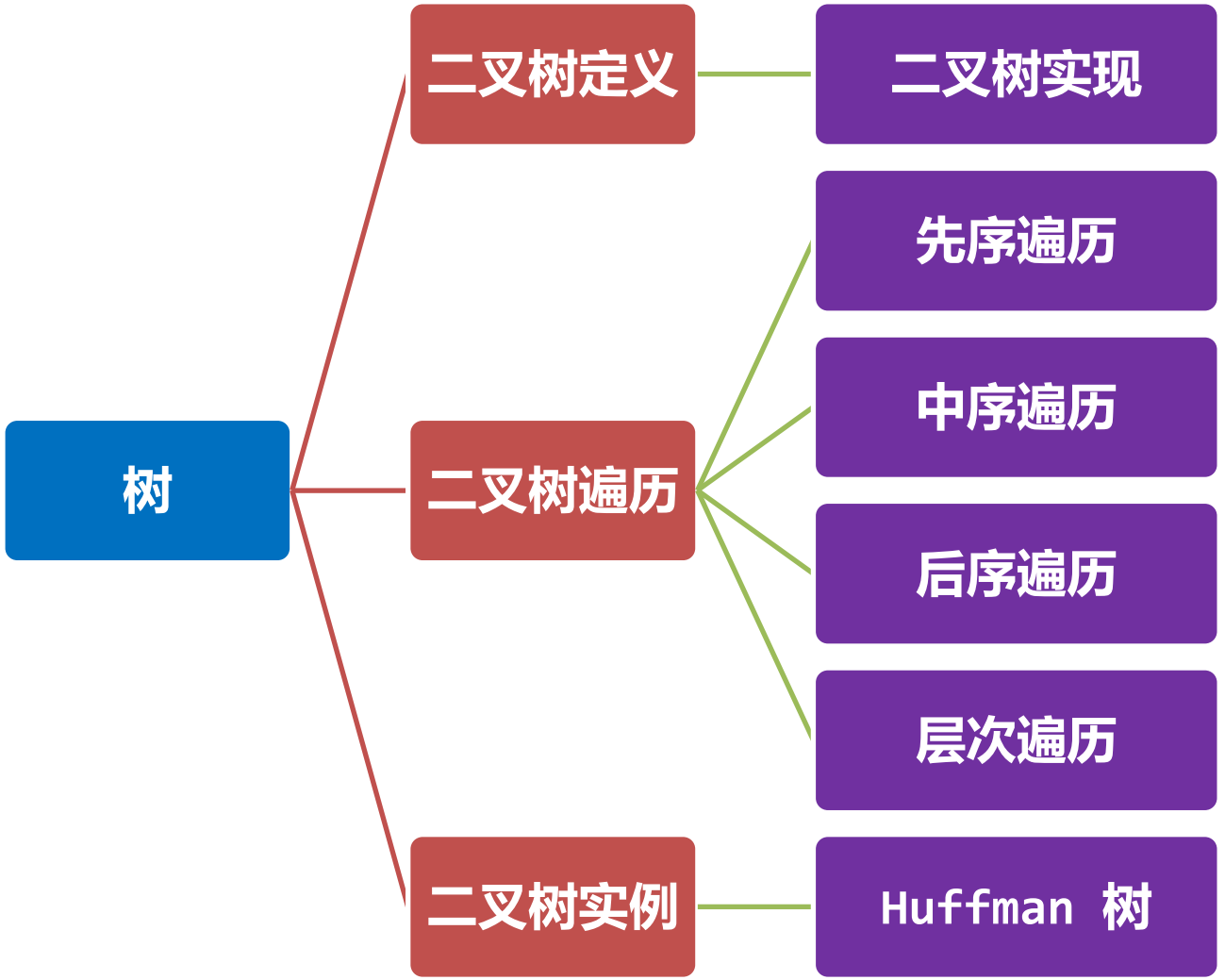
B. 二叉树（转换与实现）

C. 二叉树遍历

- 先序、中序、后序
- 层次、重构

D. Huffman 树

E. 下界与最优



05.Binary Tree

B-二叉树

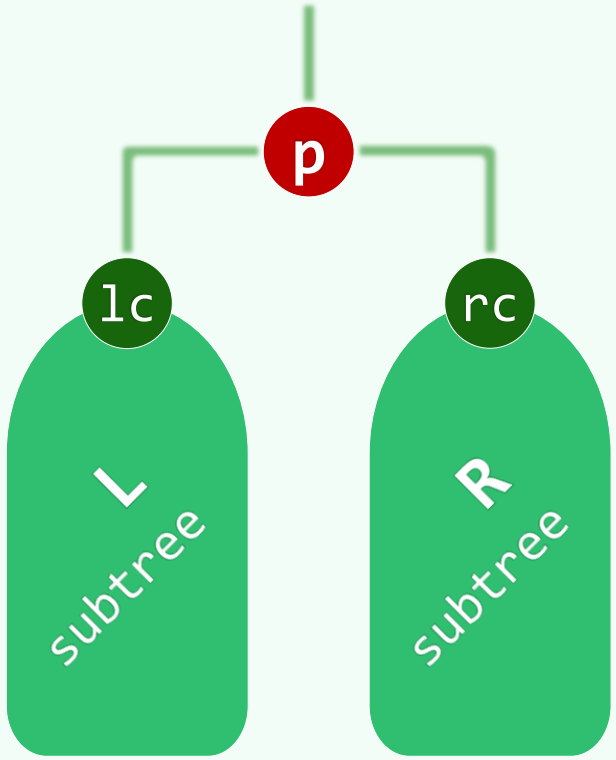
二叉树

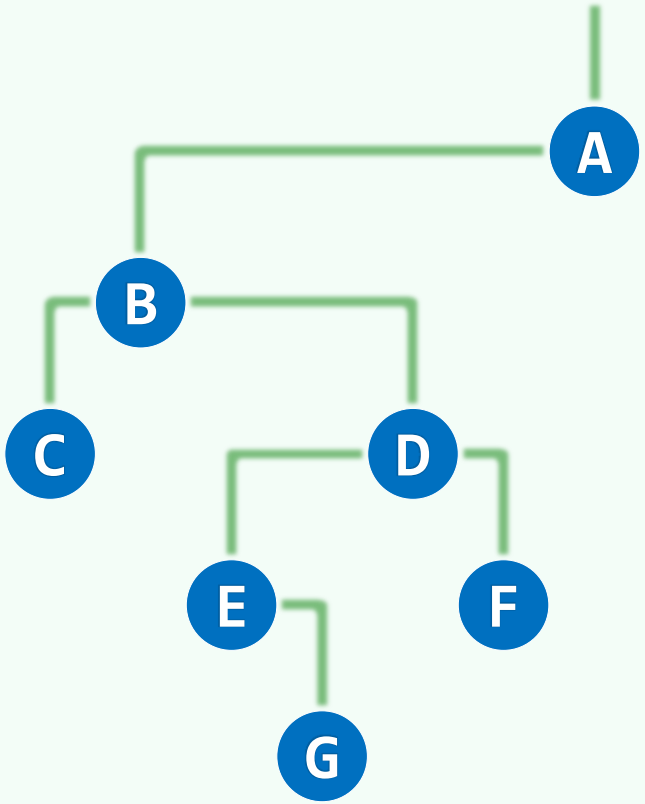
二叉树：性质与转换

邓俊辉
deng@tsinghua.edu.cn

二叉树

- Binary Tree：节点度数均不超过2
- p称lc、rc为左孩子/子树、右孩子/子树
- lc、rc称p为左父亲、右父亲
- 推而广之：左祖先、右祖先





基数：设度数为0、1和2的节点，各有n_0、n_1和n_2个

❖ 边数 $e = n-1 = n_1+2\cdot n_2$

1、2度节点，各贡献1、2条边

❖ 叶节点数 $n_0 = n_2+1$

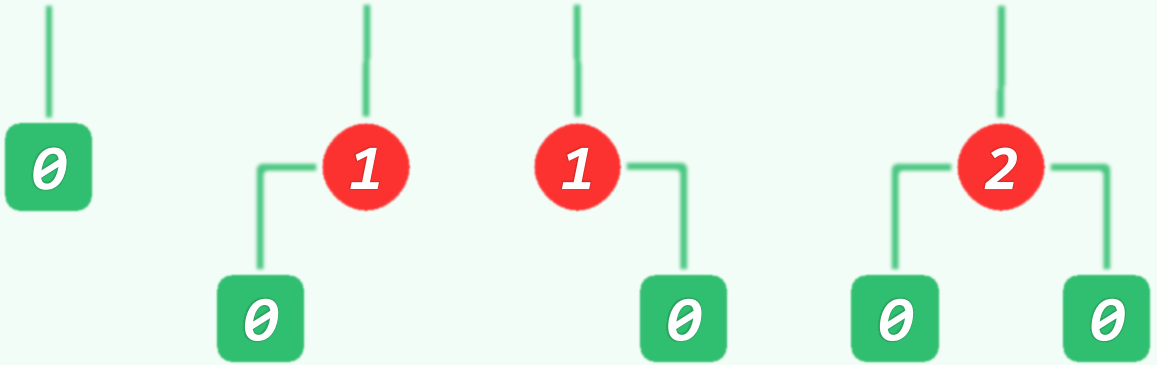
与 n_1 无关

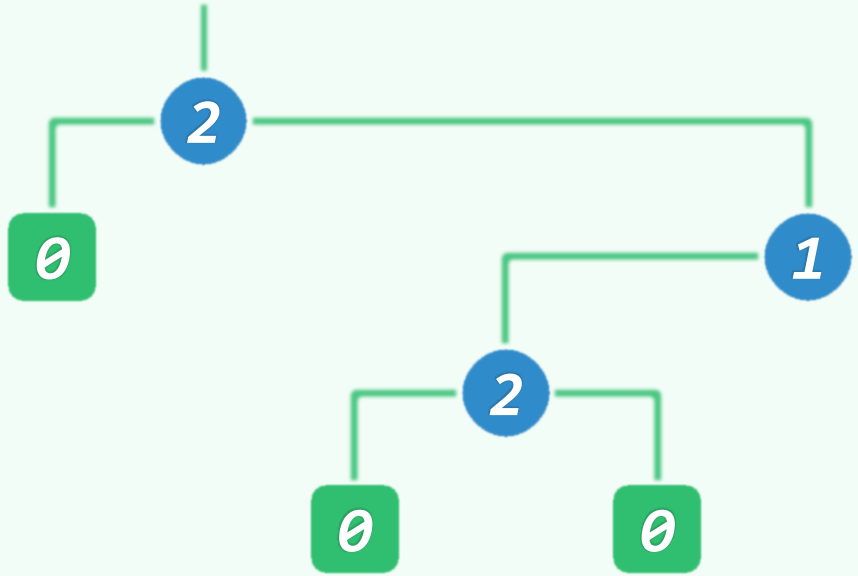
❖ 节点总数 $n = n_0+n_1+n_2 = 1+n_1+2\cdot n_2$

❖ 特别地，当 $n_1=0$ 时为真二叉树，有

$e = 2\cdot n_2$ 和 $n_0 = 1+n_2 = (n+1)/2$

此时，节点度数均为偶数，不含单分支节点...





真二叉树

- 通过引入 $n_1 + 2 \cdot n_0$ 个外部节点

  可使原有节点度数统一为2

- 如此，每棵二叉树都可转化为

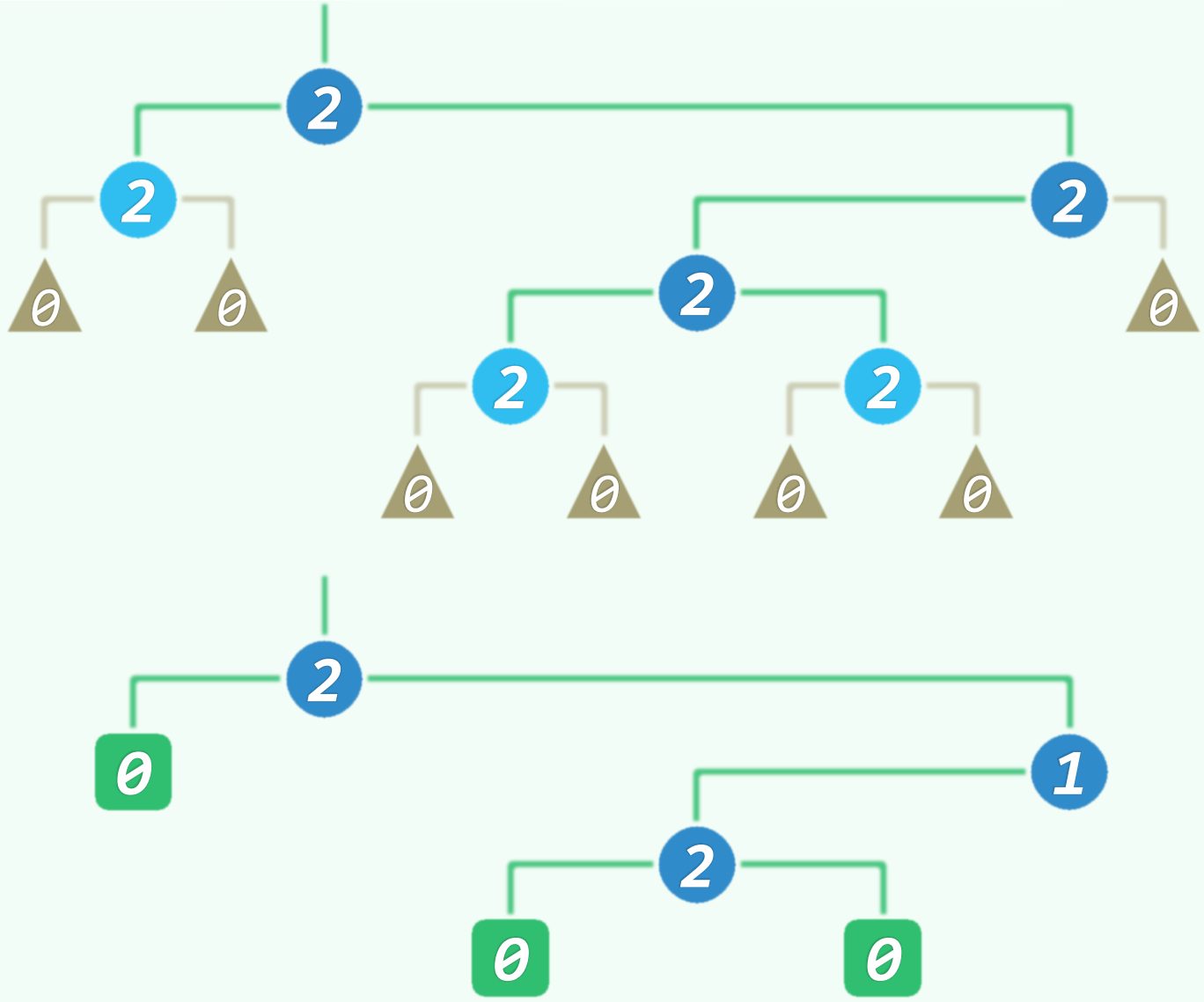
  真二叉树|proper binary tree

- 如此转换之后

  全树自身的复杂度并未实质增加

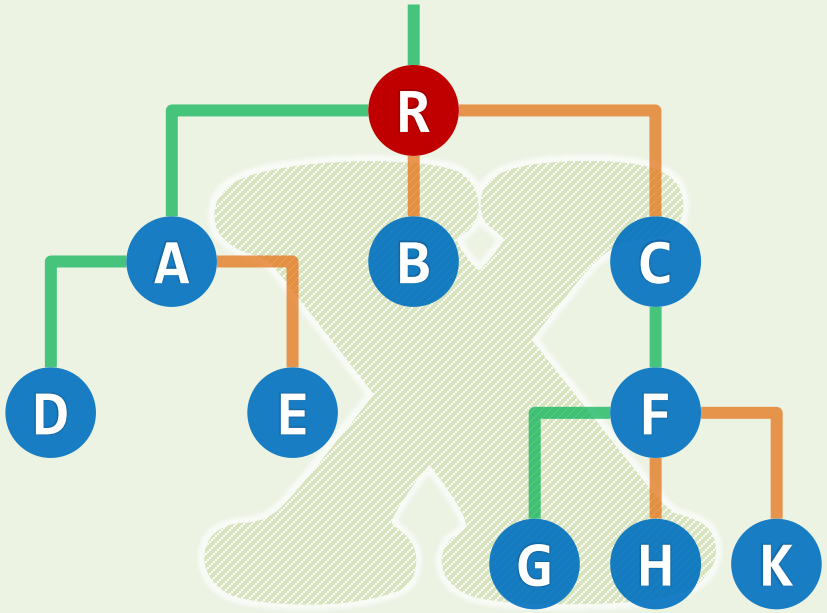
- 红黑树之类的结构经如此转换之后

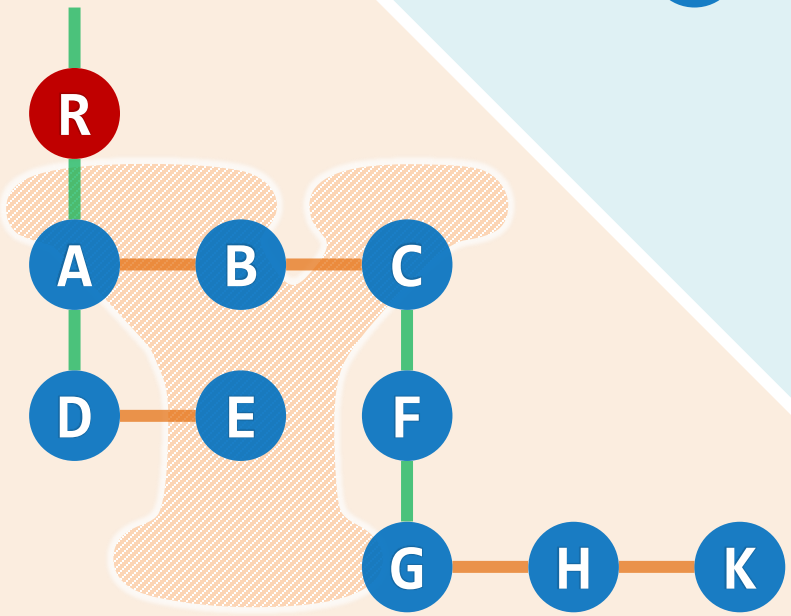
  可以简化描述、理解、实现和分析

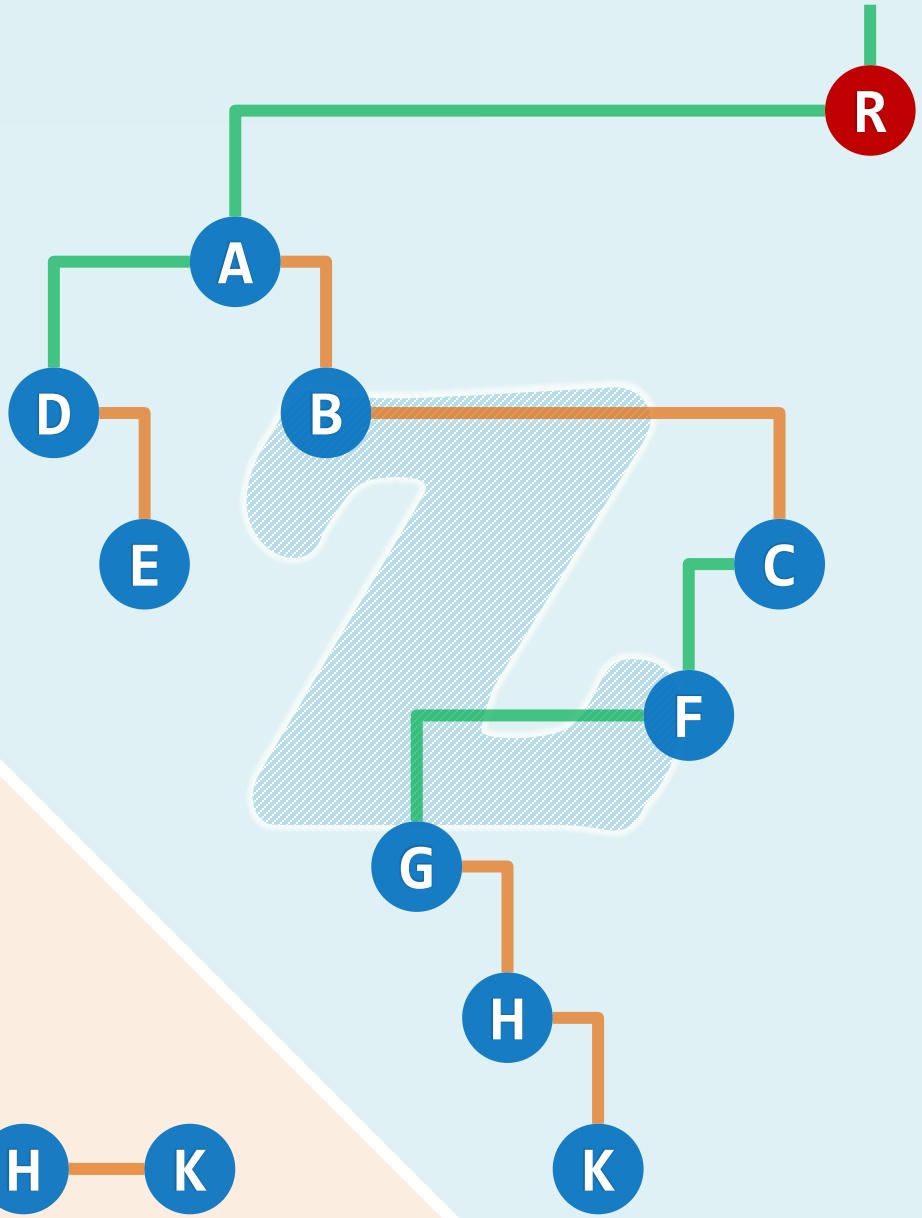


描述多叉树：“长子-弟弟”表示法

- 有根且有序的多叉树，均可转化并表示为二叉树
- 长子 ~ 左孩子　　firstChild() ~ lc()
  弟弟 ~ 右孩子　　nextSibling() ~ rc()







满树

❖ 深度为 k 的节点，至多 2^k 个

❖ 节点总数 n 与高度 h 必满足

$$h+1 \le n \le 2^{h+1}-1$$

❖ 特殊情况

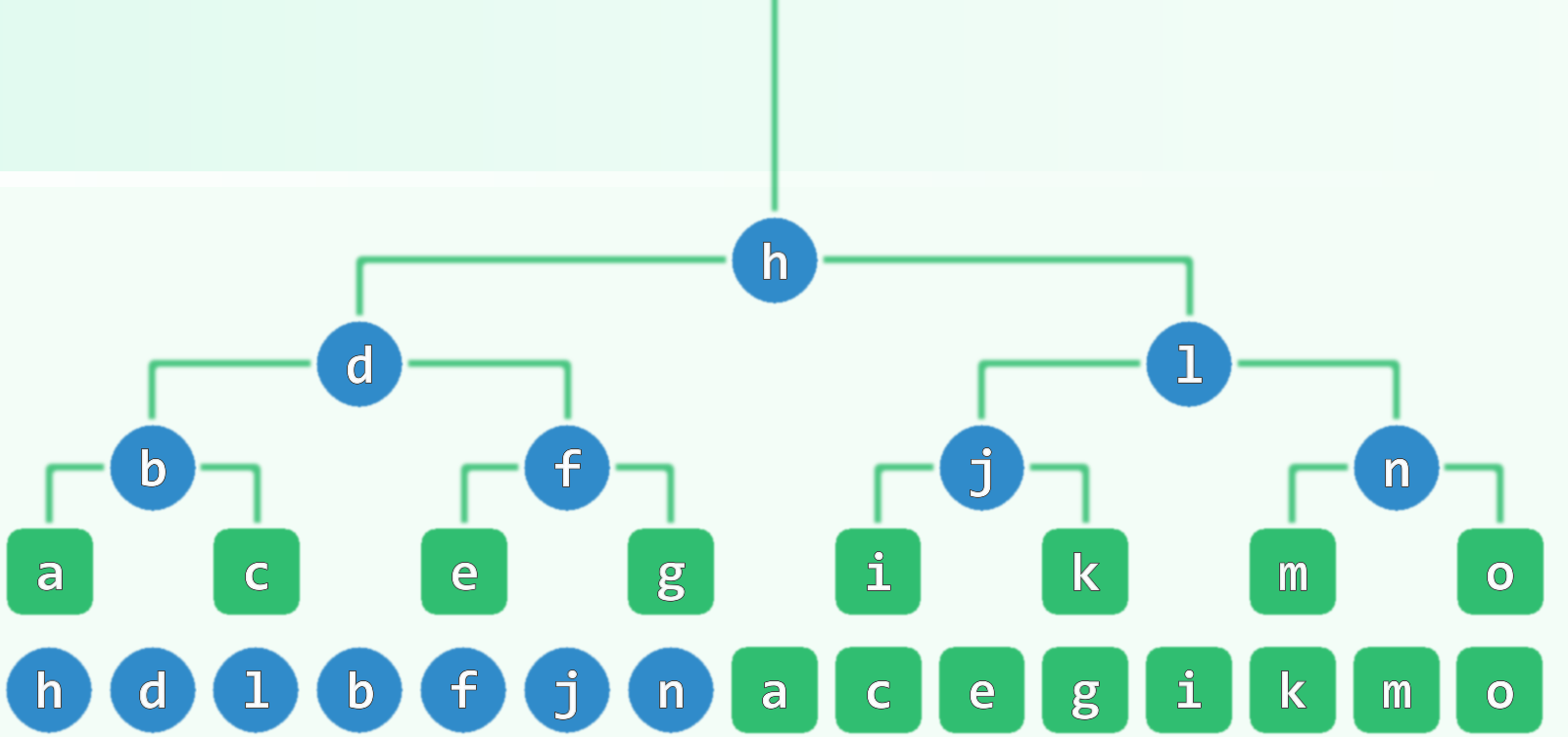
- $n = h+1$

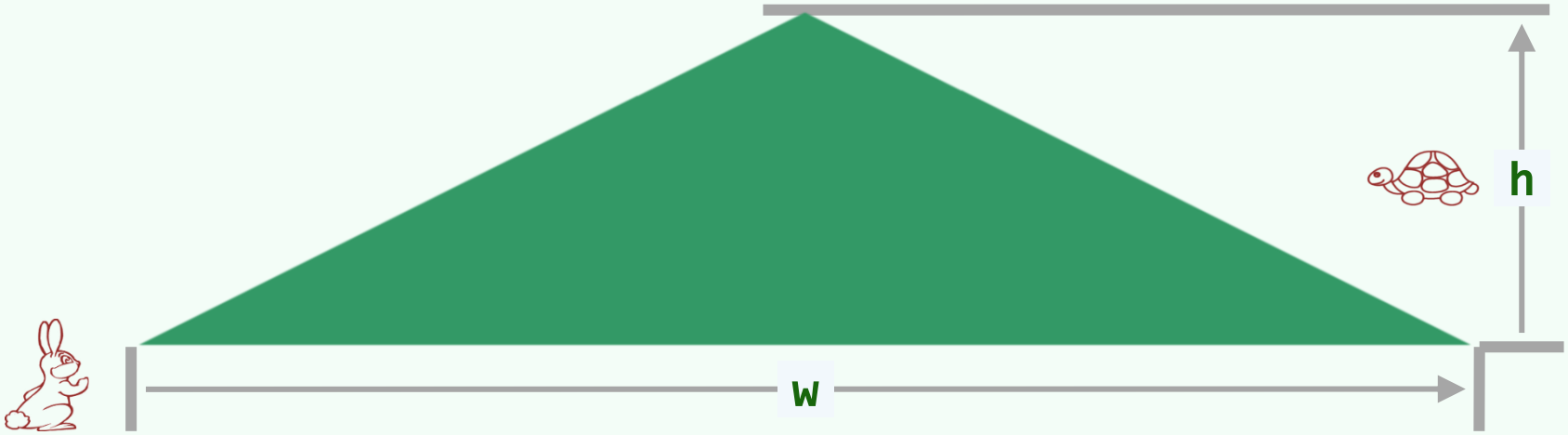
  退化为一条单链

- $n+1 = 2^{h+1}$

  满二叉树

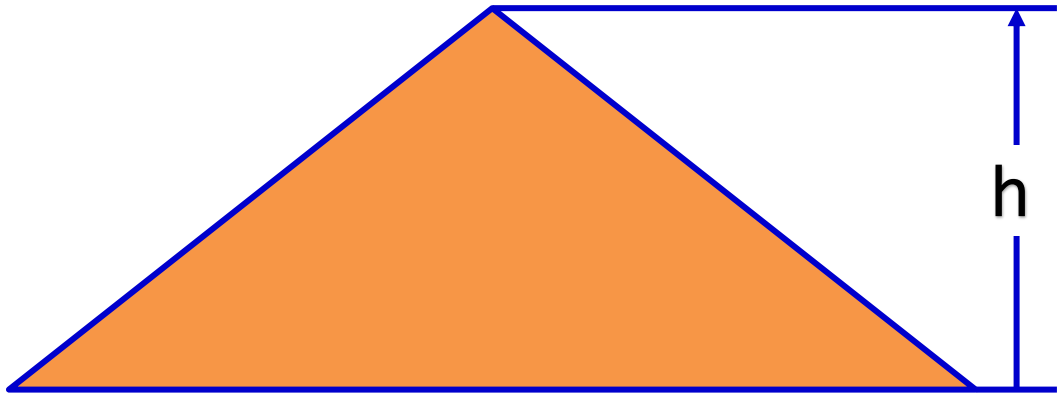
  full binary tree

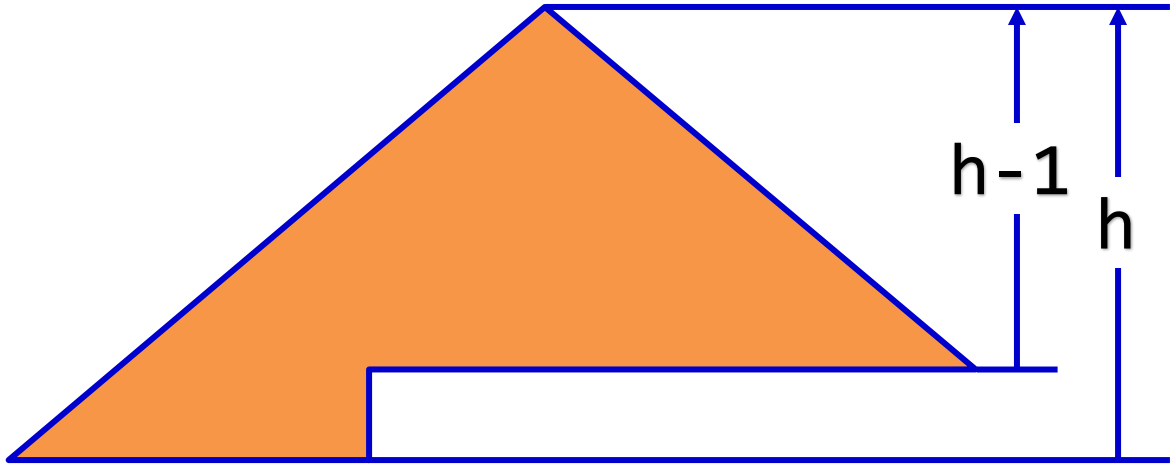




真二叉树、完全二叉树和满二叉树

- 二叉树如果不含有度数为1的节点，即为真二叉树（proper BT）
- 如果所有的叶子节点都出现在最底部的两层，而且最底层节点均处于次底层叶子节点的左侧，即为完全二叉树（complete BT）
- 如果每层节点数都到达最大，即$n = 2^{h+1} - 1$时，即为满二叉树（full BT）





二叉树

二叉树：实现

邓俊辉
deng@tsinghua.edu.cn

BinNode模板类

```
template <typename T> using BNP = BinNode<T>*; //Binary Node Position

template <typename T> struct BinNode {
   BNP<T> parent, lc, rc; //父亲、孩子
   T data; int height; RBColor color; //数据、（黑）高度/npl、颜色

   int updateHeight(); void updateHeightAbove(); //更新规模、高度

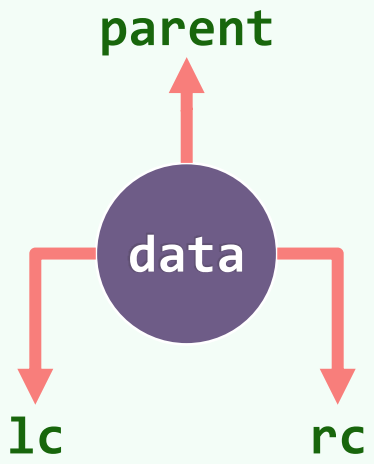
   BNP<T> insertLc( const T& ); //插入左孩子
   BNP<T> insertRc( const T& ); //插入右孩子

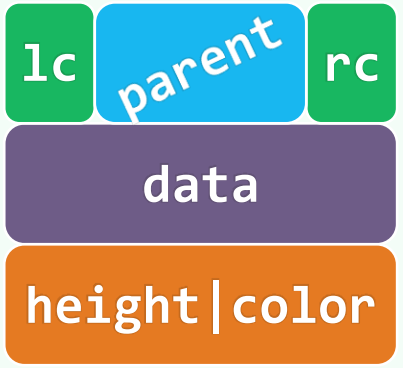
   void attachLc( BNP<T> ); //插入左子树
   void attachRc( BNP<T> ); //插入右子树

   BNP<T> succ(); //（中序遍历意义下）当前节点的直接后继

   template <typename V> void traverse( V& ); //遍历

}; //BinNode
```





节点的插入与接入

```
template <typename T>

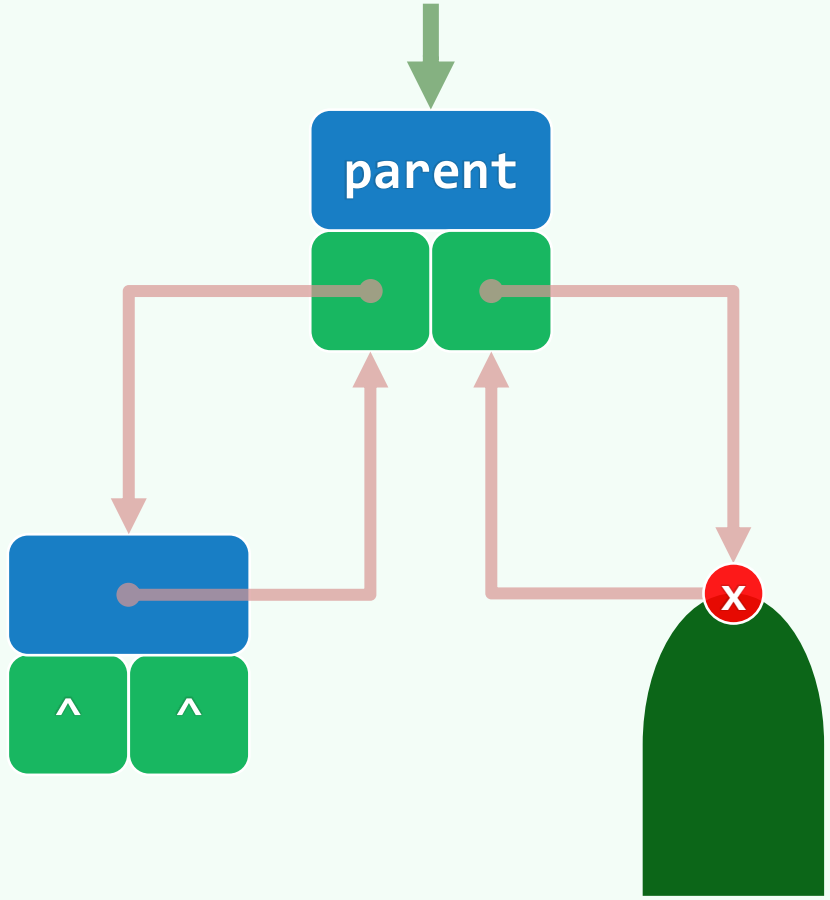
BNP<T> BinNode<T>::insertLc( const T& e )

   { return lc = new BinNode<T>( e, this ); }


template <typename T>

void BinNode<T>::attachRc( BNP<T> x )

   { rc = x; if ( x ) x->parent = this; }
```



高度的更新

```
❖ auto Height = [](auto x) //简化描述
     { return x ? x->height : -1 ); } //空树高度为-1

❖ template <typename T> int BinNode<T>::updateHeight() //O(1)
     { return height = 1 + max( Height(lc), Height(rc) ); }

❖ template <typename T> void BinNode<T>::updateHeightAbove() { //O(n = depth(x))
     for ( BNP<T> x = this; x; x = x->parent ) //可优化
        x->updateHeight();
  } //updateHeightAbove
```

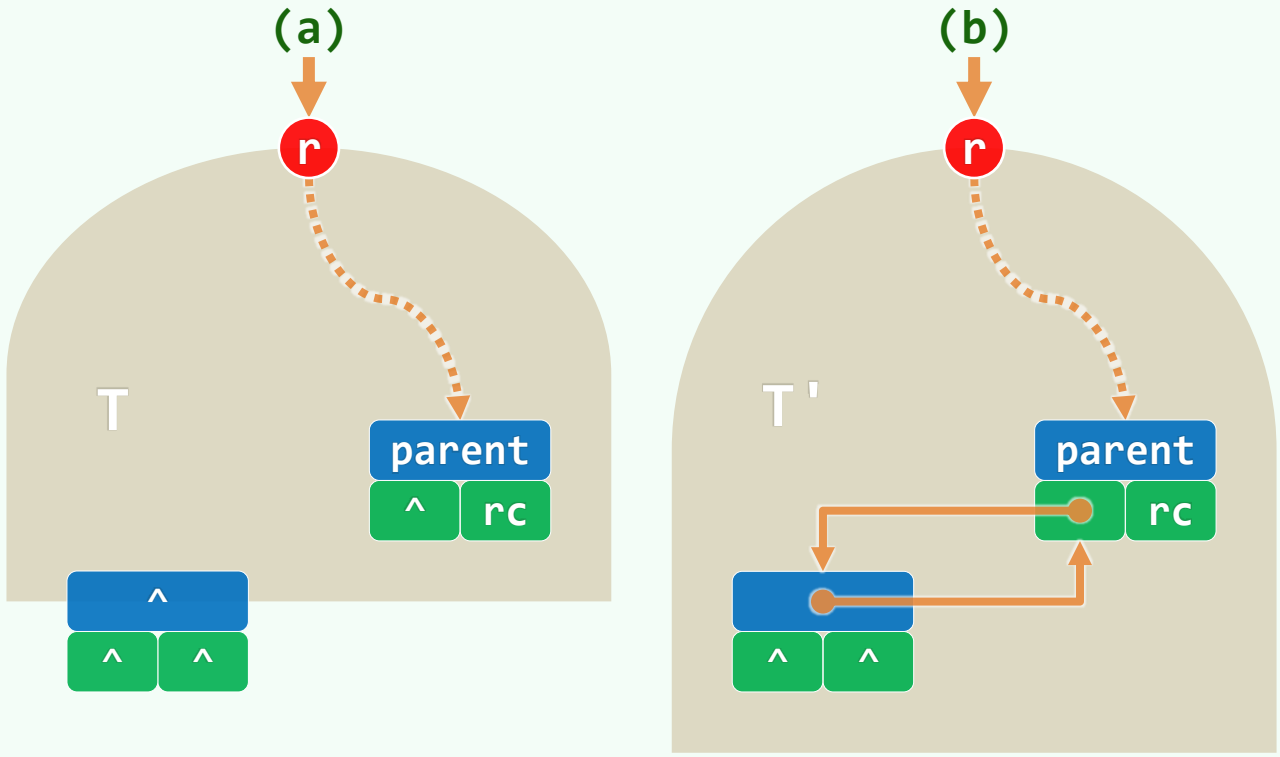
BinTree模板类

```
template <typename T> class BinTree {
   protected: Rank _size; BNP<T> _root;
   public: BNP<T> insert( const T& ); //插入根节点
           BNP<T> insert( const T&, BNP<T> ); //插入左孩子
           BNP<T> insert( BNP<T>, const T& ); //插入右孩子
           BNP<T> attach( BinTree<T>, BNP<T> ); //接入左子树
           BNP<T> attach( BNP<T>, BinTree<T> ); //接入右子树
           BinTree<T> secede( BNP<T> ); //子树分离
           Rank remove( BNP<T> ); //子树删除：需借助后序遍历，稍后再做讲解
} //BinTree
```

节点的插入

❖ template <typename T> BNP<T> BinTree<T>::insert(const T& e, BNP<T> x) { //左孩子

```
    _size++;
    x->insertLc( e );
    x->updateHeightAbove();
    return x->lc;
} //insert
```

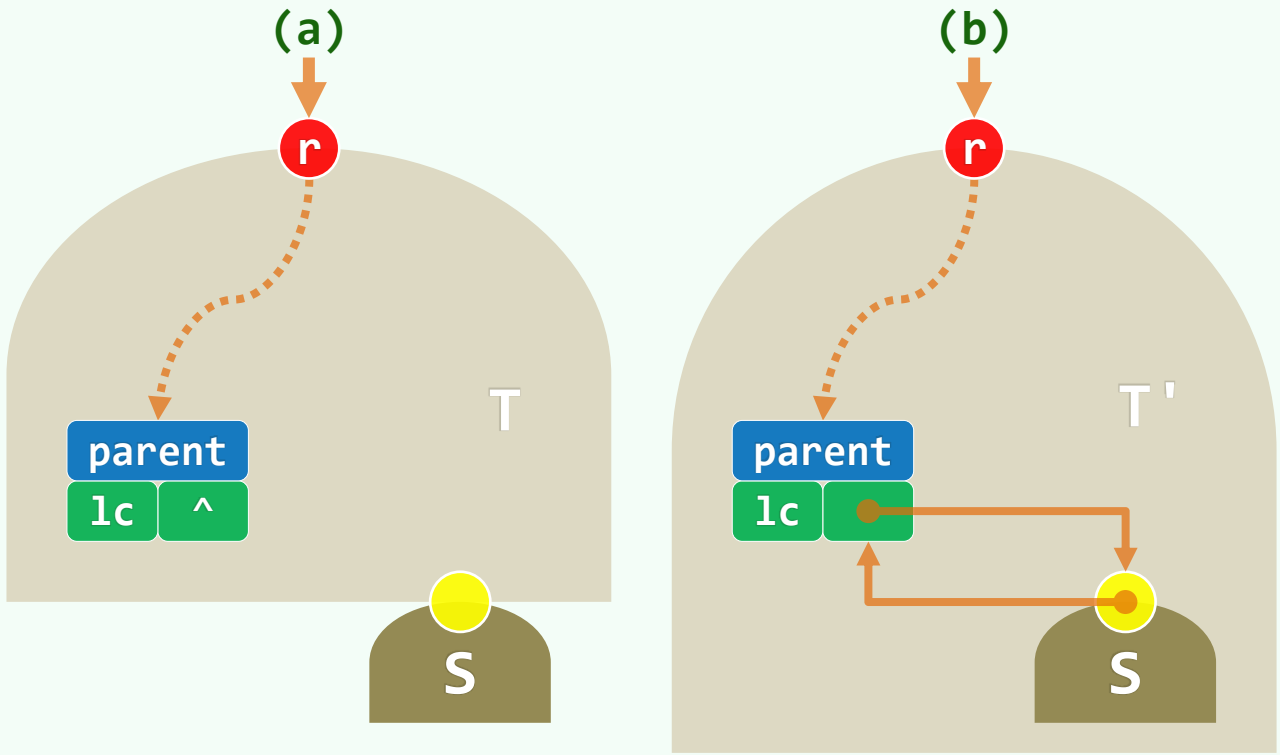


❖ template <typename T> BNP<T> BinTree<T>::insert(BNP<T> x, const T& e); //右孩子

子树的接入

❖ template <typename T> BNP<T> BinTree<T>::attach(BNP<T> x, BinTree<T> S) { //右

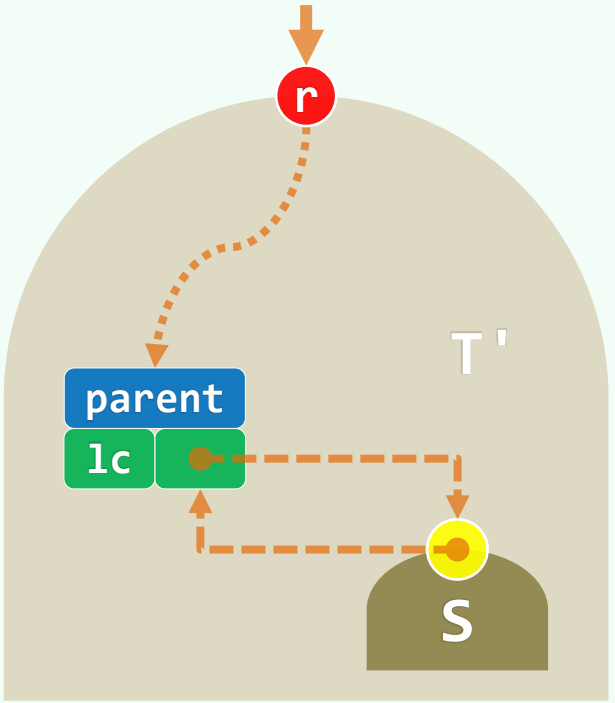
```
    if ( ! ( x->rc = S._root ) ) return nullptr;   //空树无需费事
    x->rc->parent = x;
    _size += S._size;
    x->updateHeightAbove();
    S._root = nullptr; S._size = 0;
    return x;
} //attach
```



❖ template <typename T> BNP<T> BinTree<T>::attach(BinTree<T> S, BNP<T> x); //左

子树的分离

```
❖ template <typename T> BinTree<T> BinTree<T>::secede( BNP<T> x ) {
    FromParentTo( x ) = nullptr;
    if ( x->parent ) x->parent->updateHeightAbove();
    BinTree<T> S; //创建空树
    S._root = x; x->parent = nullptr; //新树以x为根
    S._size = x->size(); _size -= S._size; //更新规模
    return S; //返回封装后的子树
  } //secede
```



05. Binary Tree

A. 树（术语和ADT）

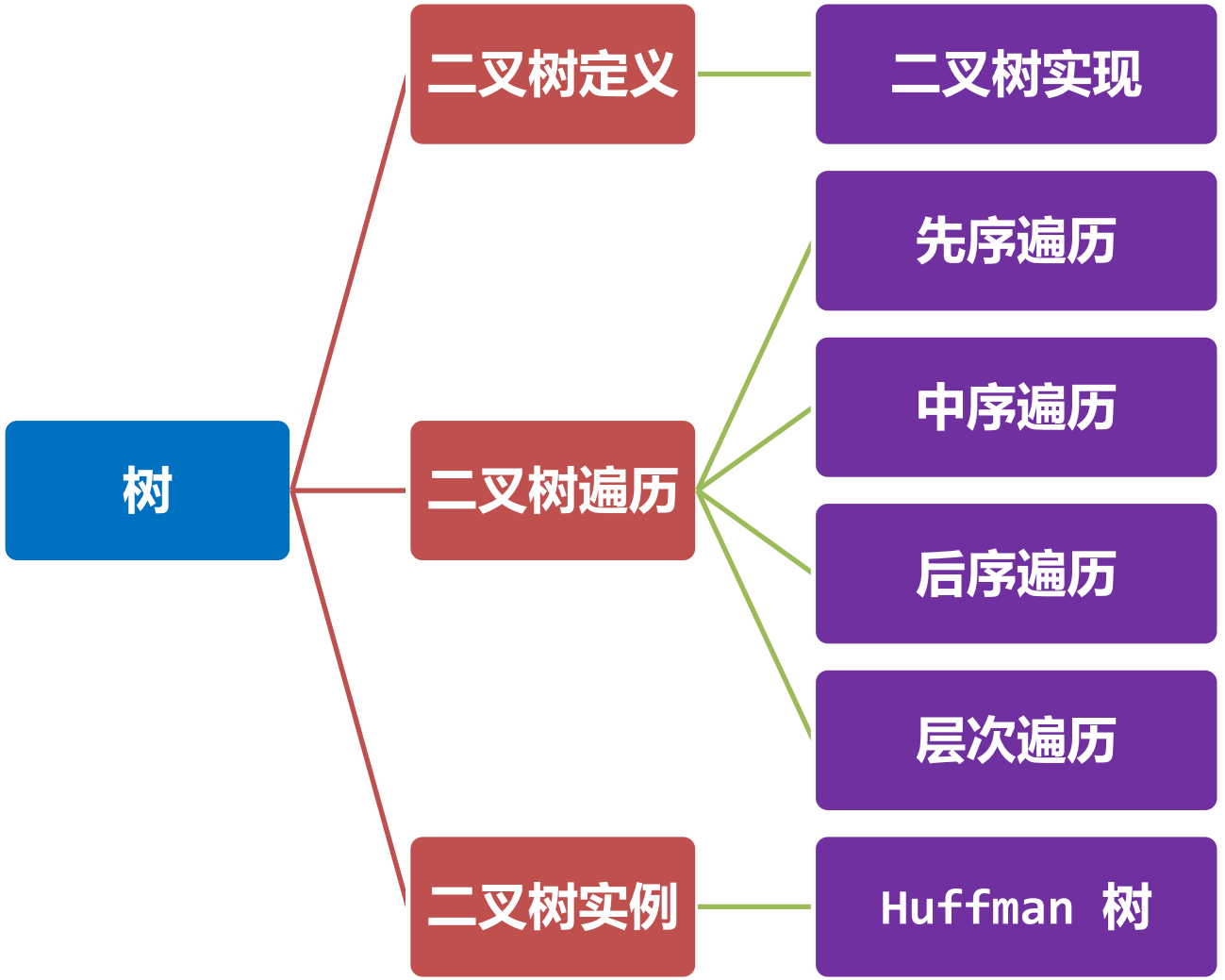
B. 二叉树（转换与实现）

C. 二叉树遍历

- 先序、中序、后序
- 层次、重构

D. Huffman 树

E. 下界与最优



05.Binary Tree

C-遍历

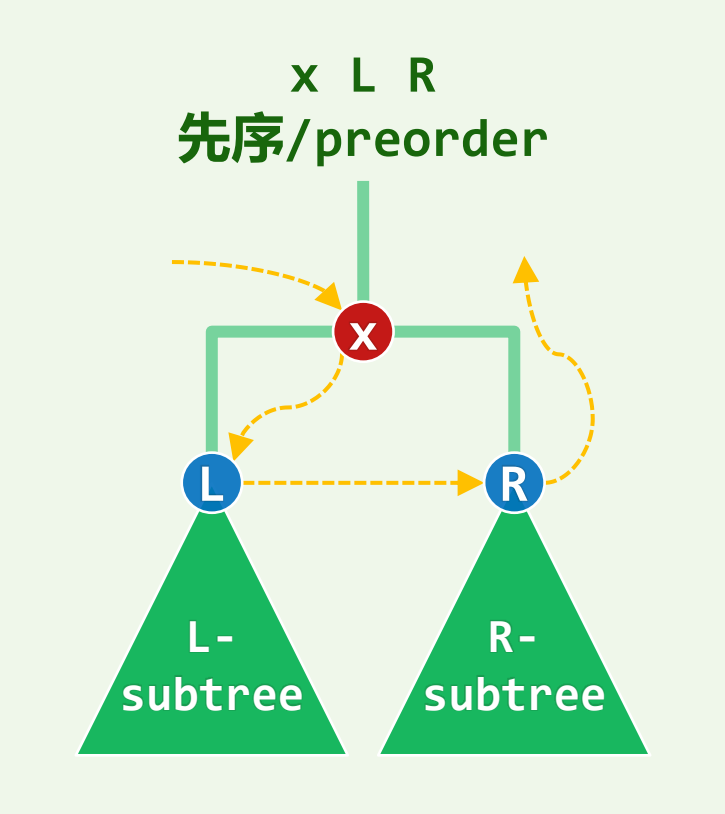
二叉树

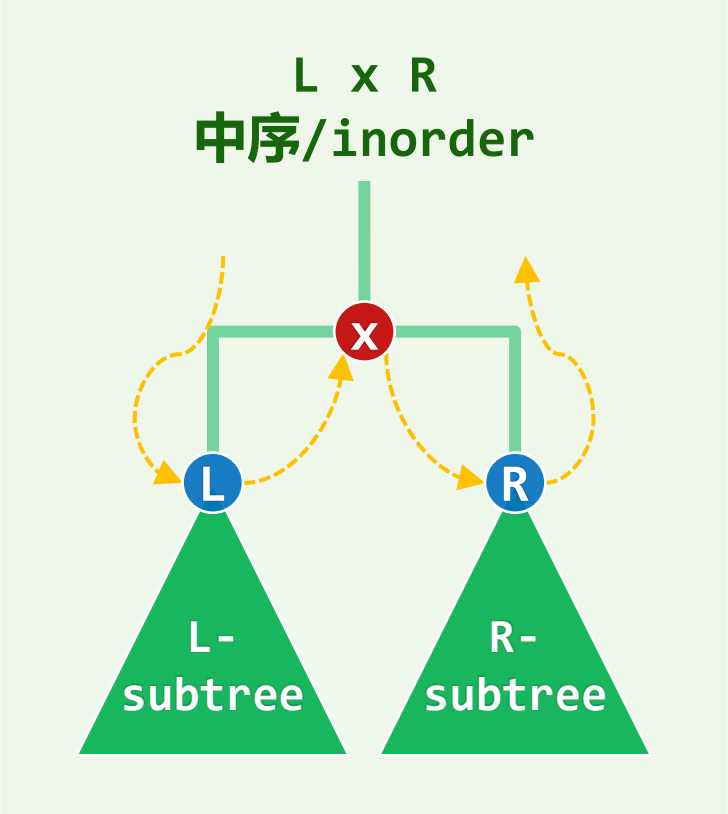
遍历：应用

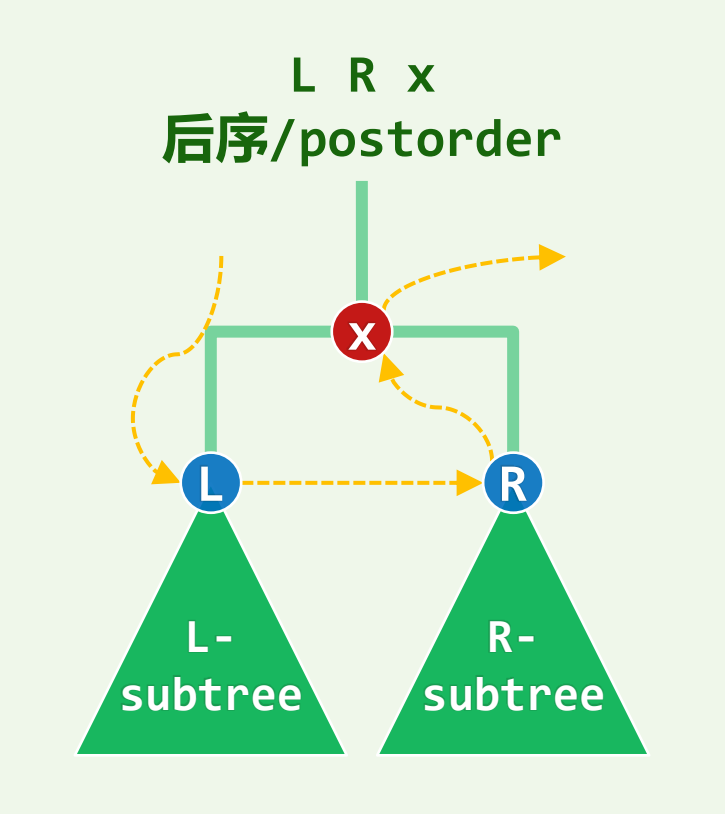
邓俊辉
deng@tsinghua.edu.cn

遍历

- 按照某种次序访问树中各节点，每个节点被访问恰好一次
- 结果 ~ 过程 ~ 次序 ~ 策略

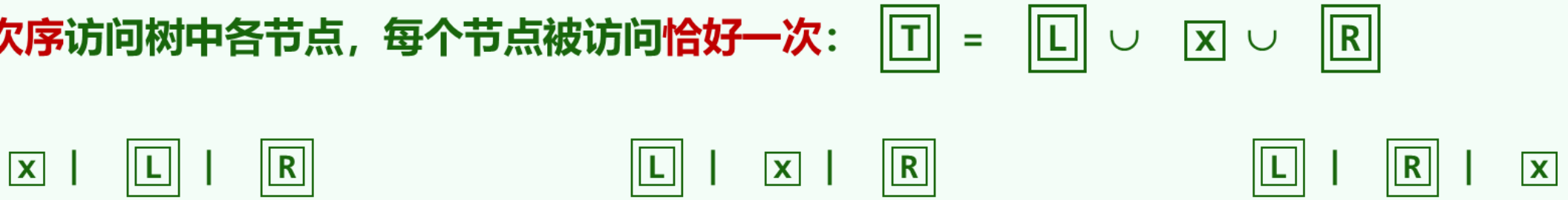


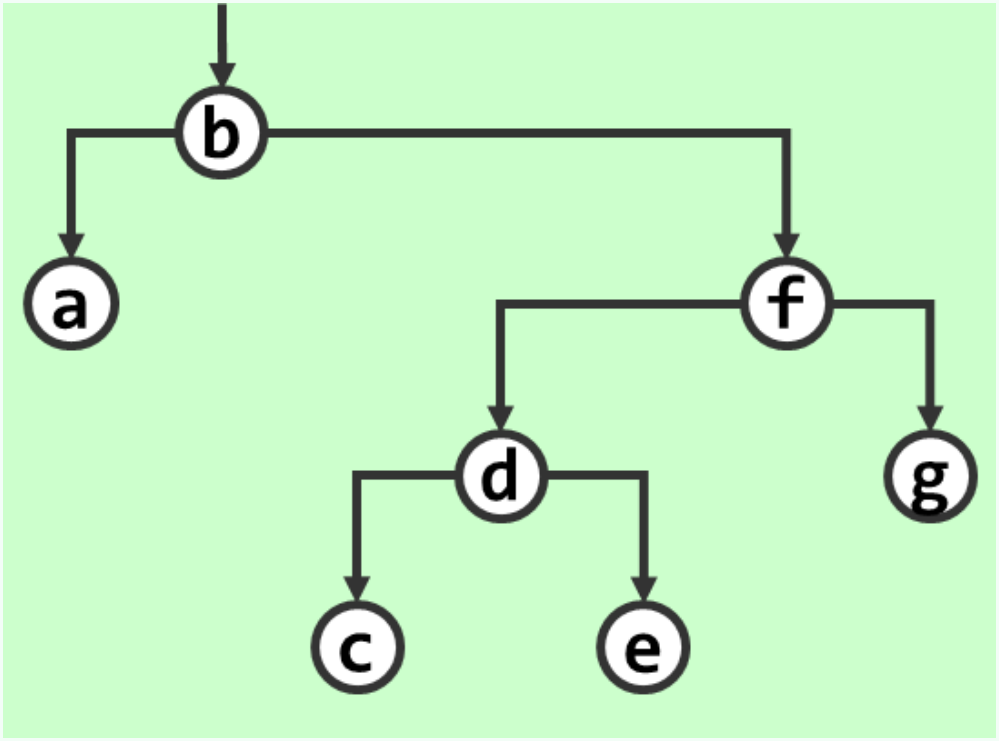




三种树遍历算法练习

❖按照某种次序访问树中各节点，每个节点被访问恰好一次：





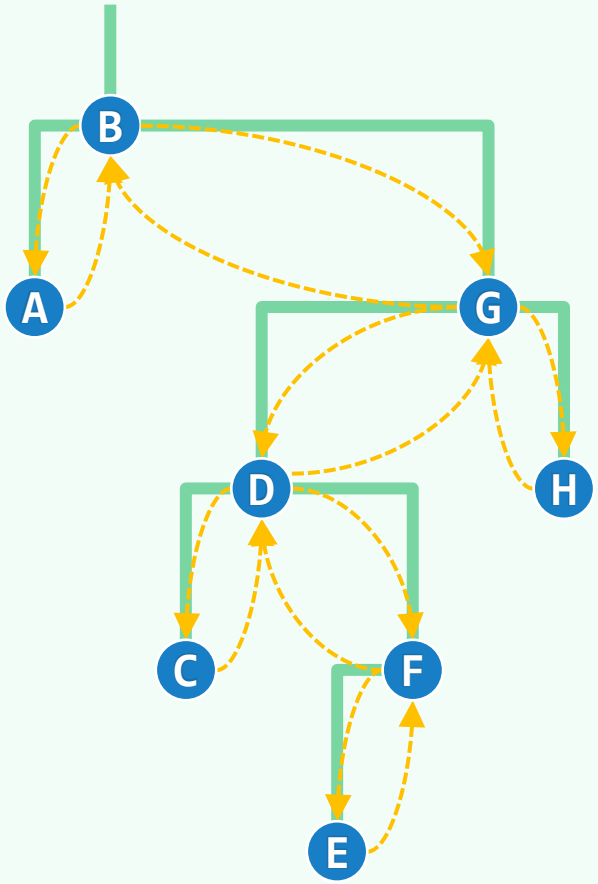
❖先序：bafdceg

❖中序：abcdefg

❖后序：acedgfb

递归实现

```
template <typename T, typename V> void traverse(BNP<T> x, V& visit) {
    if (!x) return;
    visit(x->data);          // pre(x) = { B, A, G, D, C, F, E, H }
    traverse(x->lc, visit);
    visit(x->data);          // in(x) = { A, B, C, D, E, F, G, H }
    traverse(x->rc, visit);
    visit(x->data);          // post(x) = { A, C, E, F, D, H, G, B }
} //traverse
```



❖ 假定visit()只需**常数时间**，则有：

$$T(n) = T(m) + T(n-m-1) + O(1) = \Theta(n)$$ //先**猜测**，再**验证**

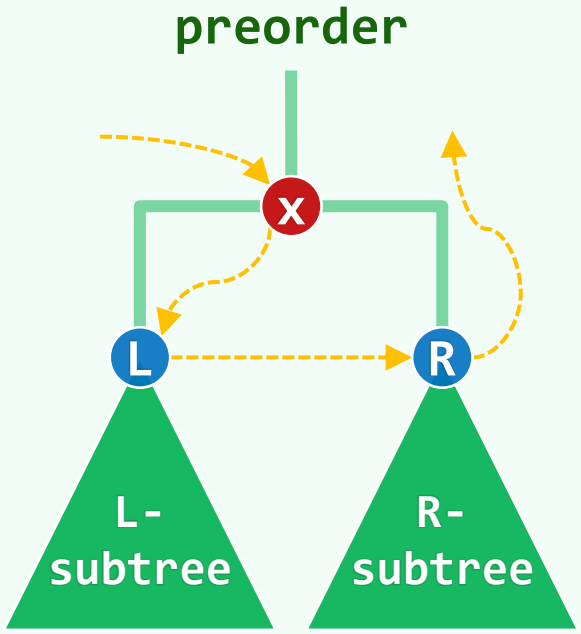
先序遍历实例：统计规模

❖ 先序输出文件树的结构

```
c:\> tree.com c:\windows
```

❖ 统计子树规模，亦即当前节点的后代总数

```
template <typename T> Rank BinNode<T>::size() {
   Rank s = 1; //先计入本身
   if (lc) s += lc->size(); //递归计入左子树
   if (rc) s += rc->size(); //递归计入右子树
   return s; //也可改由中序、后序遍历得到
} //O(n=size)：懒惰策略，现用现算；也可改为勤奋策略，始终动态维护
```



先序遍历实例：深复制：基本

```
template <typename T> //p ~ Parent; t ~ source Tree

BNP<T> clone(BNP<T> p, BNP<T> t) { //深复制

   if (!t) return nullptr;

   BNP<T> x = new BinNode<T>(t->data, p, nullptr, nullptr, t->height, t->color);

   x->lc = clone(x, t->lc);

   x->rc = clone(x, t->rc);

   return x; //由t复制出x

} //clone
```

先序遍历实例：深复制：扩展

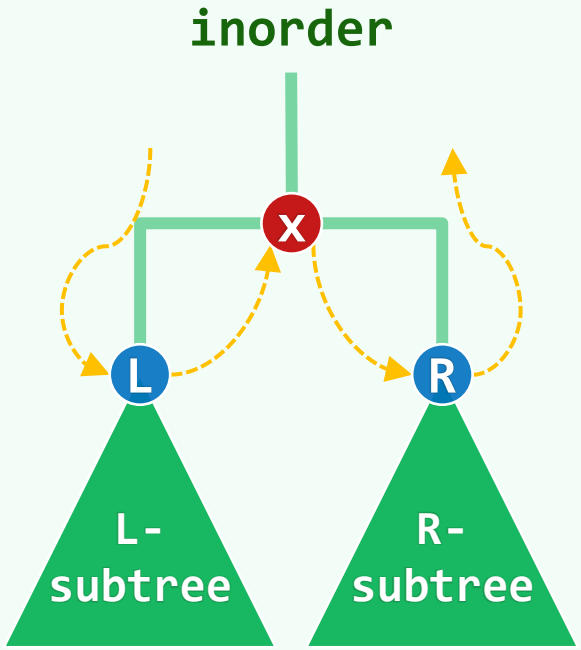
```
template <typename T> BinTree<T>::BinTree(BNP<T> t) //复制子树
   { _root = clone(nullptr, t); _size = t->size(); }

template <typename T> BinTree<T>::BinTree(const BinTree<T>& t) //复制全树
   { _root = clone<T>(nullptr, t._root); _size = t.size(); }

template <typename T> BinTree<T>& BinTree<T>::operator=(const BinTree<T>& t) {
   if (_root != t._root)
      { remove(_root); _root = clone<T>(nullptr, t._root); _size = t.size(); }
   return *this;
} //通过重载“operator=”实现赋值
```

中序遍历实例：输出二叉树

```
template <typename T>
void printBinTree(BNP<T> x, int depth) {
   if (!x) return;
   printBinTree(x->rc, depth+1); //右子树（在上）
   print(x); //根节点居中
   printBinTree(x->lc, depth+1); //左子树（在下）
} //printBinTree
```

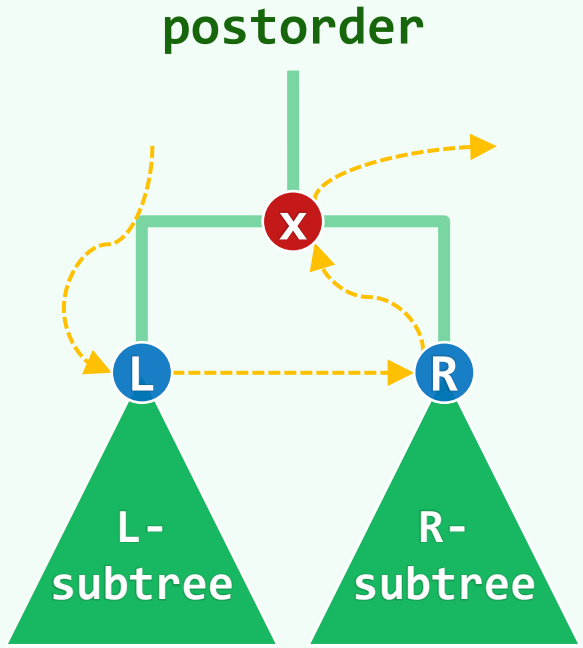


❖ 中序序列in(x)：全然不顾纵向的深度，只是依照横向的齿序逐个枚举

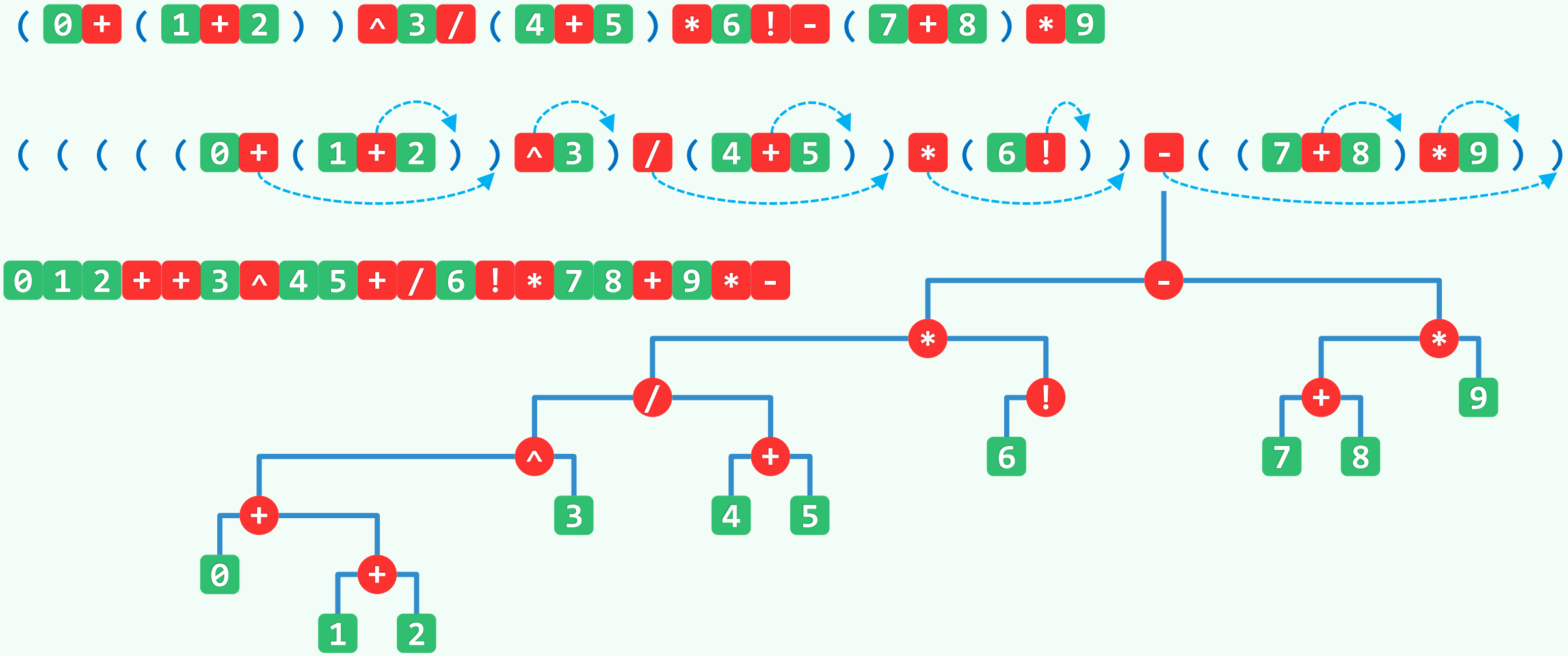
后序遍历实例：子树的删除与析构

```
template <typename T> Rank BinTree<T>::remove(BNP<T> x) {
   if (!x) return 0;  FromParentTo(x) = nullptr;
   if (x->parent) x->parent->updateHeightAbove();
   Rank n = clean(x); _size -= n; return n;
} //remove

template <typename T> Rank clean(BNP<T> x) {
   if (!x) return 0;
   Rank n = 1 + clean(x->lc) + clean(x->rc);
   delete x; return n;
} //clean
```



后序遍历实例：表达式树：后序遍历 = RPN

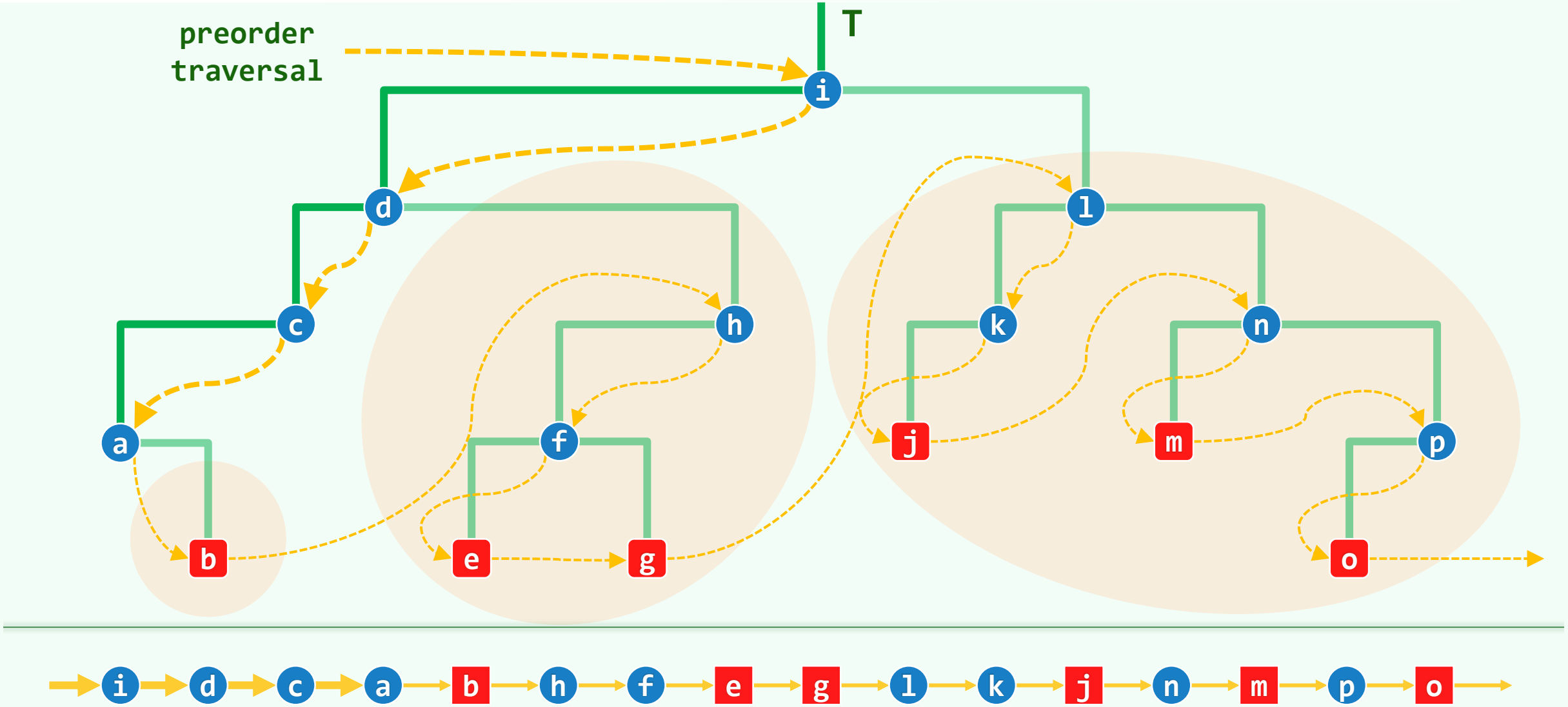


二叉树

遍历：迭代式先序

邓俊辉
deng@tsinghua.edu.cn

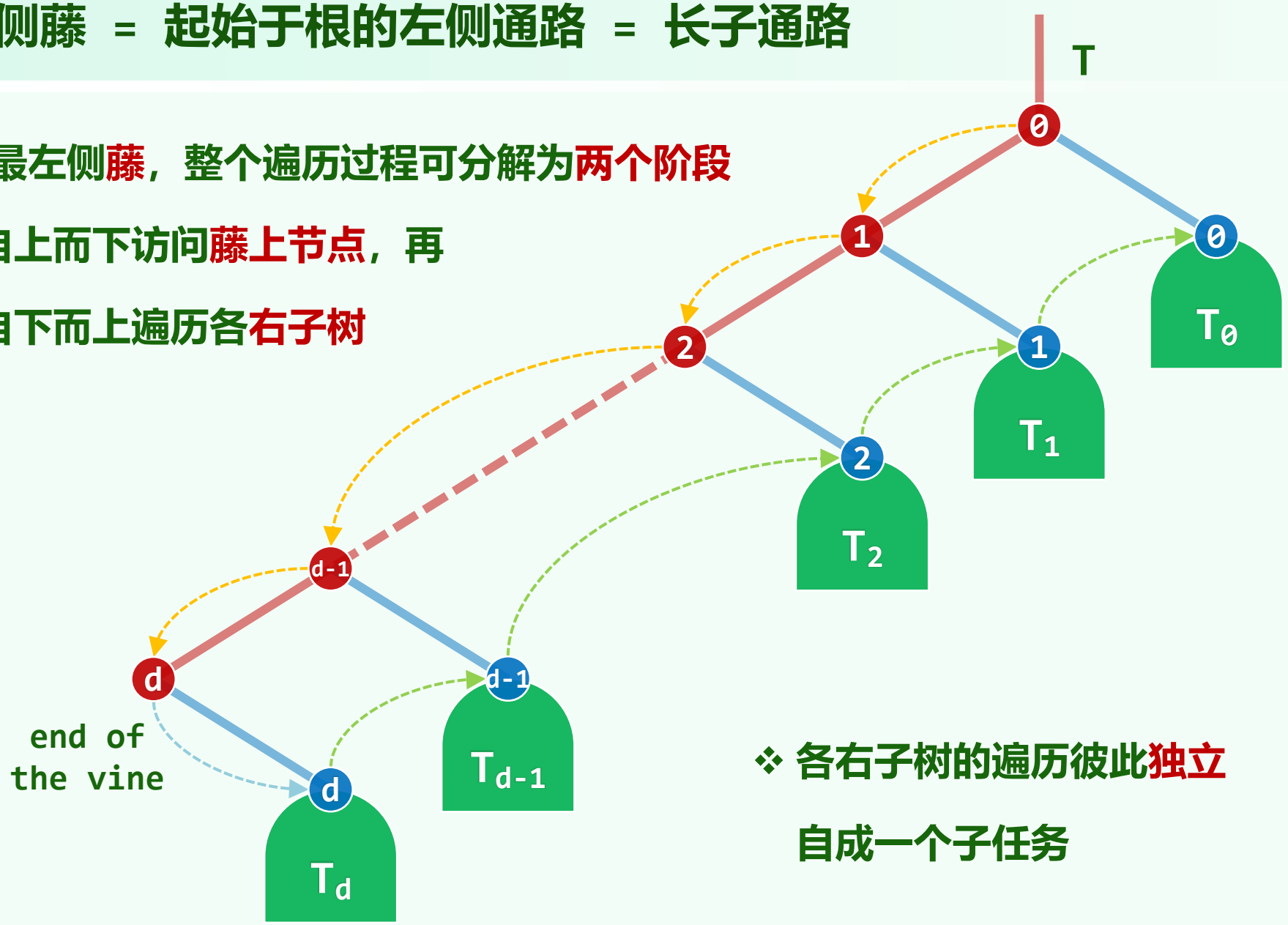
不依赖递归，如何实现先序遍历？效率如何？



最左侧藤 = 起始于根的左侧通路 = 长子通路

❖ 沿着最左侧藤，整个遍历过程可分解为两个阶段

- 自上而下访问藤上节点，再
- 自下而上遍历各右子树



❖ 各右子树的遍历彼此独立

自成一个子任务

```
visit(v0)
visit(v1)
...
visit(vd)
preorder(Td)
...
preorder(T1)
preorder(T0)
```

算法

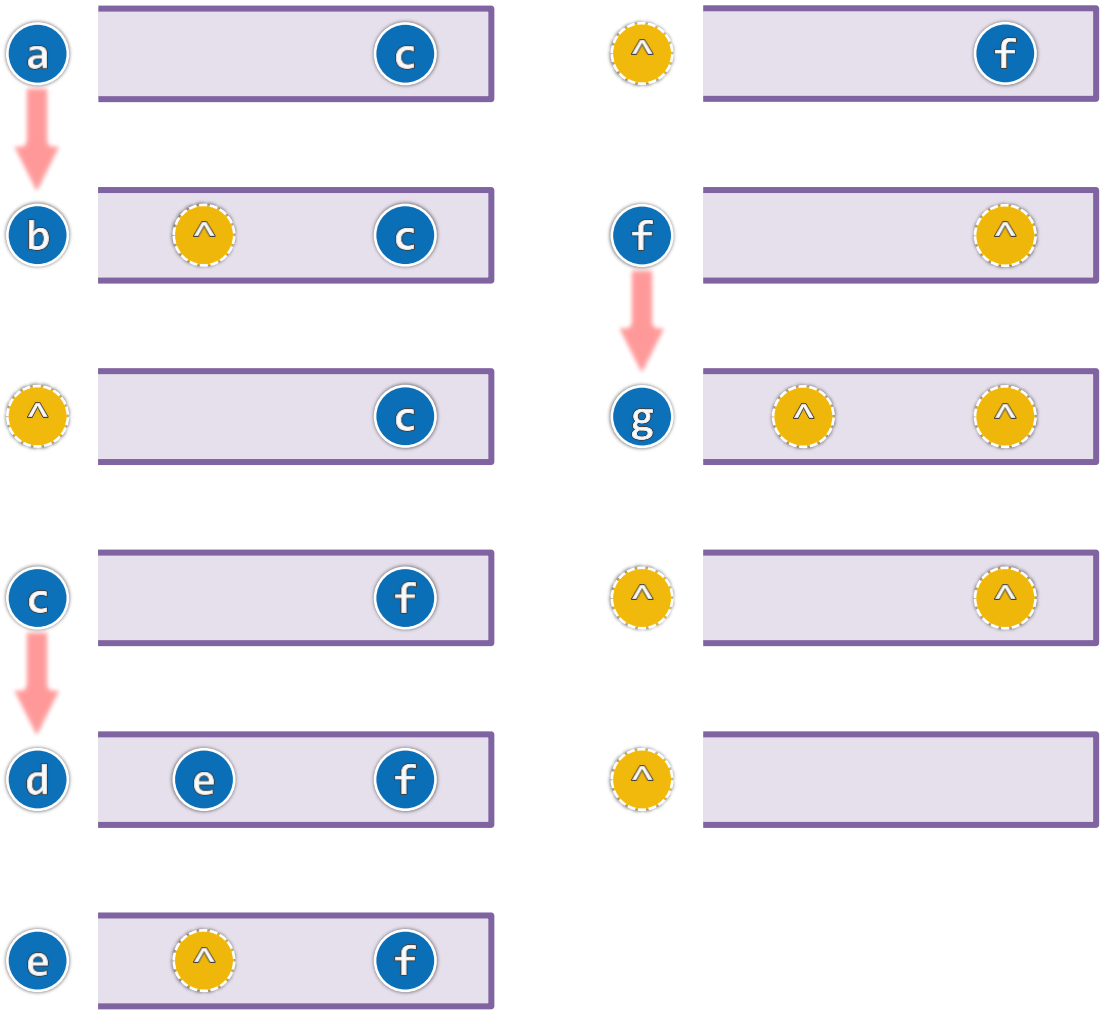
```
template <typename T, typename V> void preorder1(BNP<T> x, V& visit) {
    Stack<BNP<T>> S; S.push(x); //从当前节点出发
    while (!S.empty()) //以右子树为单位，依次访问每一条藤
        for (x = S.pop(); x; x = x->lc) //自x出发，沿着藤依次
            { visit(x->data); S.push(x->rc); } //访问各节点，并令右孩子（或空）入栈
} //preorder1
```

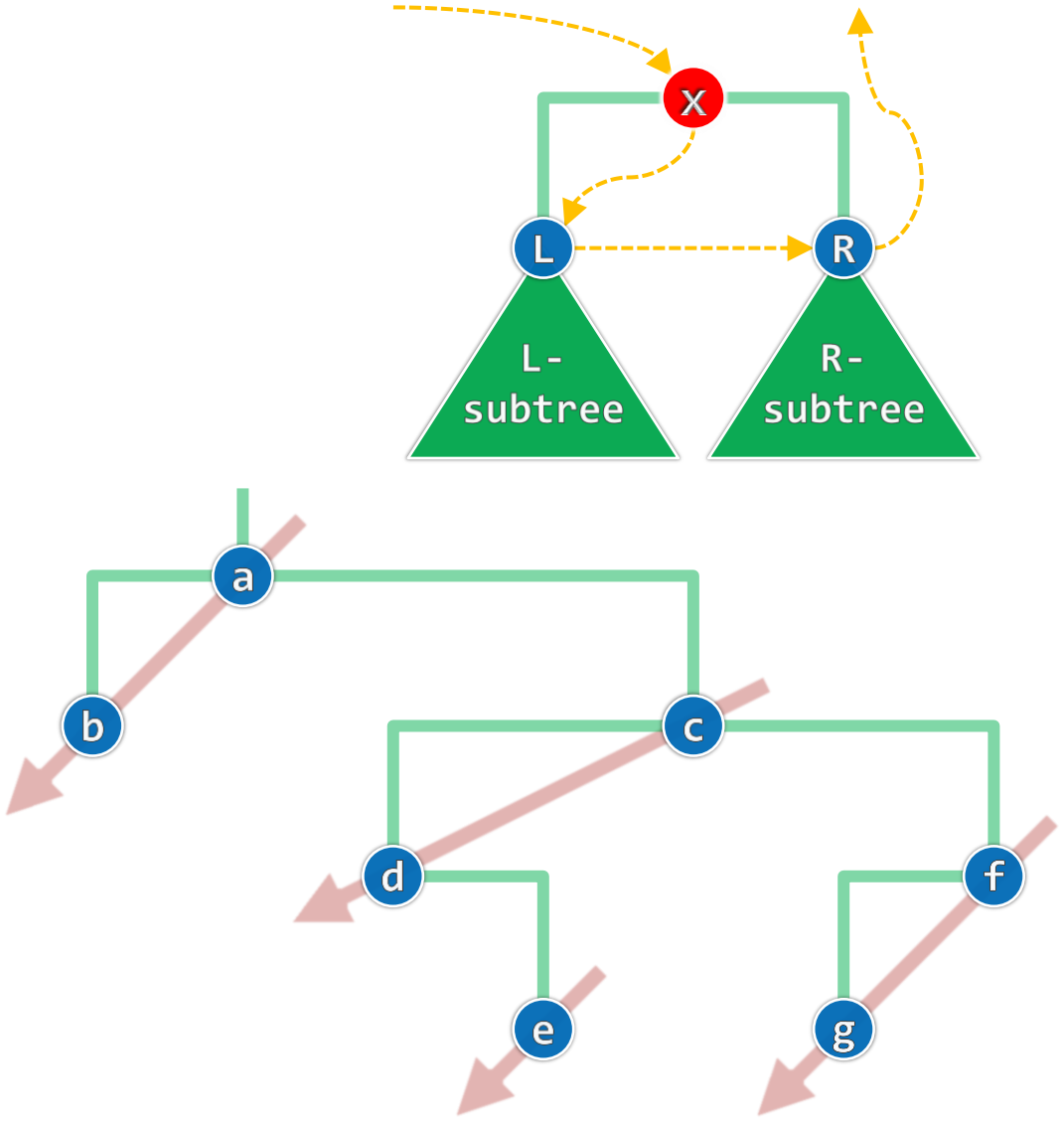
- while循环$O(n)$，for循环$O(n)$，岂不是...$O(n^2)$？

  其实，总共不过$O(n)$；摊还至每个节点，不过$O(1)$！

  为此，只需数数栈操作的累计次数...

实例：迭代式先序遍历





均摊|aggregate

```
❖ template <typename T, typename V> void preorder1(BNP<T> x, V& visit) {
    Stack<BNP<T>> S; S.push(x); //从当前节点出发
    while (!S.empty()) //以右子树为单位，依次访问每一条藤
       for (x = S.pop(); x; x = x->lc) //自x出发，沿着藤依次
          { visit(x->data); S.push(x->rc); } //访问各节点，并令右孩子（或空）入栈
  } //preorder1
```

❖ 唯有右孩子才能（也必须）进栈，#push = #rc = n+1 //含空的右孩子|树根也等同于右孩子

#pop = #push = n+1; #empty = #pop + 1 = n+2

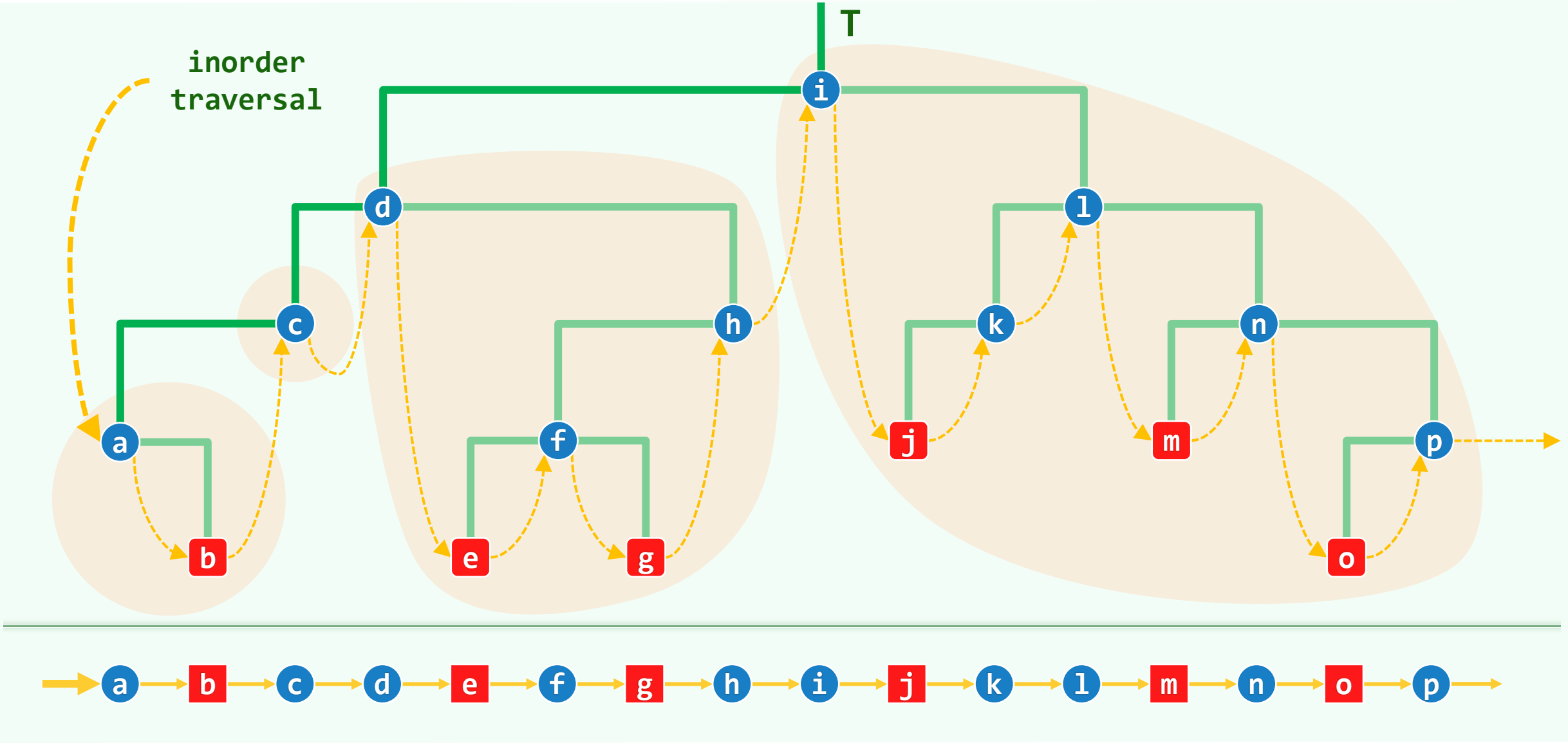
❖ 空间 = 显式的栈S：虽渐近地仍是$O(h)$，但每个元素仅4个字节，常系数已降至极限

二叉树

遍历：迭代式中序

邓俊辉
deng@tsinghua.edu.cn

不依赖递归，如何实现中序遍历？效率如何？



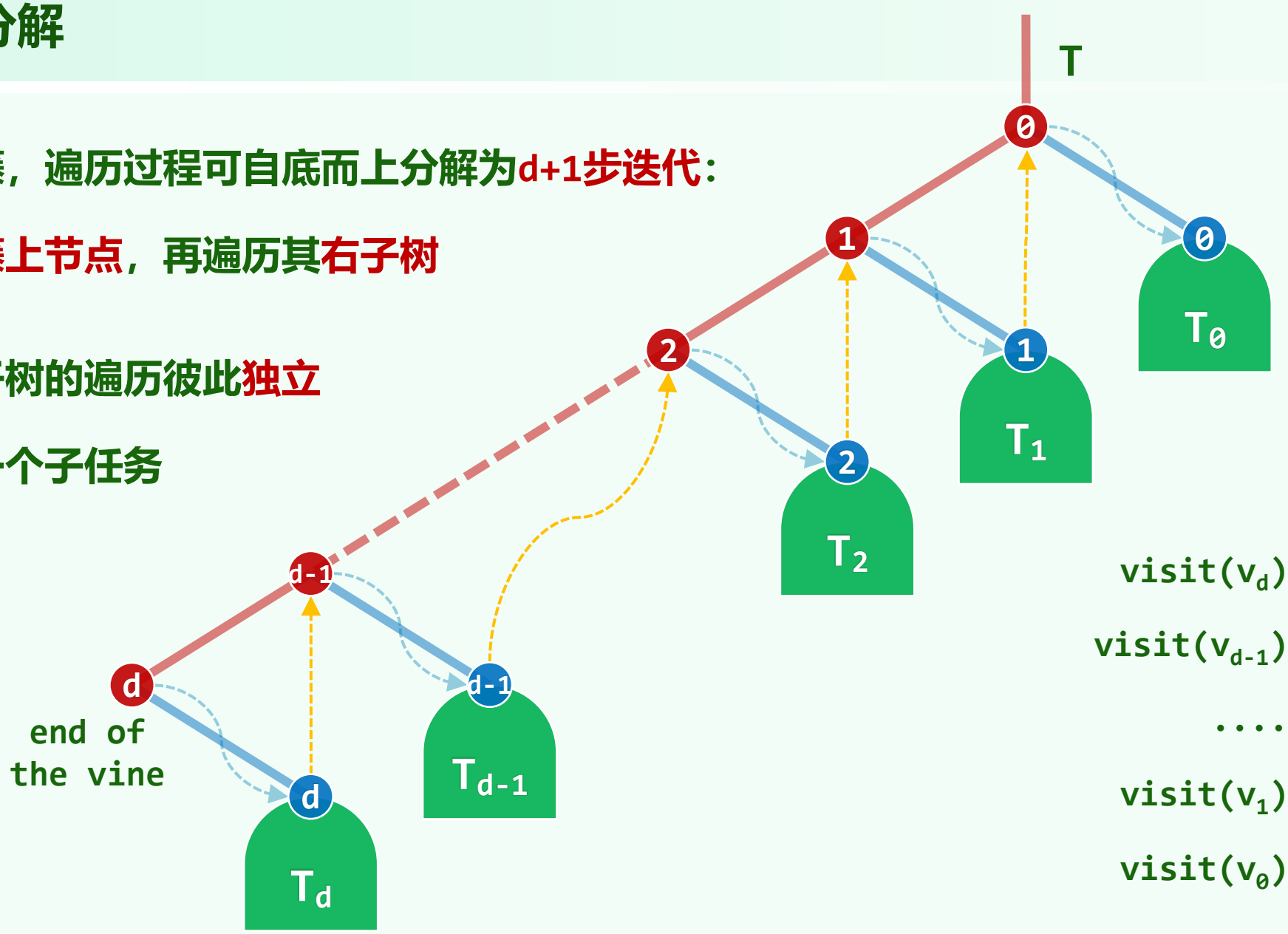
沿藤分解

- 沿着藤，遍历过程可自底而上分解为d+1步迭代：

  访问藤上节点，再遍历其右子树

- 各右子树的遍历彼此独立

  自成一个子任务



visit(v_d), inorder(T_d)

visit(v_{d-1}), inorder(T_{d-1})

...., ...

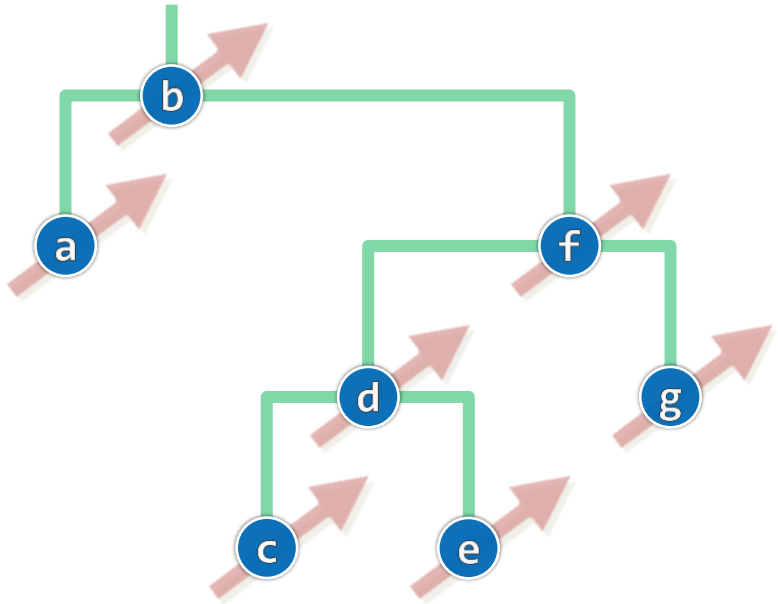
visit(v_1), inorder(T_1)

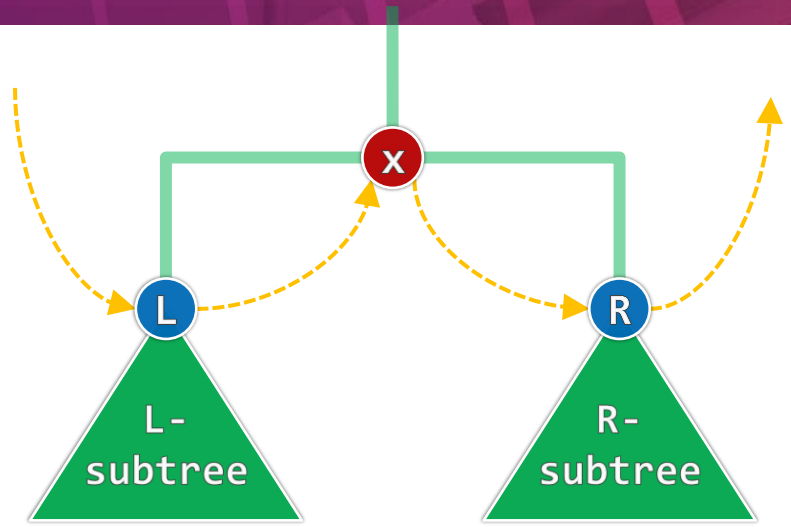
visit(v_0), inorder(T_0)

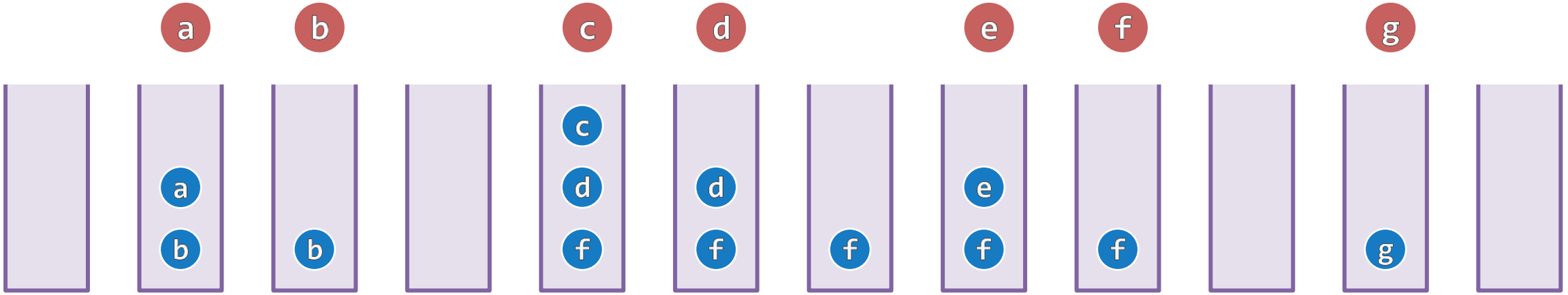
算法

```
template <typename T, typename V> void inorder1(BNP<T> x, V& visit) {
   Stack<BNP<T>> S; //辅助栈
   while (x || !S.empty()) { //只要x非nullptr，或S仍非空
      for ( ; x; x = x->lc) S.push(x); //藤上节点便自顶而下入栈
      visit((x = S.pop())->data); //取出并访问末节点，再
      x = x->rc; //转向右子树
   } //while
} //inorder1
```

实例：迭代式中序遍历







正确性：数学归纳

❖ 假设算法可正确地遍历所有规模不超过n的二叉树

❖ 考查任一规模为n+1的二叉树，其根记作x

将最左侧藤之终点记作t，t之右孩子记作y

❖ 整个算法分为五个阶段：

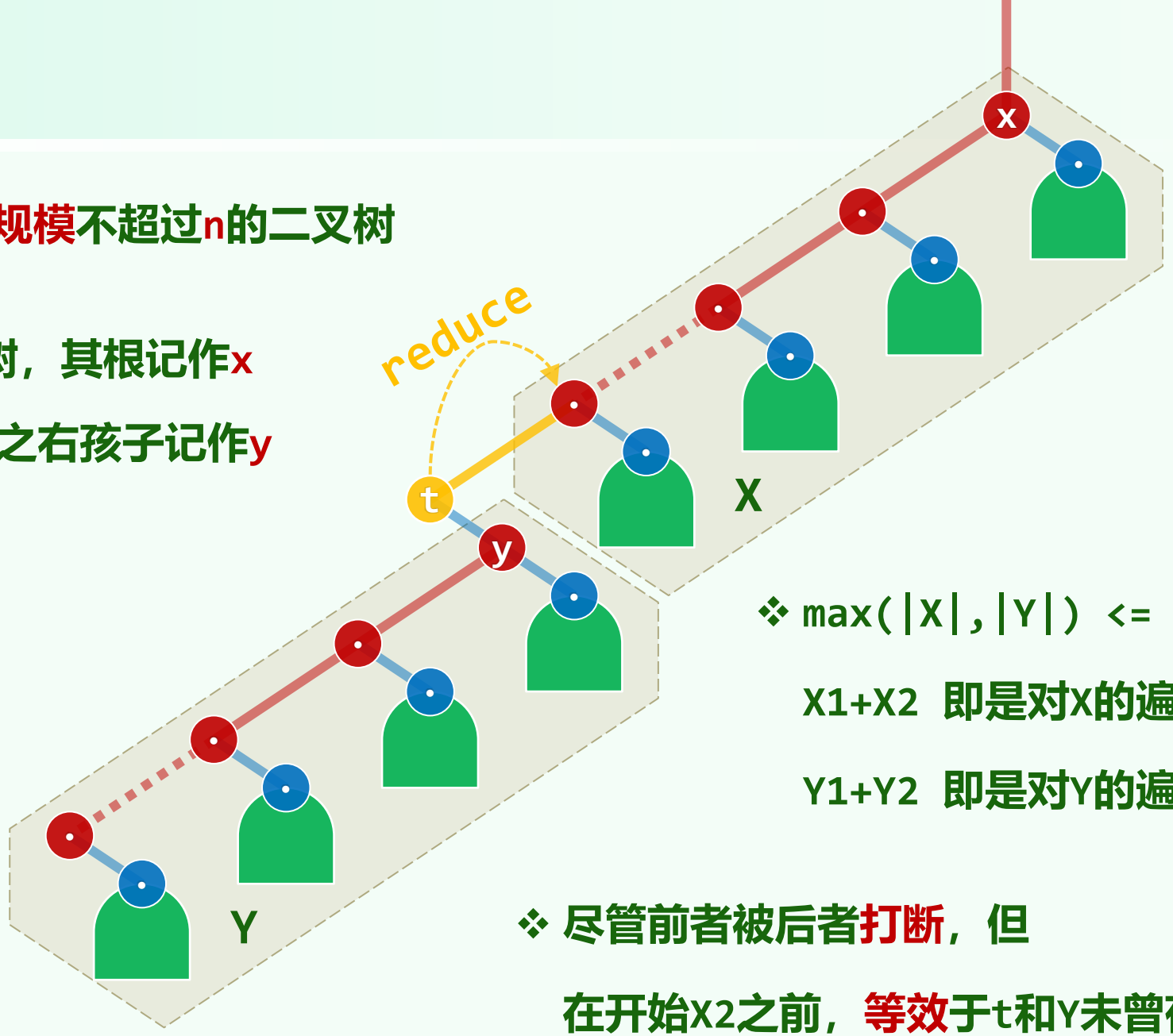
X1: for(x)..push(x)

T: visit(t = S.pop())

Y1: for(y)..push(y)

Y2: inorder(Y)

X2: inorder(X)



❖ max(|X|,|Y|) <= n，故

X1+X2 即是对X的遍历

Y1+Y2 即是对Y的遍历

❖ 尽管前者被后者打断，但

在开始X2之前，等效于t和Y未曾存在过

均摊|accounting

- 空间：与先序遍历一样，最大递归深度有常数倍的增加，可吞吐的树的最大规模有指数量级的增加
- 时间：依然不是$O(n^2)$；均摊分析可知，总体不过$O(n)$...

  事先，为每个节点派发3枚金币
- 时间消耗的主体部分仍是栈操作，每一次都有节点来支付报酬
  - push(x)：不会有节点重复入栈，故可以就由x支付
  - x = pop(x)：也不会有节点重复出栈，故亦可由x支付
  - empty()：仅出现在while的逻辑判断中

    除了最后一次，每次执行都对应于某棵空的右子树

    虽然空树没钱，但它的父亲却有...子债...父还

二叉树

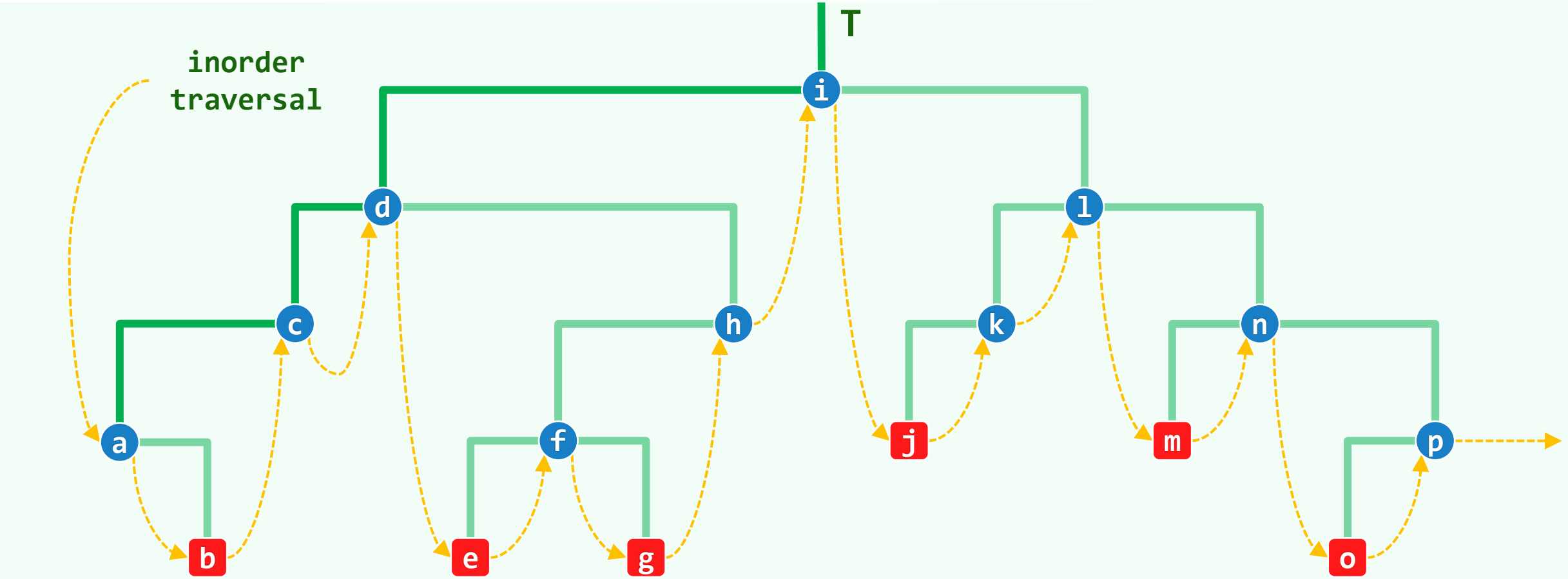
遍历：中序后继

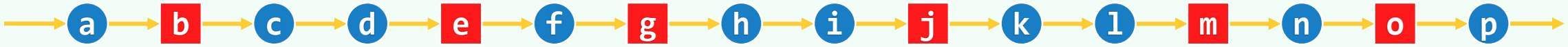
有RC，右孩子最左（右孩子里最小的）

无RC，左祖先最低（祖先左孩子里最大的）

邓俊辉
deng@tsinghua.edu.cn

简明遍历: for (x = first(x); x; x = x->succ())





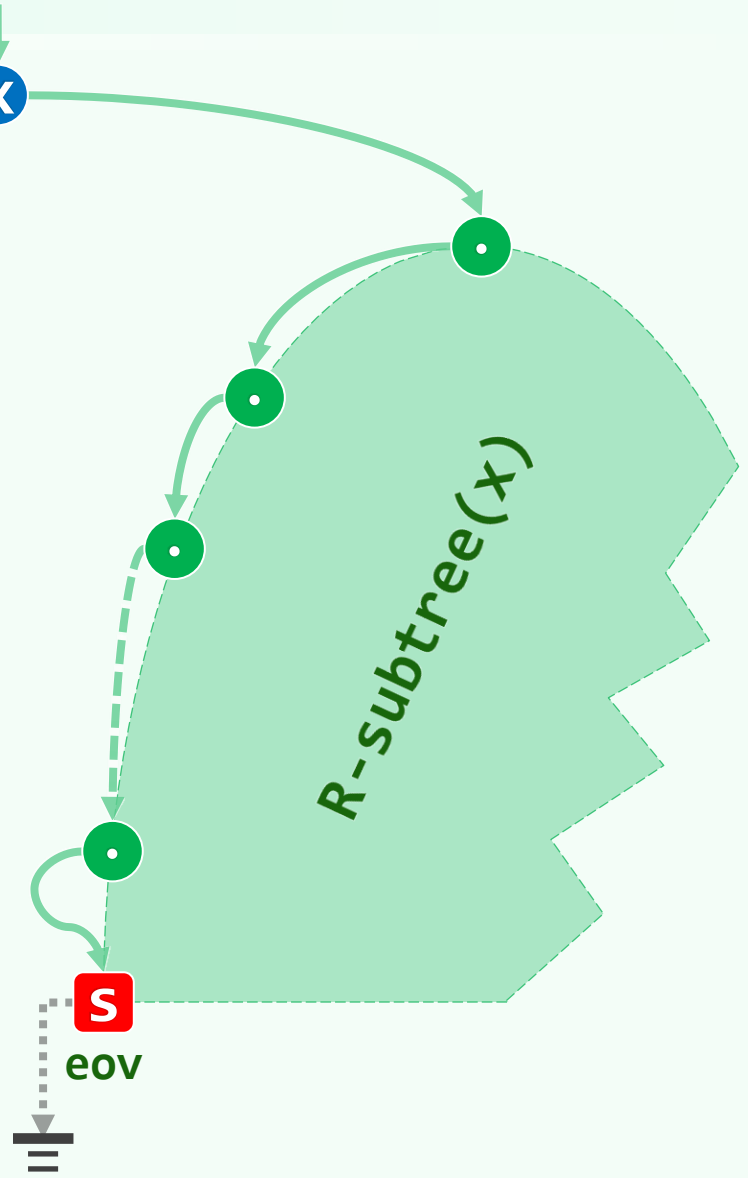
直接后继：右子树中藤之终点

有RC，右孩子最左（右孩子里最小的）

```
template <typename T> BNP<T> BinNode<T>::succ()
    { return rc ? eov(rc) : lla(this); }
```

无非两种情况，取决于是否有右子树

```
template <typename T>
BNP<T> eov(BNP<T> x) { //End Of the Vine
    if (x) //从x出发
        for ( ; x->lc; x = x->lc); //顺藤下行
    return x; //直至终点
} //eov
```



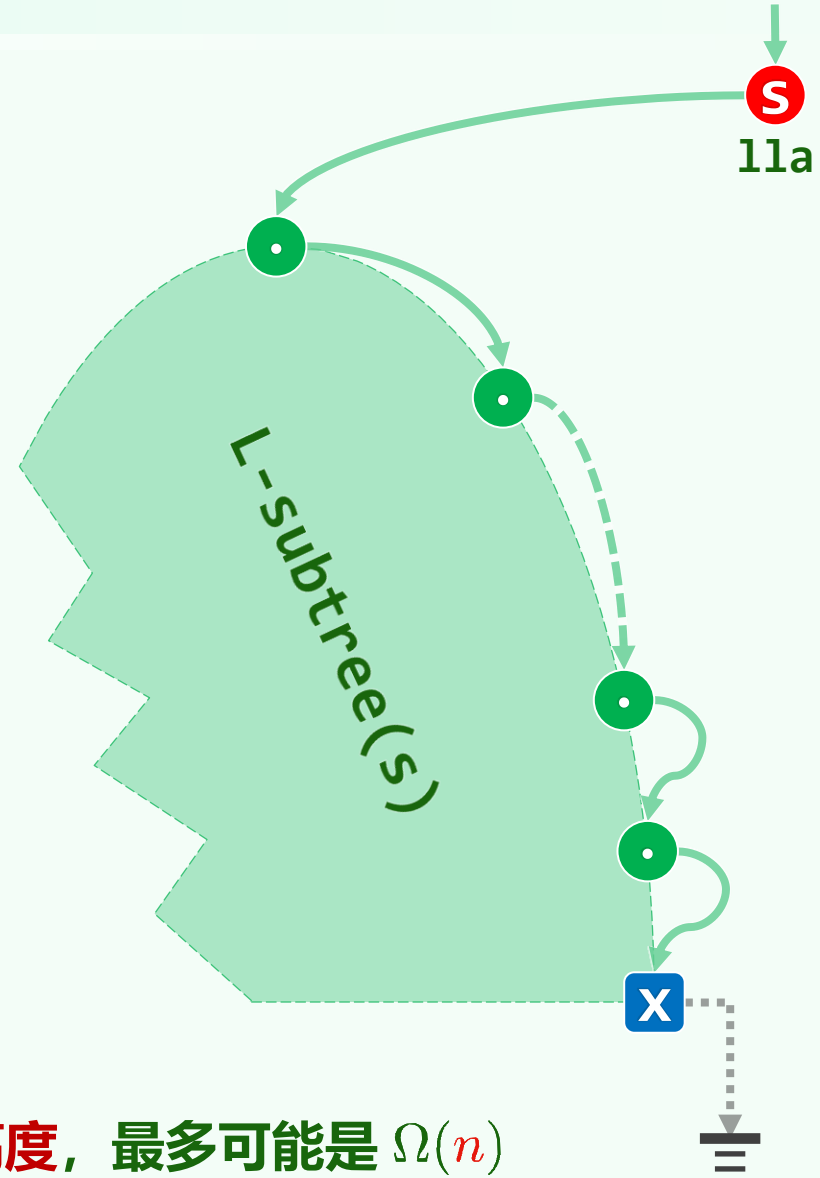
直接后继：最低的左祖先

无RC，左祖先最低（祖先左孩子里最大的）

❖ 没有右子树时，逆向思维：哪个节点以x为前驱？

对称地，搜索方向由下行转为上行

❖
```
template <typename T>
BNP<T> lla(BNP<T> x) { //Lowest Left Ancestor
   while (x->parent && x == x->parent->rc)
      x = x->parent; //右父亲的...右父亲的
   return x->parent; //左父亲（可能是nullptr）
} //lla
```



❖ 运行时间分别为height(x)与depth(x)：均不超过全树的高度，最多可能是 $\Omega(n)$

规范简约的遍历：算法

```
template <typename T, typename V> void inorder2(BNP<T> x, V& visit) {
    for (x = first(x); x; x = x->succ()) //依次
        visit(x->data); //枚举所有节点
} //inorder2
```

- 任何遍历都可套用这个**范式**，只要写好各自的first()和succ()

  就中序序列而言，首节点就是EOV：`auto first = [](auto x) { return eov(x); };`

- 最坏情况下，**单次**succ()就需 $\Omega(n)$ 时间，难道总体是 $\Omega(n^2)$ ？

  形式上的极致简约，是以计算**效率**的牺牲为代价？

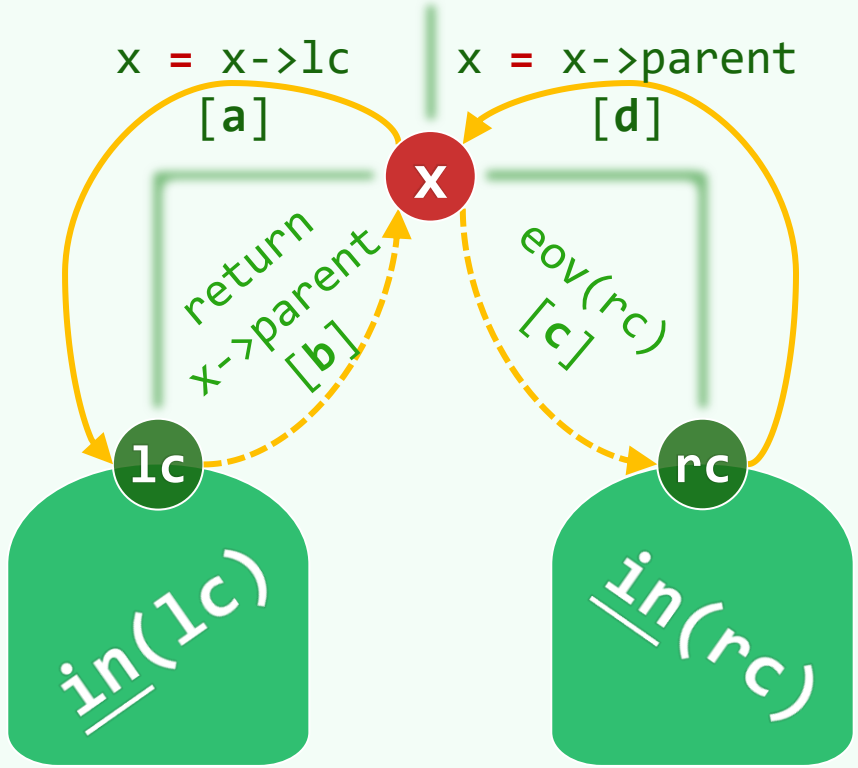
- 我们来统计一下，所有的eov()和lla()中，x改变**位置（被赋值）**的次数，记作 $T(n)$

规范简约的遍历：性能

❖ 验证：$T(1) = 0$；以下导出其递推方程

❖ 立足于树根x，整个遍历过程可以等效地分为七个阶段：

- ✓ [a] 经first(x)的第一次赋值转至lc
- \- （等效于递归地）完成inorder(lc)
- ✘ [b] 经lla(x->pred)的最后一步返回x
- ✘ visit(x)访问x
- ✘ [c] （为查找eov(rc)而）直接转入rc
- \- （等效于递归地）完成inorder(rc)
- ✓ [d] 经最后一次lla()的最后一次赋值返回x



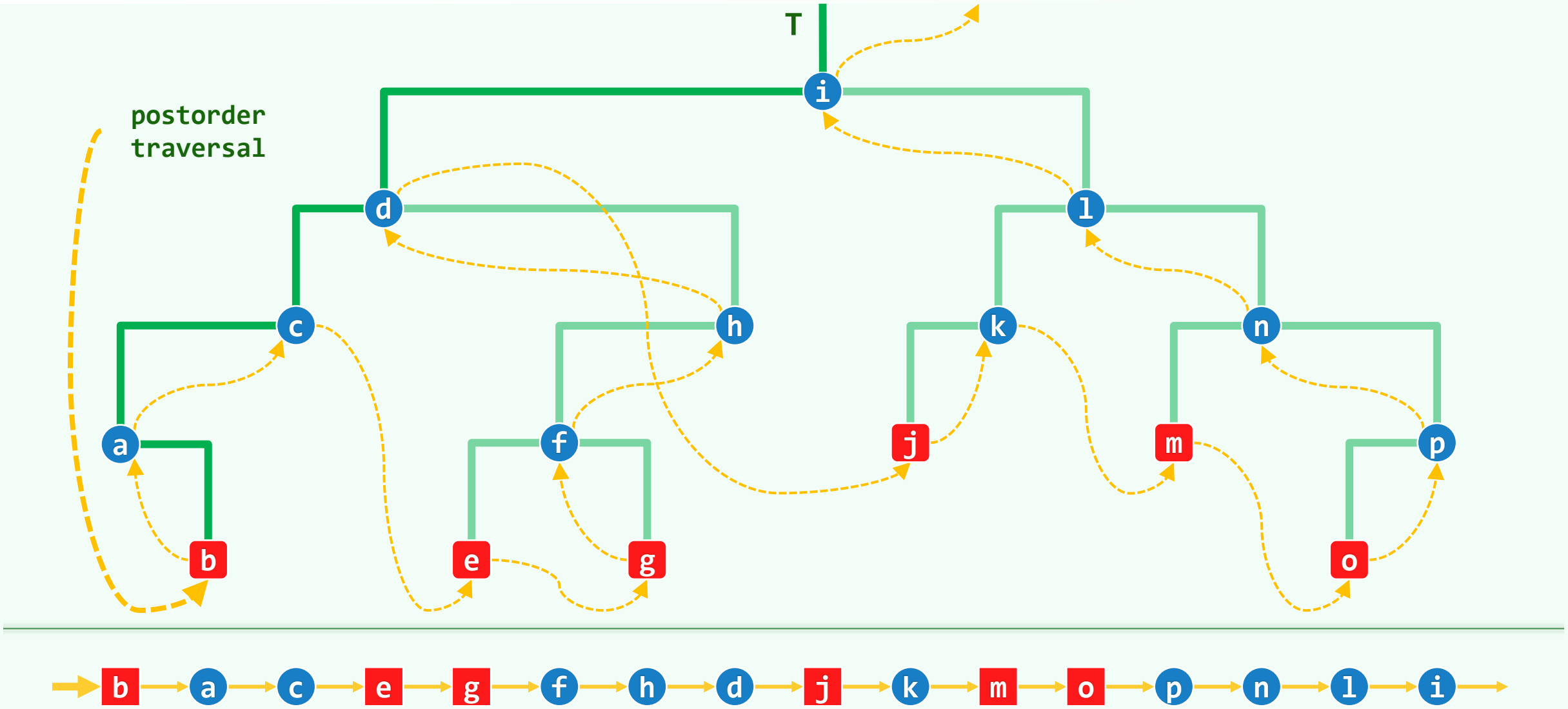
❖ 故有：$T(n) = T(m) + T(n-m-1) + 2 = 1 \cdot n - 1 = \mathcal{O}(n)$

二叉树

遍历：迭代式后序

邓俊辉
deng@tsinghua.edu.cn

不依赖递归，如何实现后序遍历？效率如何？



沿藤分解

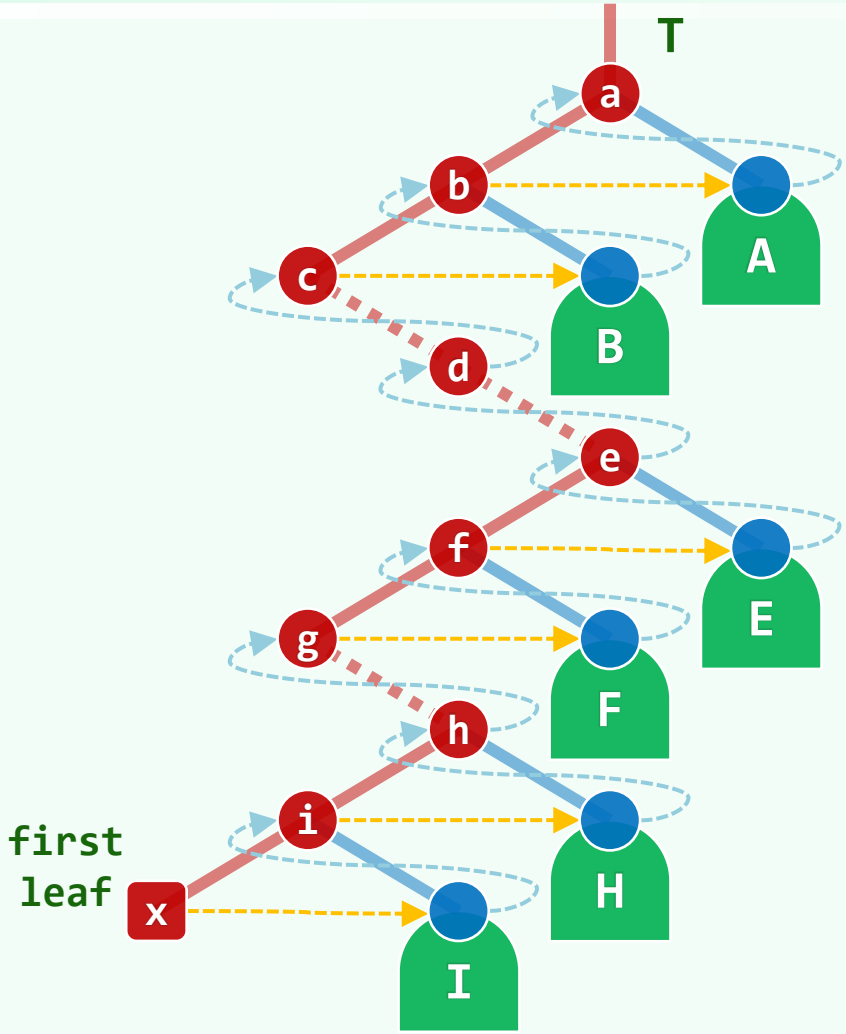
- 思路不变，还是

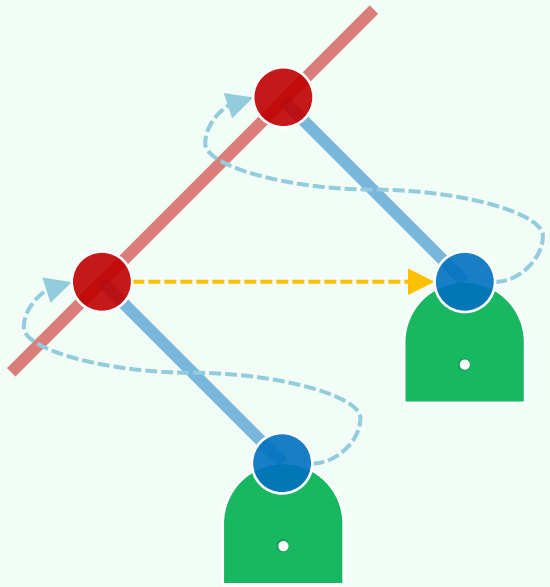
  从根出发，梳理出一条藤

- 稍显复杂，原理相同
  - 尽可能向左
  - 实不得已才向右
  - 直至全树中

    最靠左的叶节点

    亦即遍历的起点

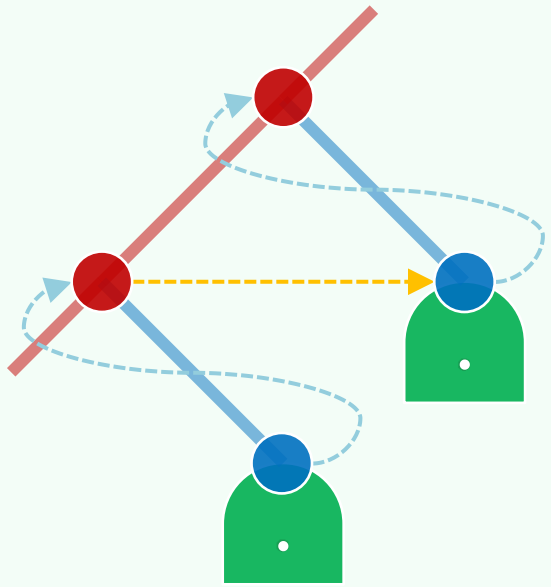




- 如此，遍历序列便可分解为：$\underline{post}(T) = x;\ \underline{post}(I), i;\ \ldots;\ \underline{post}(C), c;\ \underline{post}(B), b;\ \underline{post}(A), a;$

序曲

```
❖ template <typename T> void vine(Stack <BNP<T>>& S) { //展开起始于栈顶节点的藤
     for (BNP<T> x = S.top(); x; ) { //自顶而下
        if (x->rc) //右孩子
           S.push(x->rc); //先进后出
        if (x->lc) //若有左孩子，则优先左转
           { S.push( x->lc ); x = x->lc; }
        else x = x->rc; //实不得已，才向右
     } //for
  } //vine
```

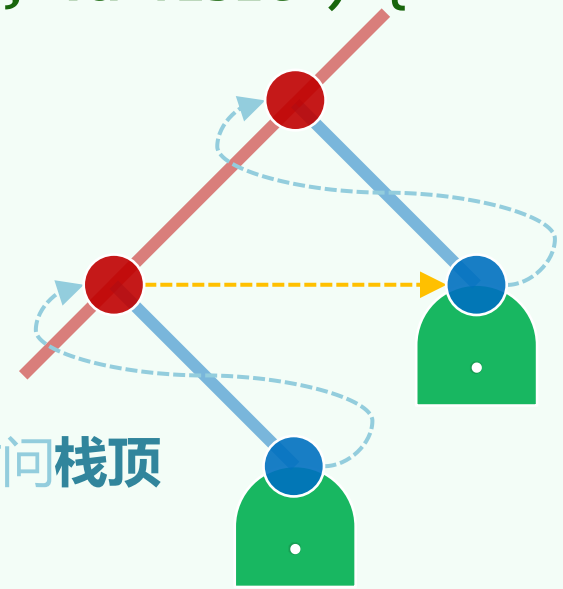


❖ **右子树的新藤，总是续接在栈S的顶部：后进先出**

全曲

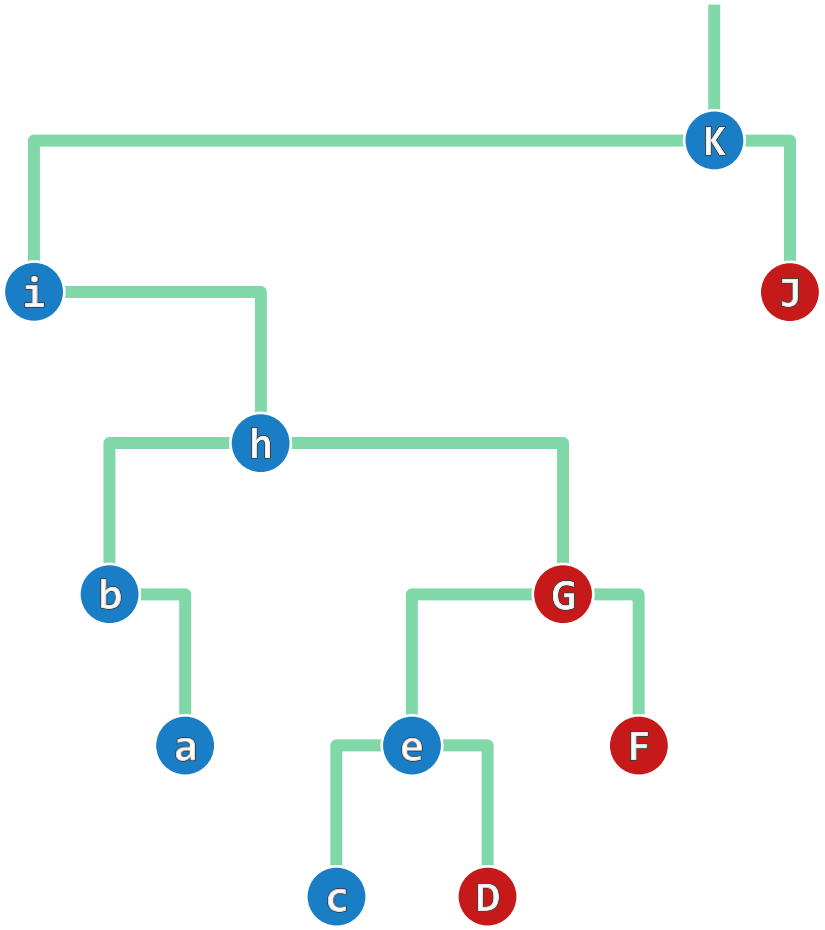
- 访问x之后，借助S总是可以确定下一个访问的节点；视栈顶节点的身份，无非两种情况

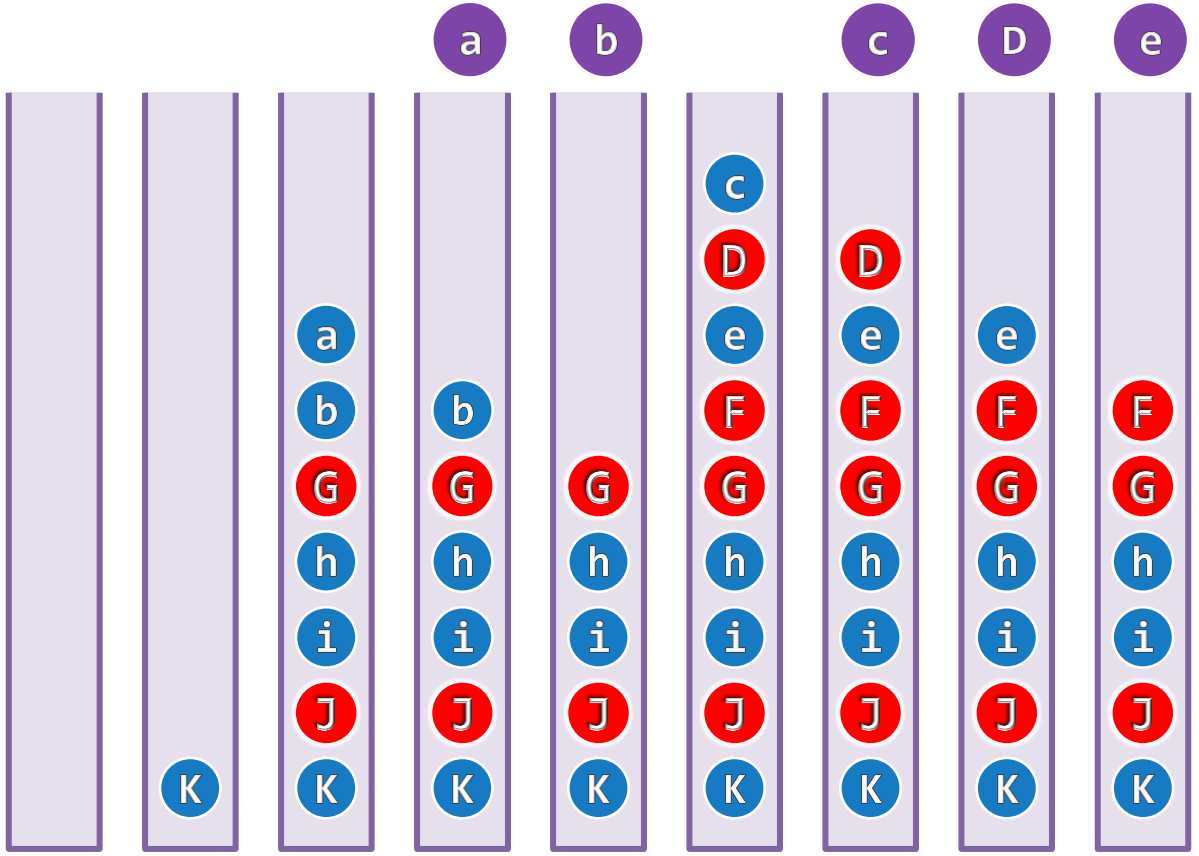
```
template <typename T, typename V> void postorder1( BNP<T> x, V& visit ) {
   Stack<BNP<T>> S;  if ( x ) S.push( x ); //辅助栈
   while ( ! S.empty() ) { //x为刚被访问过的节点
      if ( S.top() != x->parent ) //若栈顶非x之父（而为右兄）
         vine( S ); //则在右兄子树中展开对应的藤（栈顶为eov）
      x = S.pop(); visit( x->data ); //无论如何，接下来只需访问栈顶
   } //while
} //postorder1
```



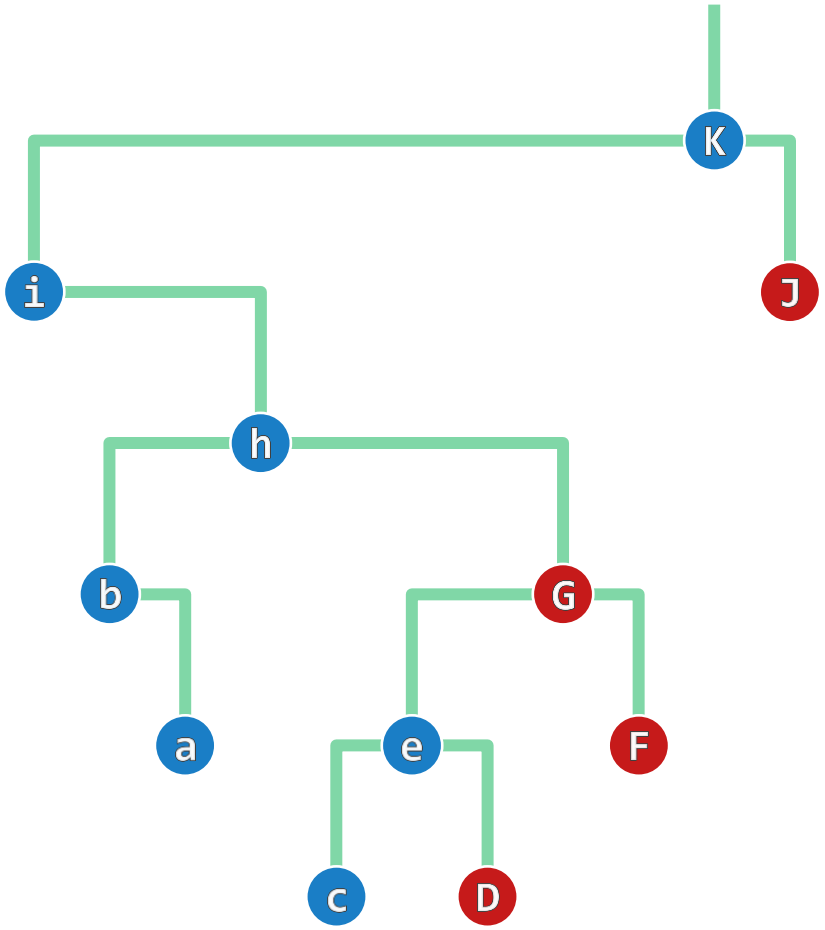
- 若栈顶是x的父亲：直接弹出并访问
- 否则（栈顶必是x的右兄弟），经调用一次vine(S)之后，新栈顶即是下一应被访问者

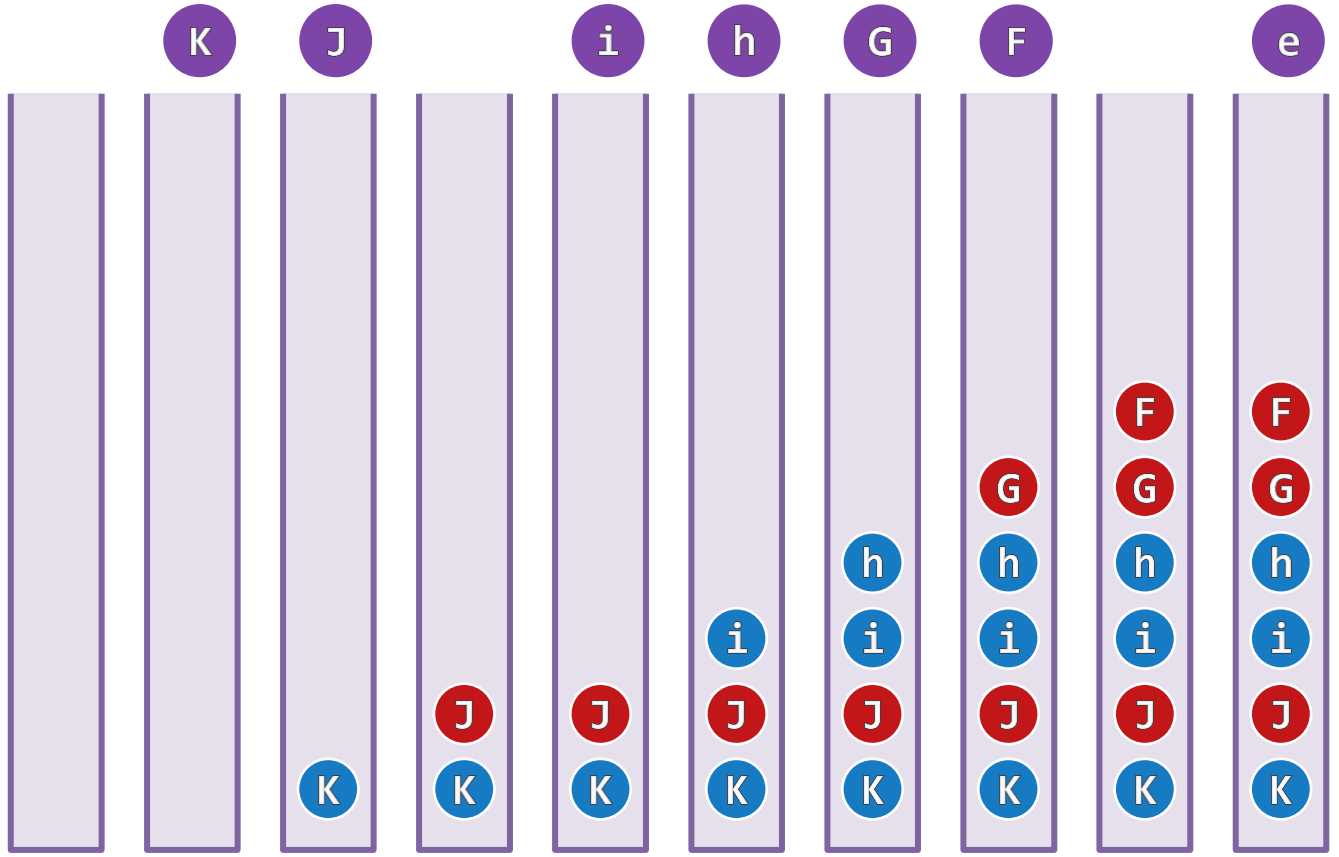
实例：迭代式后序遍历（1）




a
b
c
D
e
K
K
K

实例：迭代式后序遍历（2）


K
i
J
h
b
G
a
e
F
c
D


K
J
i
h
G
F
e
K
K
J
J
i
h
h
G
F
F
F
G
G
G
h
h
h
h
h
i
i
i
i
i
J
J
J
J
J
J
K
K
K
K
K
K
K

二叉树

遍历：层次

真君曰：“昔呂洞賓居廬山而成仙，鬼谷子居雲夢而得道，今或無此吉地麼？”璞曰：“有，但當遍歷耳。”

我们注定是扎根于前半生的，即使后半生充满了强烈的和令人感动的经历

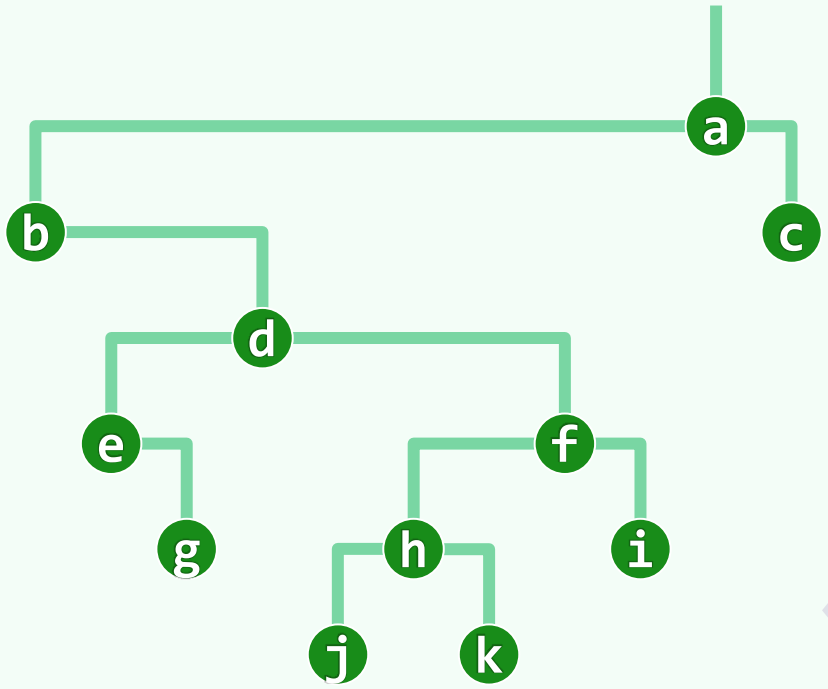
邓俊辉
deng@tsinghua.edu.cn

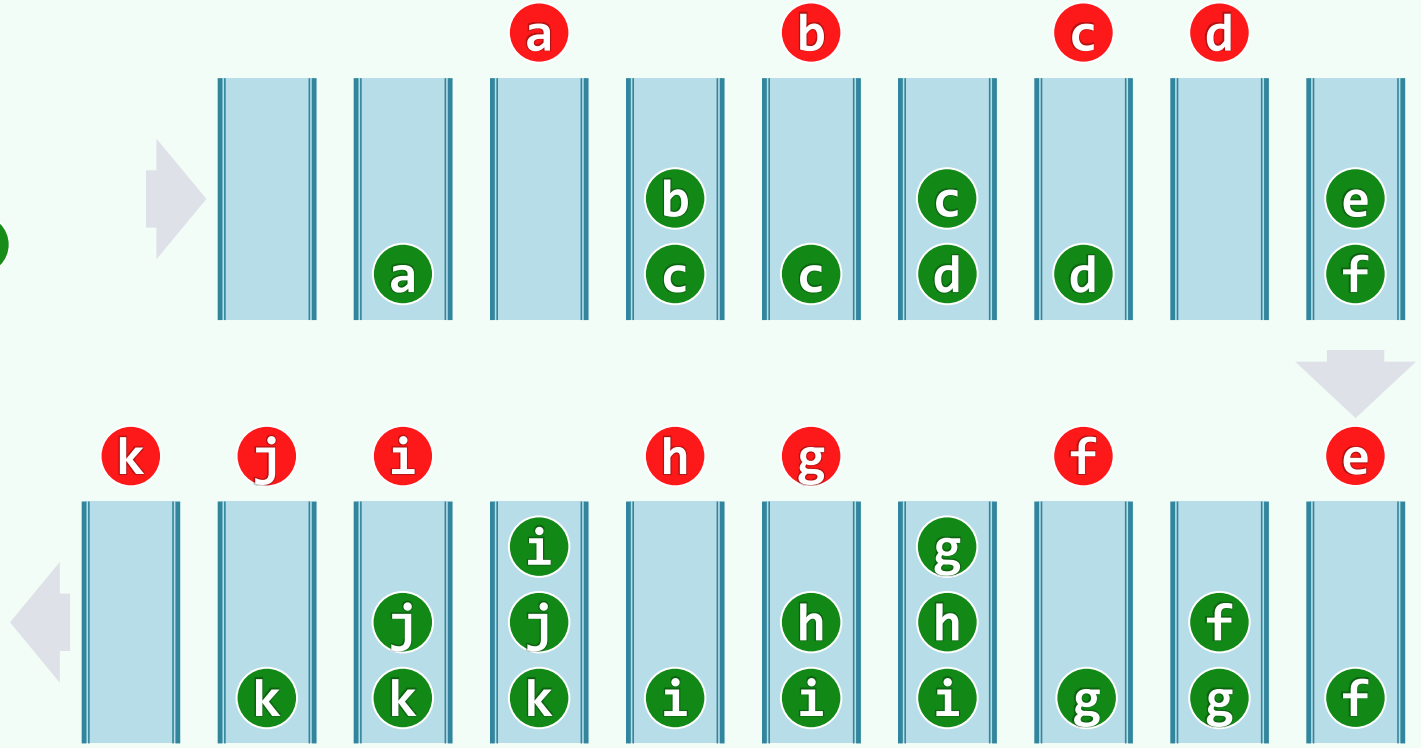
算法实现

```
template <typename T, typename VST>
void levelorder(BNP<T> x, VST& visit) { //二叉树层次遍历
   Queue<BNP<T>> Q; Q.enqueue(x); //引入辅助队列，根节点入队
   while (!Q.empty()) { //在队列再次变空之前，反复迭代
      x = Q.dequeue(); visit(x->data); //取出队首节点并随即访问
      if (x->lc) Q.enqueue(x->lc); //左孩子入队
      if (x->rc) Q.enqueue(x->rc); //右孩子入队
   } //while
} //levelorder
```

实例与效率

- 每个节点或早或迟都会入队、出队并接受访问，各自消耗$O(1)$时间，故总共$O(n)$
- 辅助队列的规模，始终不超过$\lceil n/2 \rceil$，空间复杂度$O(n)$





完全二叉树|complete binary tree

❖ 辅助队列的规模真能达到$\lceil n/2 \rceil$，而且

最晚入队的两个节点中有一个是左孩子 //定义

❖ $2^h \leq n < 2^{h+1}$，$h = \lfloor \log_2 n \rfloor = \mathcal{O}(\log n)$

同规模的二叉树中，高度最小

❖ 将所有节点按层次遍历次序存入向量

- 长度为$\lfloor n/2 \rfloor$的前缀都是内部节点
- 长度为$\lceil n/2 \rceil$的后缀都是叶节点

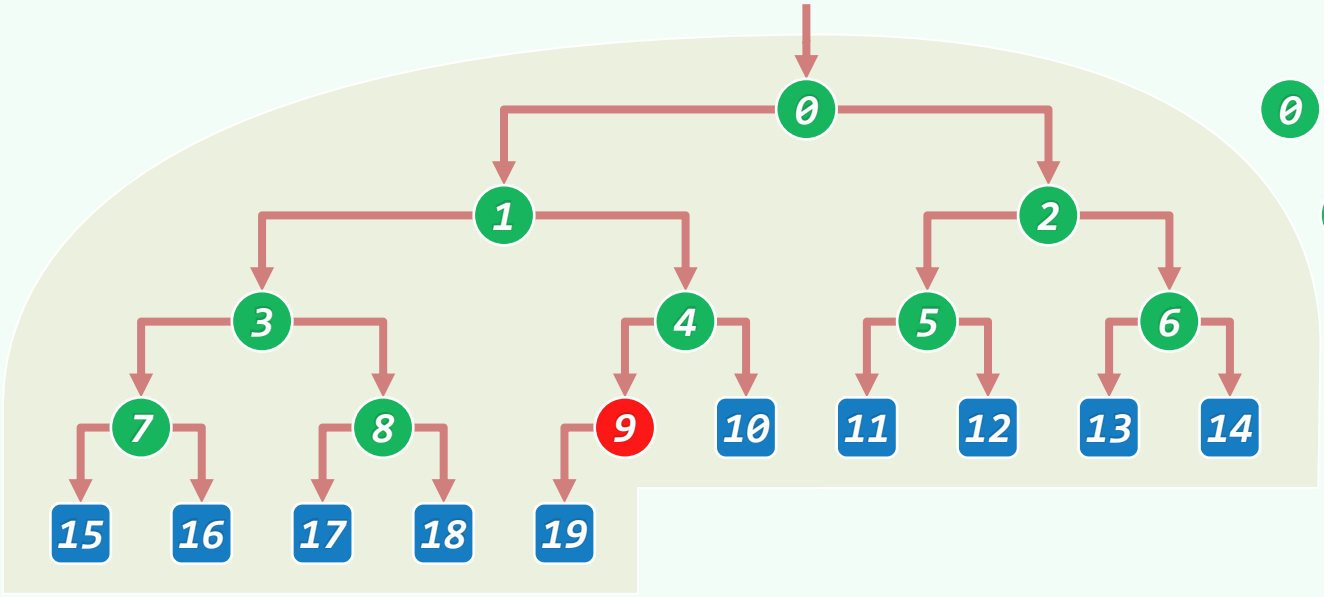
❖ 更重要的是，在父与子、兄与弟之间

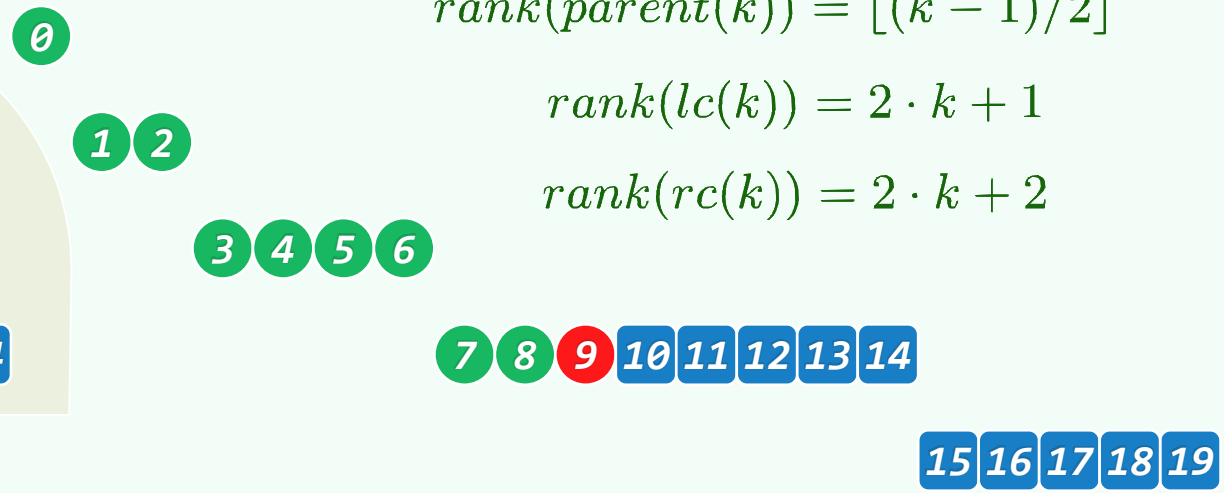
依据秩，有着简明的对应关系

$$rank(parent(k)) = \lfloor (k-1)/2 \rfloor$$

$$rank(lc(k)) = 2 \cdot k + 1$$

$$rank(rc(k)) = 2 \cdot k + 2$$





满树|full binary tree

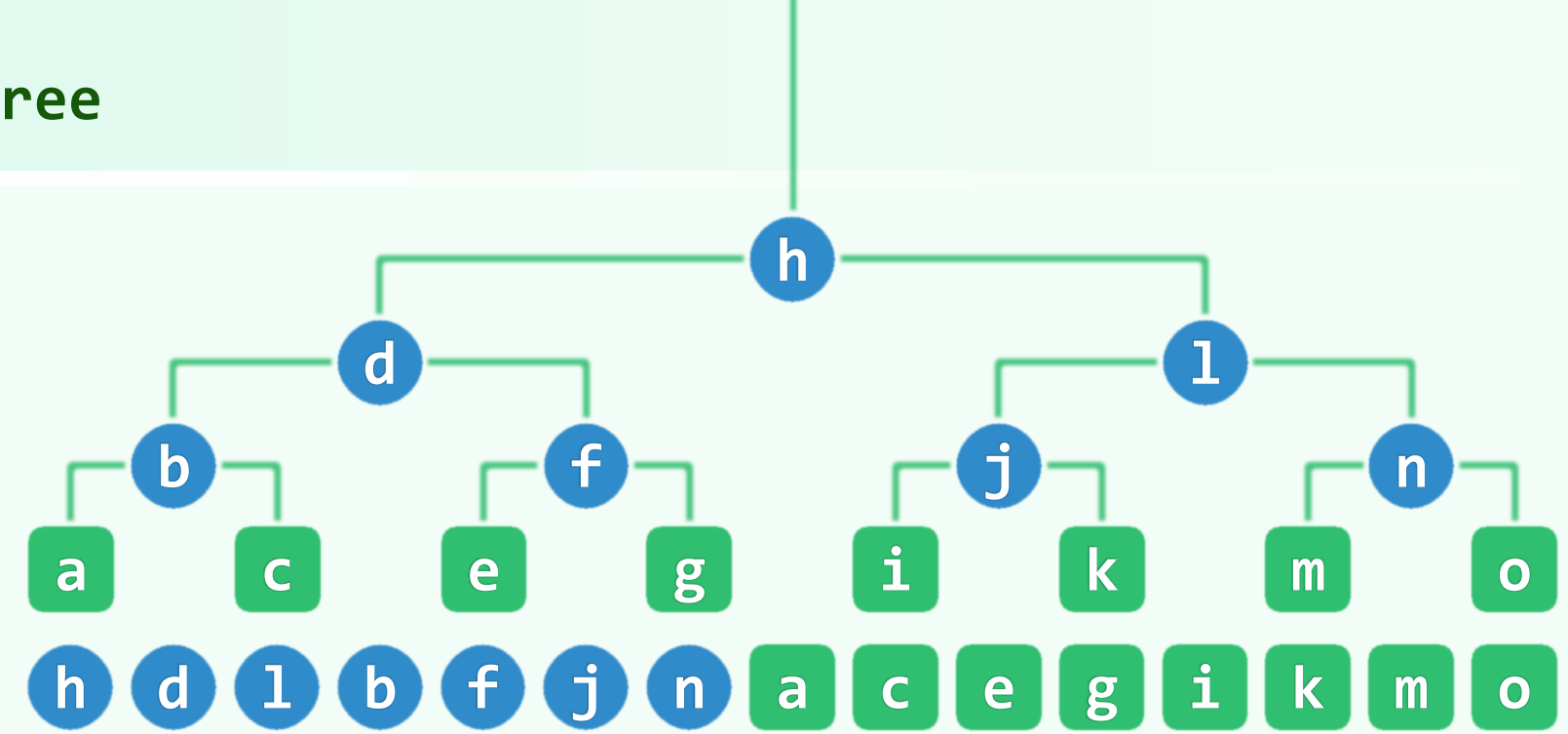
❖ 在高度同为 h 的

所有二叉树中

规模最大者

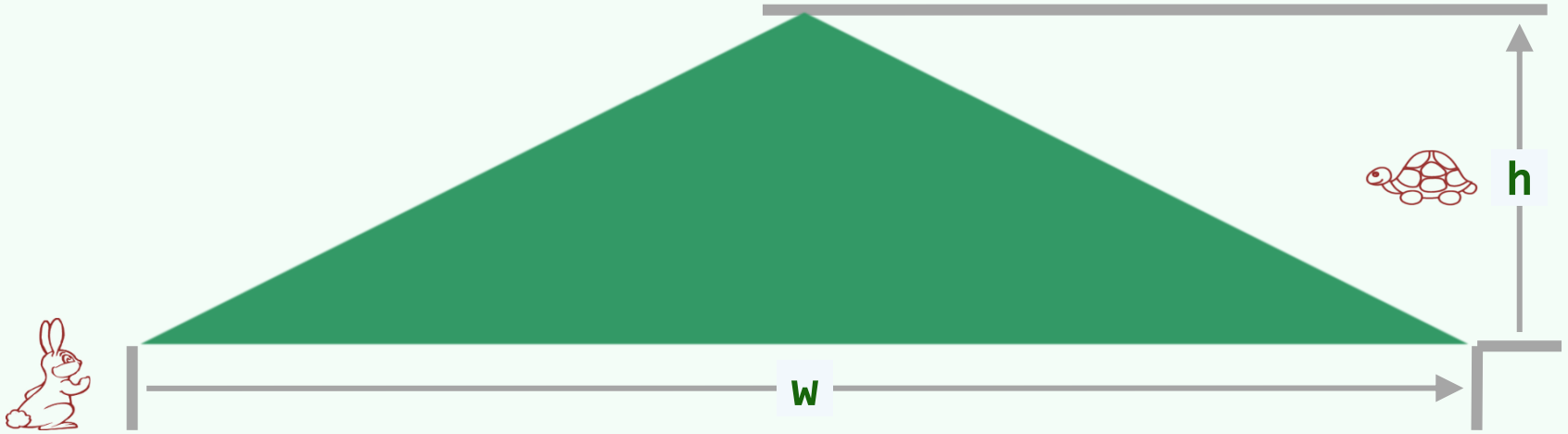
称作满树

$$n = 2^{h+1} - 1$$

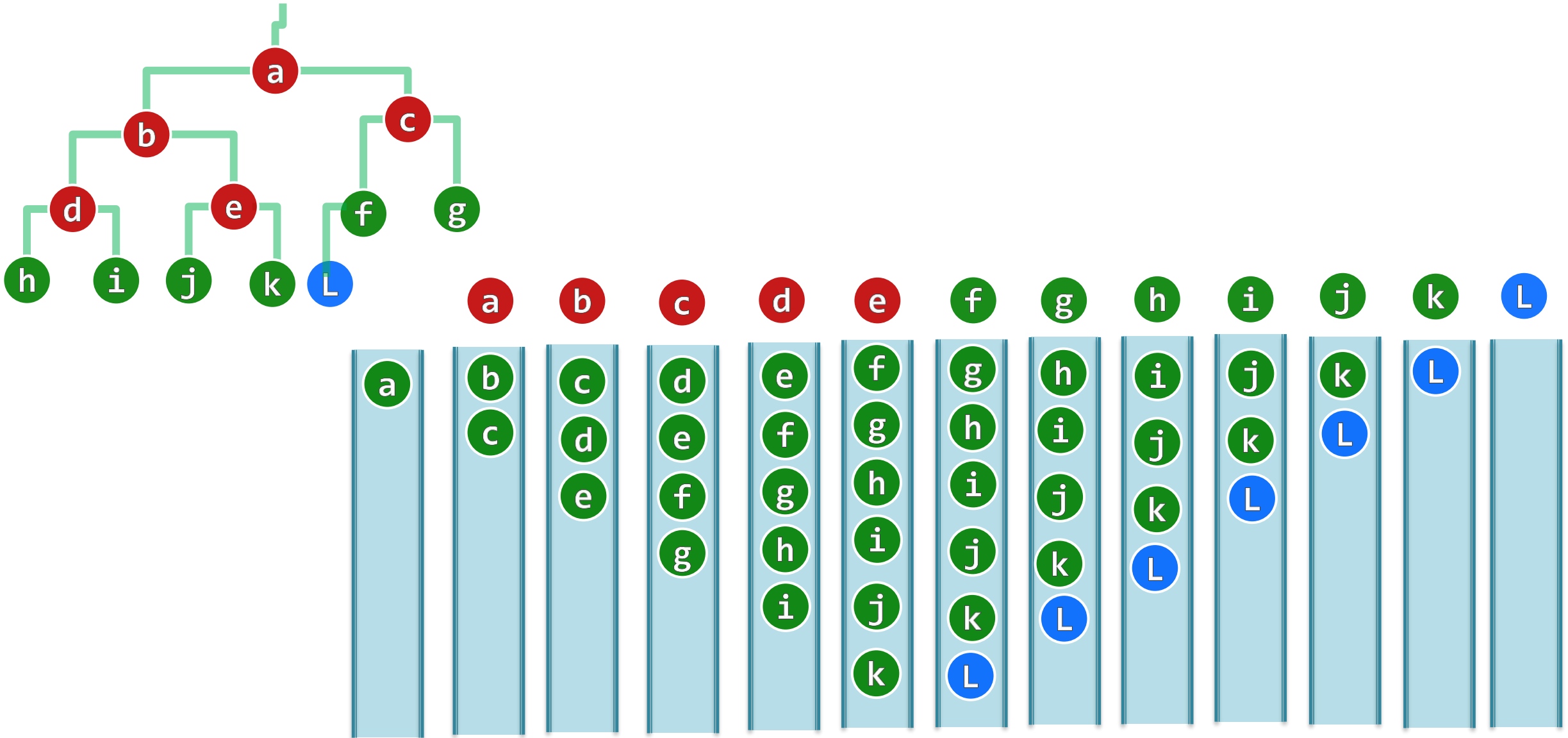


❖ 反过来：

$$h = \log_2(n+1) - 1$$



实例：完全二叉树的层次遍历



二叉树

遍历：重建

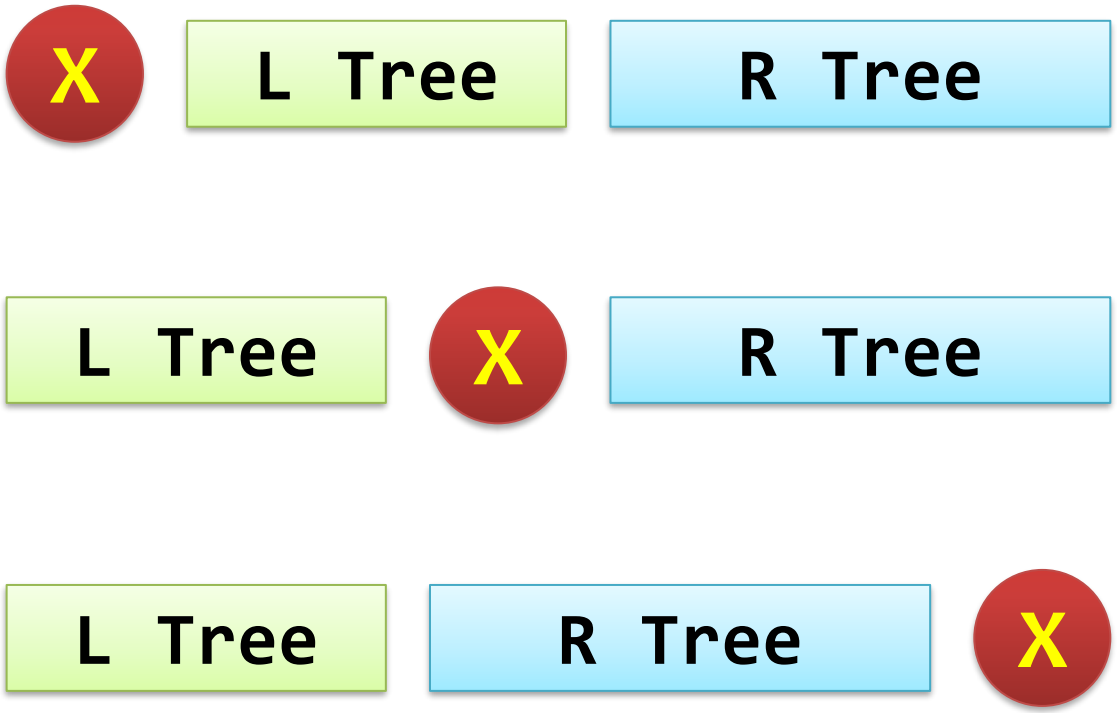
No matter where they take us,

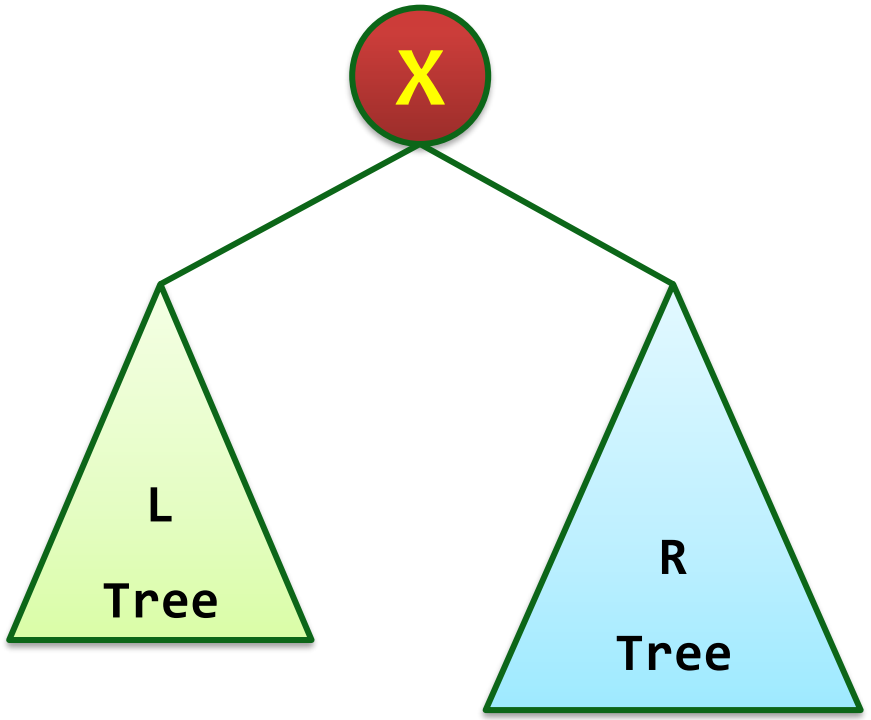
We'll find our own way back.

邓俊辉

deng@tsinghua.edu.cn

观察、观察、观察



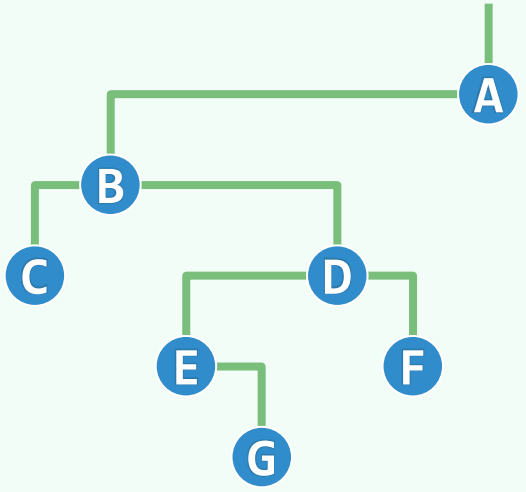


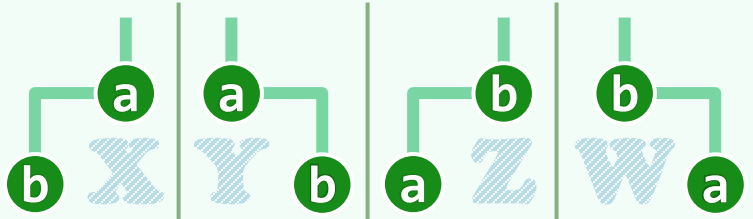
重建|rebuild：由先序|中序|后序|层次遍历序列，反推一棵二叉树的完整结构

- 任何一种序列，都不足以独自地完成重建

- 考查由2个节点构成的二叉树：无非2种遍历序列，却可能有4种姿势
  - 层次序列与先序序列完全一致；后序序列则与它们恰好相反
  - 唯有中序序列，记录了与众不同的信息...

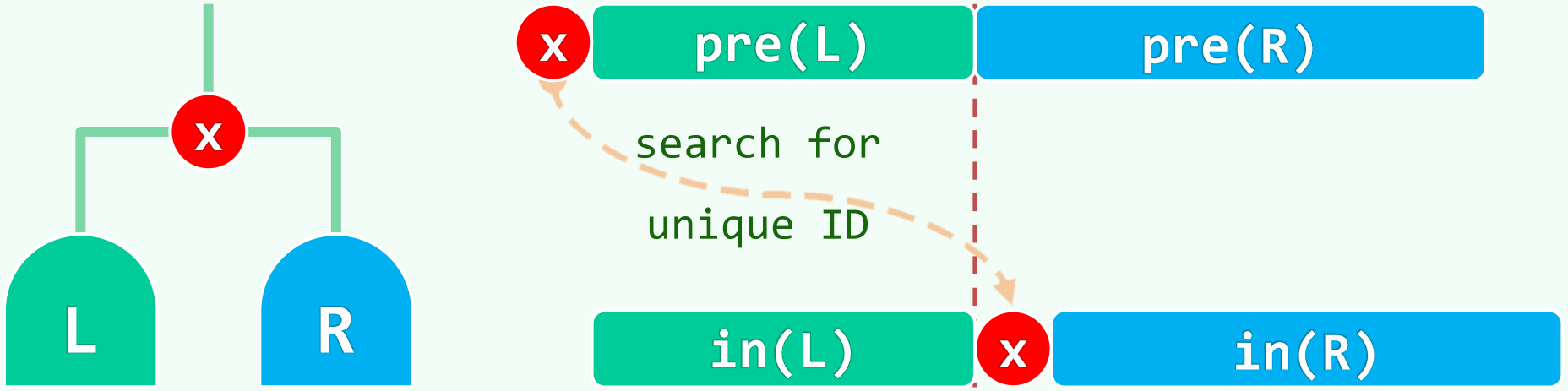
level = ABCDEFG
pre = ABCDEGF
post = CGEFDBA
in = CBEGDFA



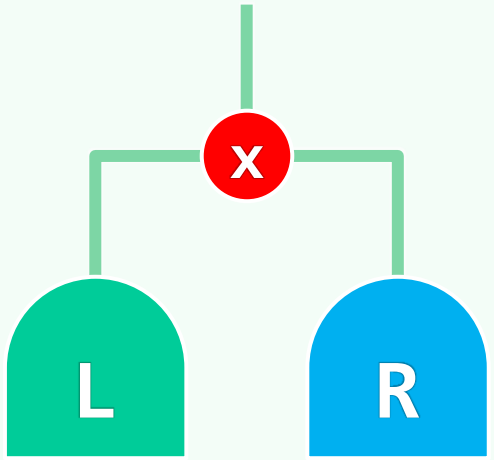


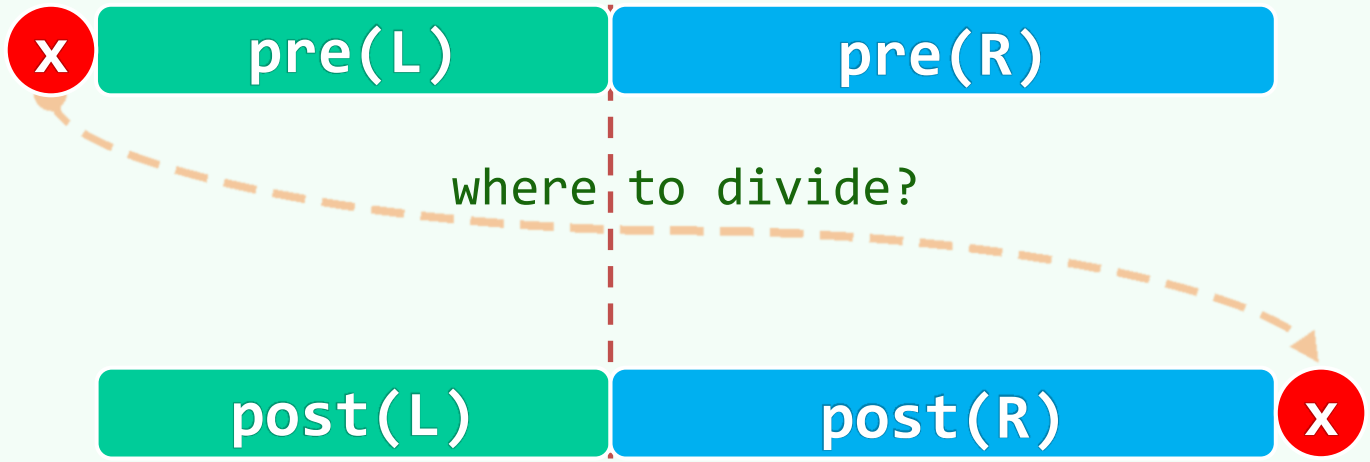
level	ab	ab	ba	ba
pre	ab	ab	ba	ba
post	ba	ba	ab	ab
in	ba	ab	ab	ba

[先序|后序] + 中序 ~ 分治!

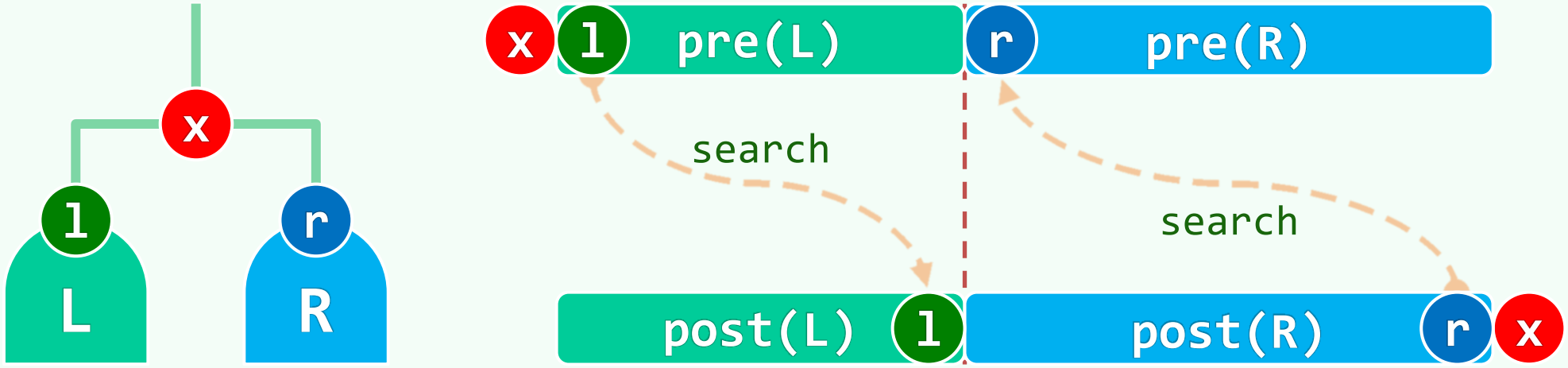


先序 + 后序 ~ 分治?

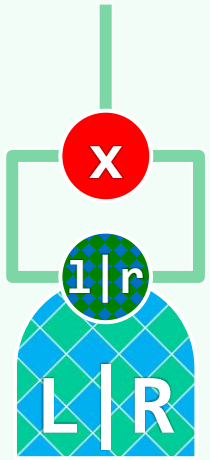


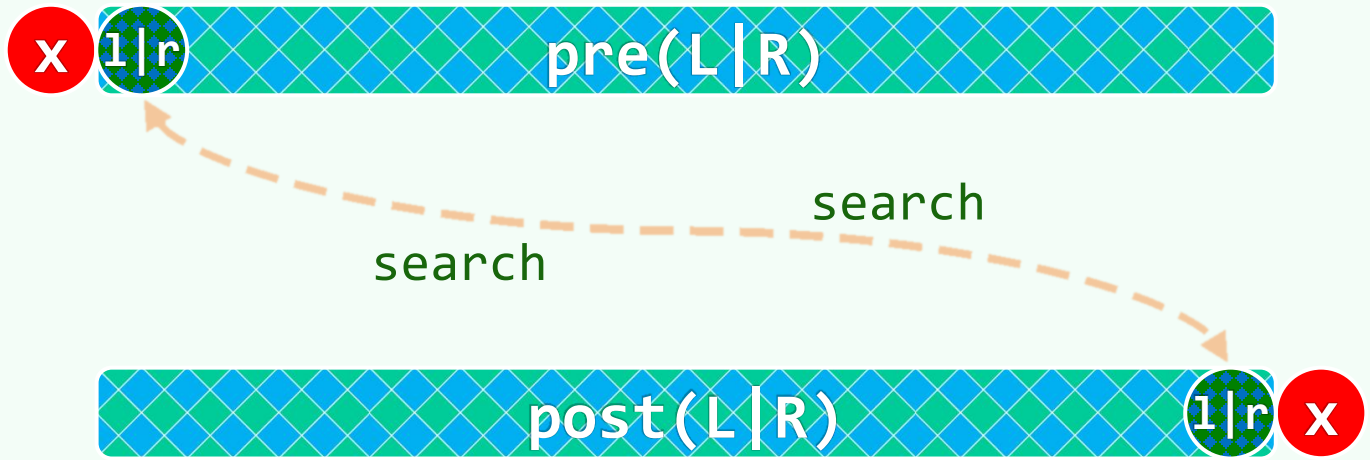


先序 + 后序 ~ 分治?！？

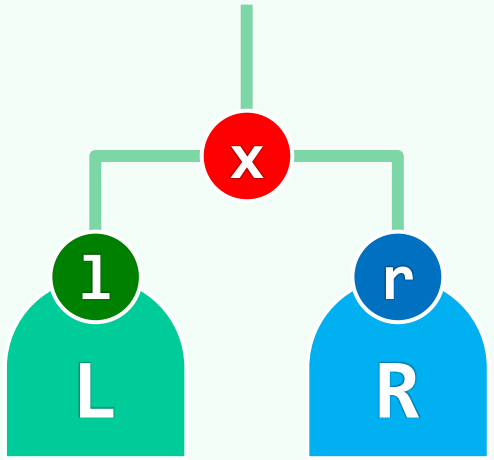


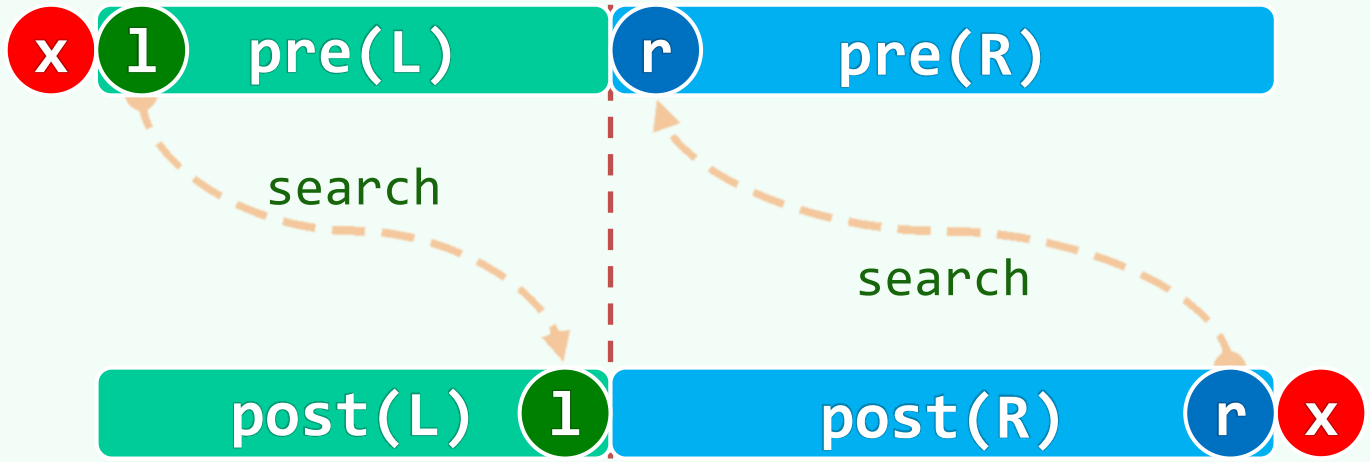
先序 + 后序 ~ 分治|减治





[先序+后序] x 真 ~ 分治!





增强序列：遍历过程中遇到的nullptr，也一并输出为“^”

❖ 由增强的先序遍历序列，可重建原二叉树

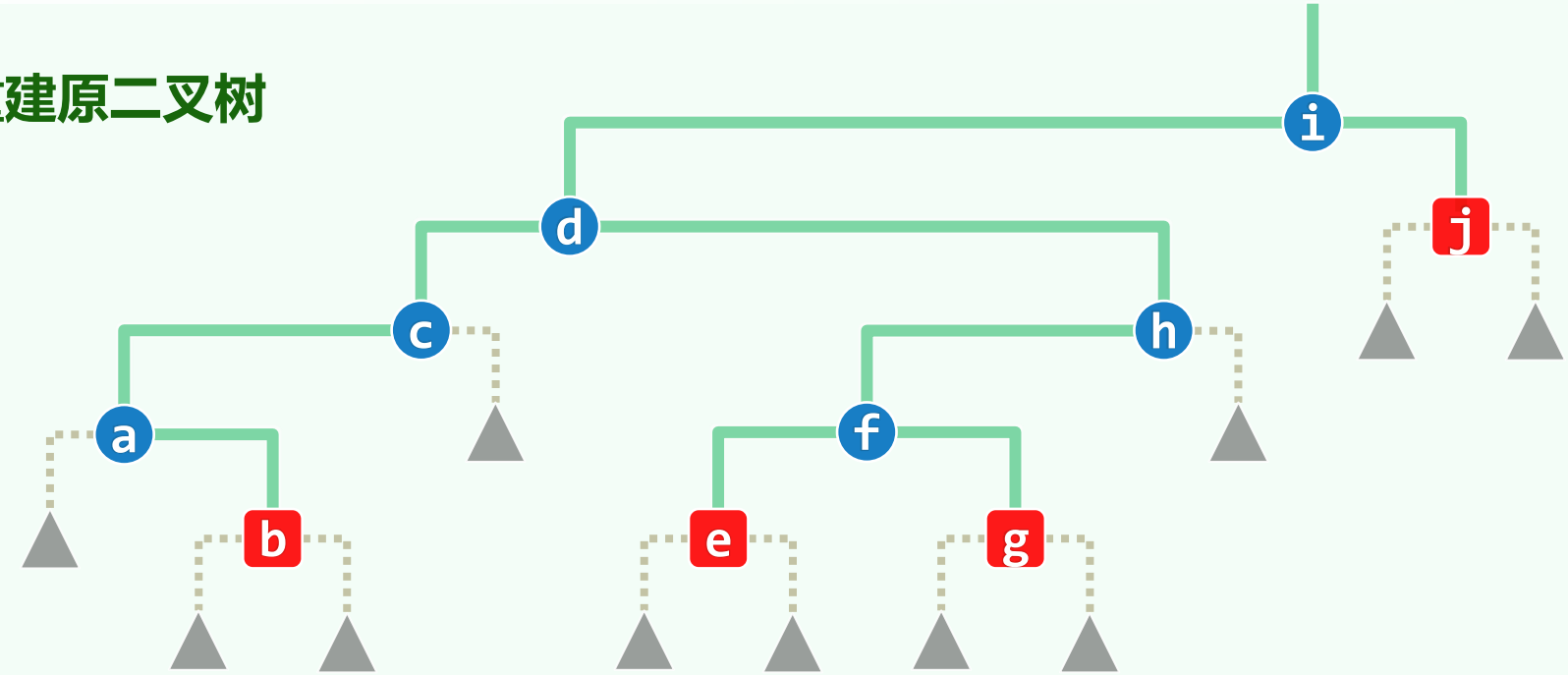
数学归纳：

- 所有局部的“ x ^ ^ ”

——一对应于叶节点x

- 将其替换为“ ^ ”

...



❖ 后序遍历，亦是如此

❖ 而中序遍历，则未必

比如，“ ^ a ^ b ^ ”

preorder : i d c a ^ b ^ ^ ^ h f e ^ ^ g ^ ^ ^ j ^ ^

inorder : ^ a ^ b ^ c ^ d ^ e ^ f ^ g ^ h ^ i ^ j ^

postorder : ^ ^ ^ b a ^ c ^ ^ e ^ ^ g f ^ h d ^ ^ j i

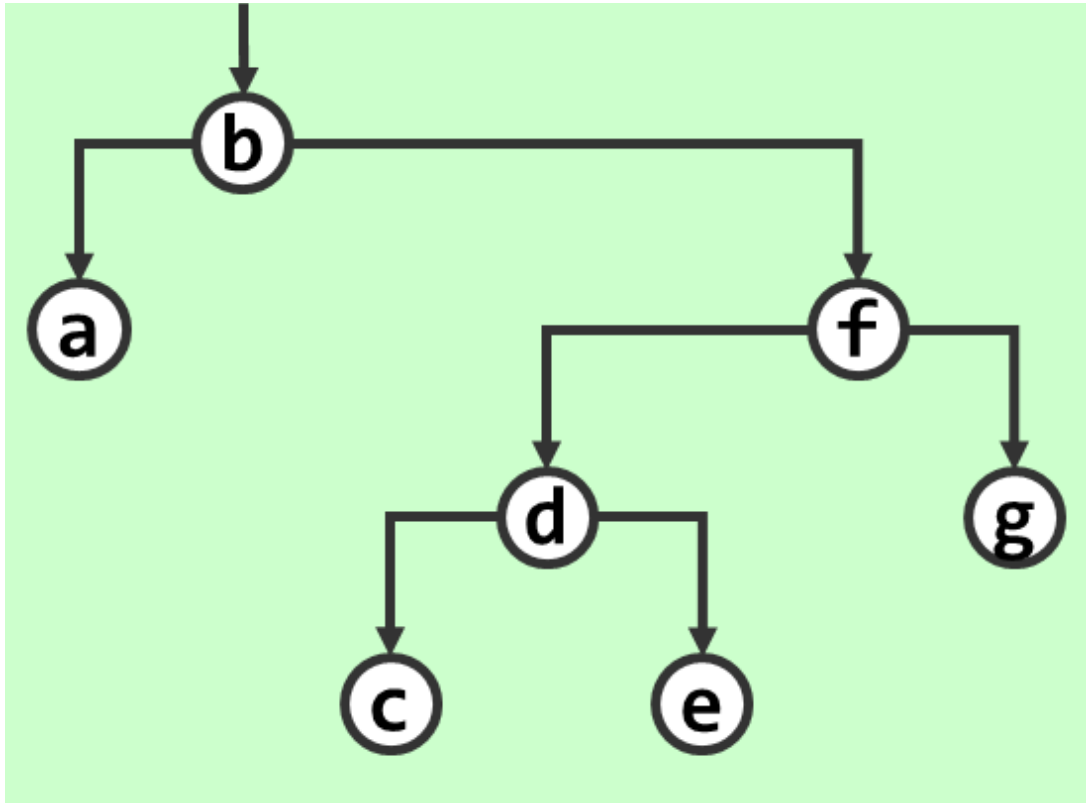
[先序 | 后序] + 中序

❖举例1

- ➢中序：a b c d e f g
- ➢先序：b a f d c e g

❖举例2

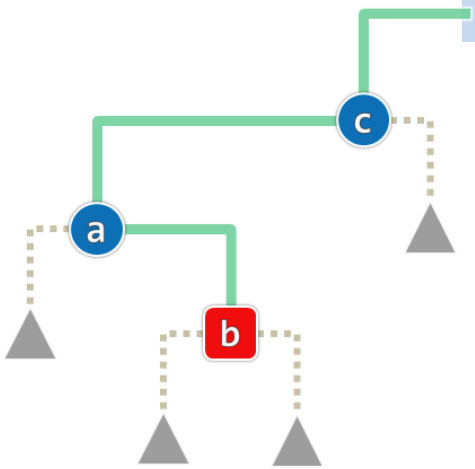
- ➢中序：a b c d e f g
- ➢后序：a c e d g f b



举例：增强序列的重构

preorder : i d c a ^ b ^ ^ ^ h f e ^ ^ g ^ ^ ^ l k j ^ ^ ^ n m ^ ^ p o ^ ^ ^

postorder : ^ ^ ^ b a ^ c ^ ^ e ^ ^ g f ^ h d ^ ^ j ^ k ^ ^ m ^ ^ o ^ p n l i



05. Binary Tree

A. 树（术语和ADT）

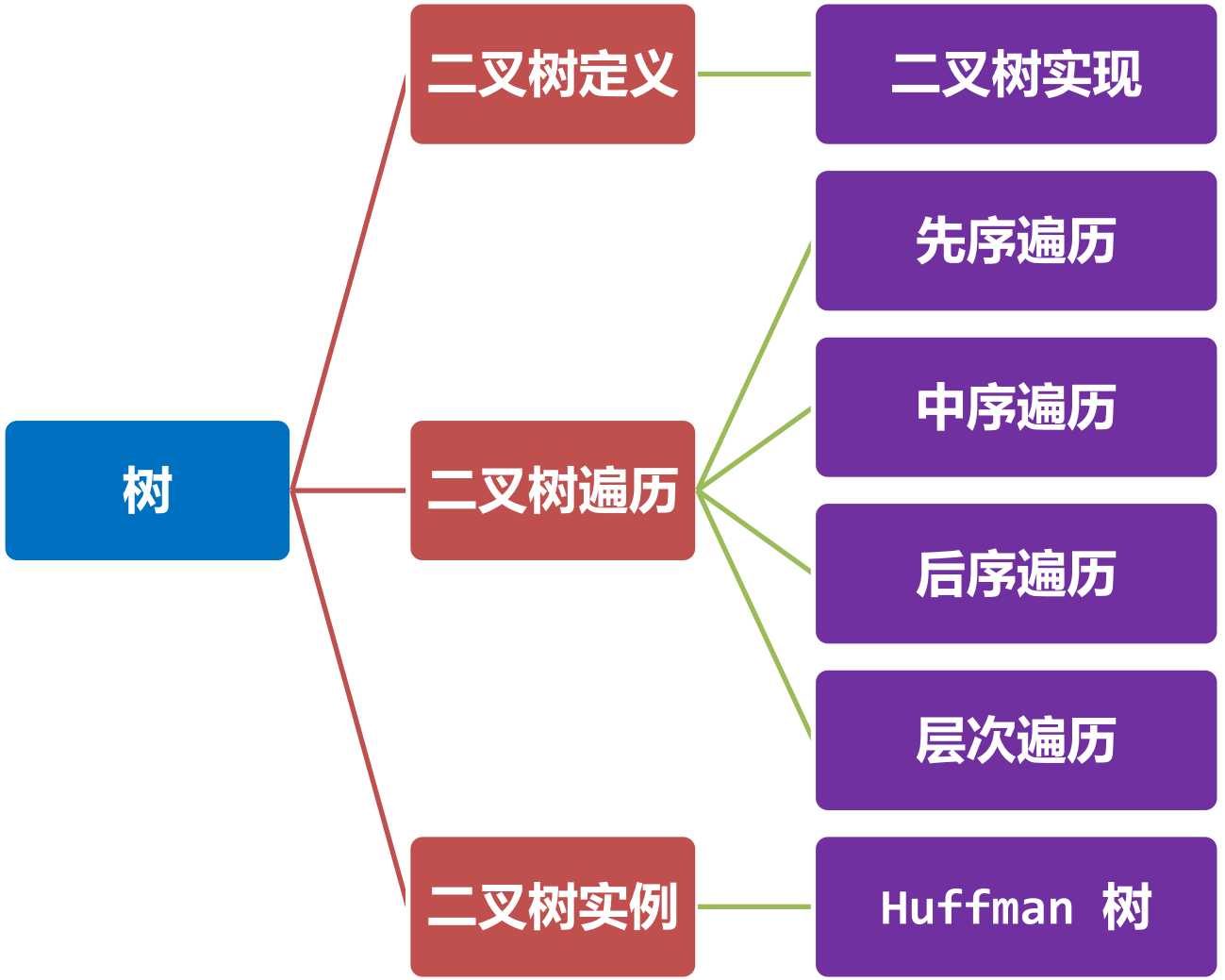
B. 二叉树（转换与实现）

C. 二叉树遍历

- 先序、中序、后序
- 层次、重构

D. Huffman 树

E. 下界与最优



05.Binary Tree

D-Huffman编码树

二叉树

Huffman编码树：问题与算法

邓俊辉
deng@tsinghua.edu.cn

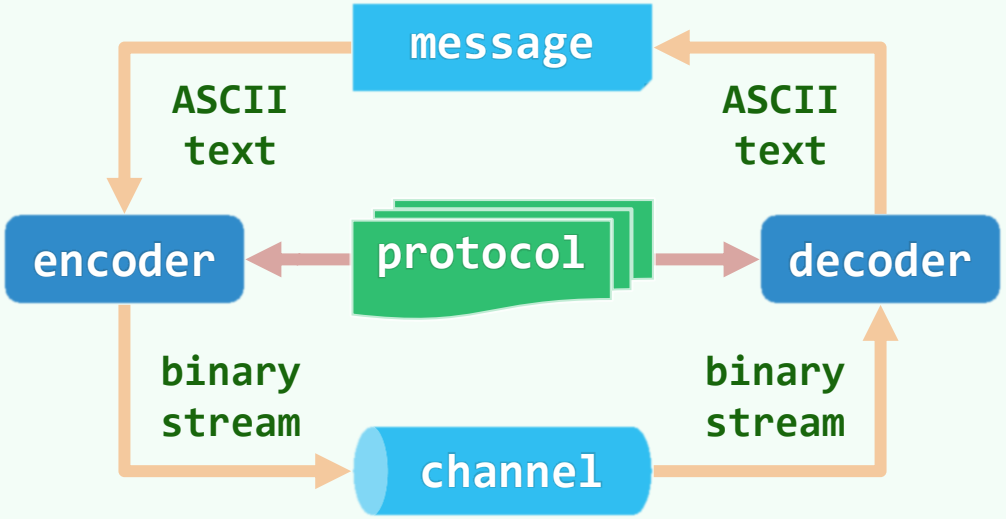
数字通讯中的编码

❖ 组成数据文件的字符

均来自约定的字母表Σ，|Σ|=n

❖ 字符 ~ 互异的二进制编码

字符串 ~ 字符编码的串接

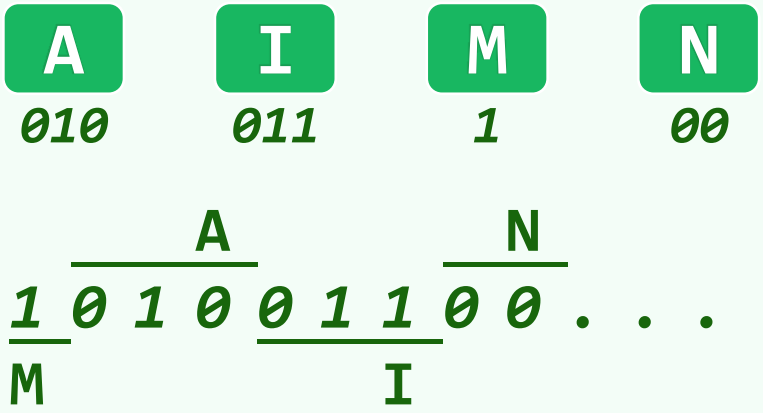


❖ 句读难题 ~ 歧义：

某字符的编码，可能恰是另一字符编码的前缀

❖ 比如，倘若将I的编码改作001，那么

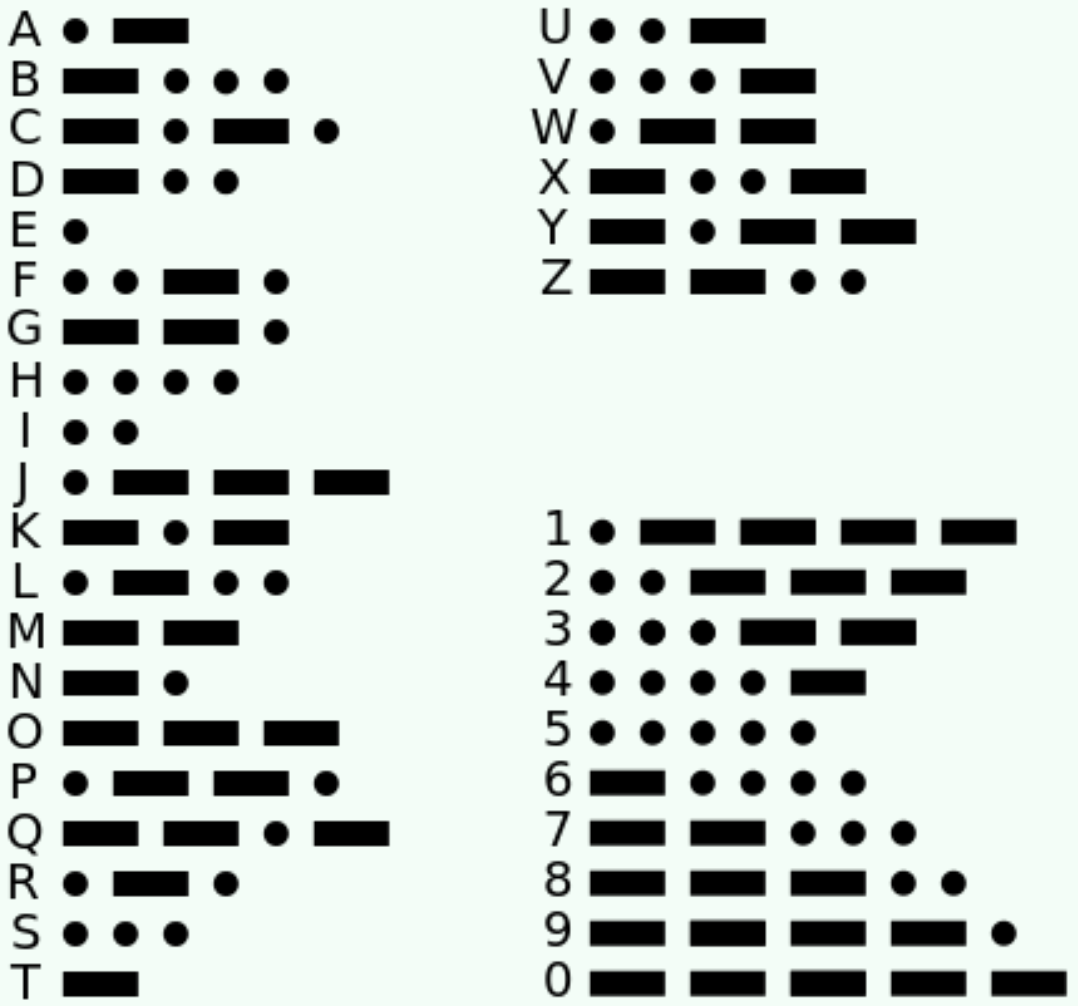
解码者应当把001译作I，还是NM?



摩尔斯电码

.. .-.. --- ...- . -.-. .... .. -. .-

- 短促的点信号“·”，读“滴”；
- 保持一定时间的长信号“–”，读“嗒”
- 间隔时间：
  - 滴=1t
  - 嗒=3t
  - 滴嗒间=1t
  - 字符间=3t
  - 单词间=7t



International Morse Code

1. The length of a dot is one unit.
2. A dash is three units.
3. The space between parts of the same letter is one unit.
4. The space between letters is three units.
5. The space between words is seven units.

前缀无关编码|prefix-free coding

❖ 将字符分别作为一个叶节点

随意地组成一棵真二叉树

❖ 字符x ~ 从根到x的通路 ~ x的编码

L|R ~ 0|1

❖ 在每一条通路上，除终点外都是内部节点

如此，自然就保证了前缀无关

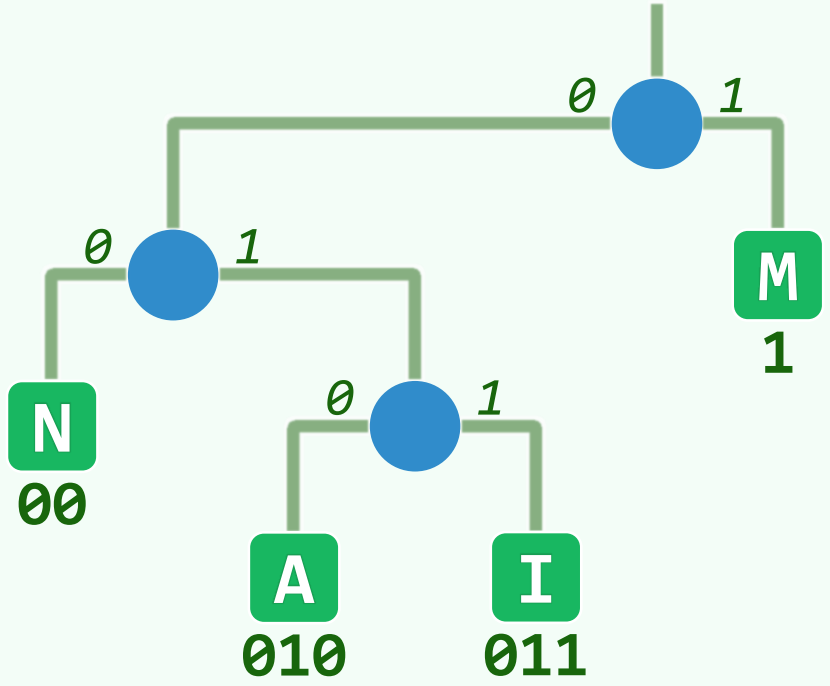
❖ PFC并不唯一

何者的编码效率最高、成本最低?

首先要定义好衡量与评估的尺度

❖ 编码长度|code length

- 字符 ~ 对应叶节点的深度
- 整体 ~ 所有字符的编码长度总和



最优编码树|optimal coding tree

- CL更短 ~ 更优

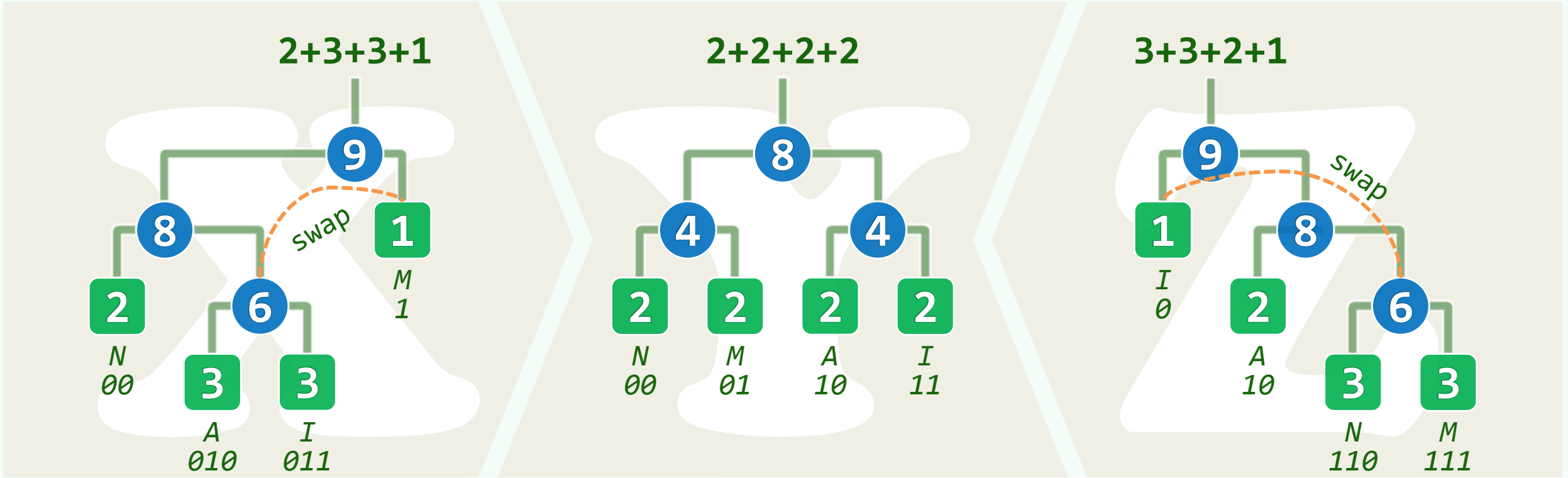
  CL最短 ~ 最优

- OCT必然存在，虽未必唯一

- 叶子只能出现在倒数两层

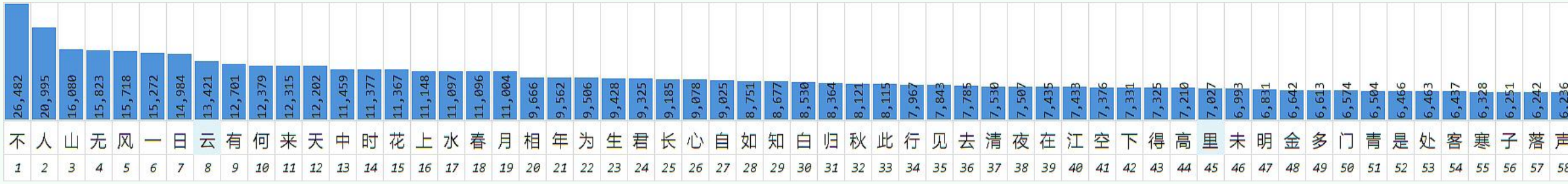
  否则，通过节点交换，总能降低CL

- 真的完全树即是OCT...



频数|frequency

❖



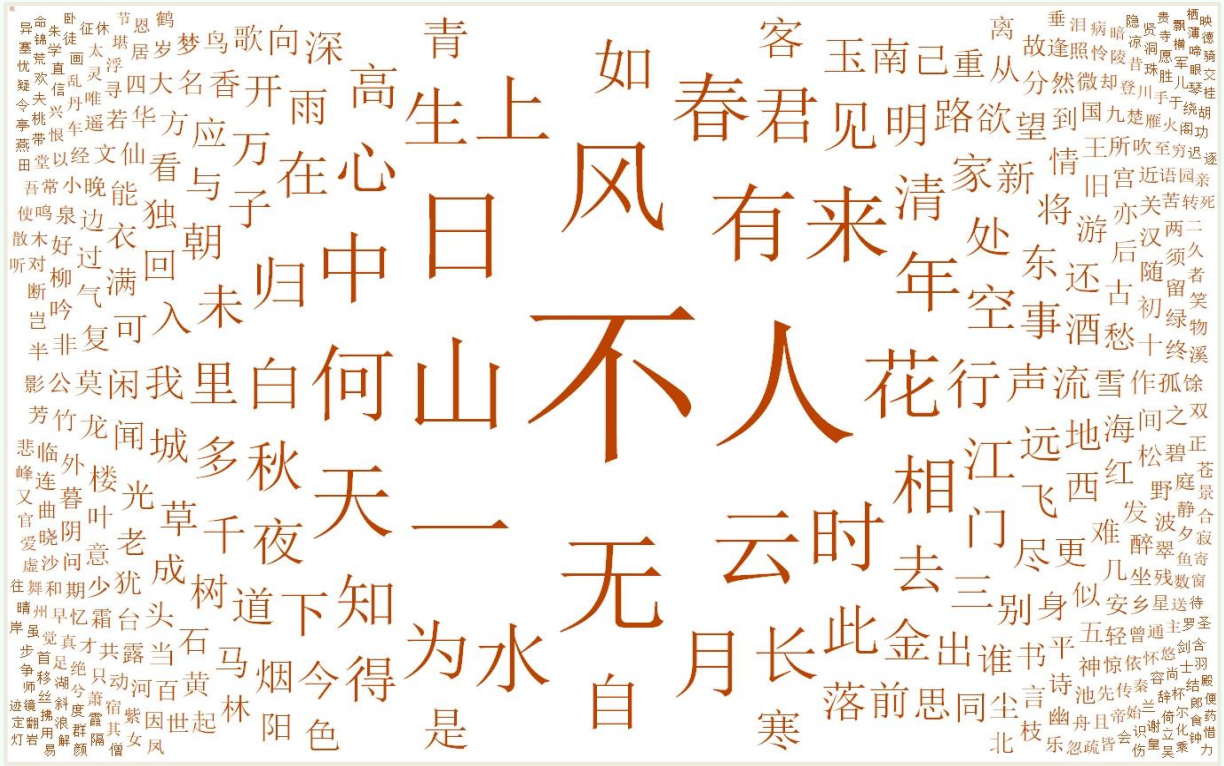
❖ **在不同的文化、时代、专业领域中**

字符的出现概率或频度不尽相同

甚至，往往相差极大...

❖ **已知各字符的期望频数**

如何构造最优编码树?


不 人 山 一 无 风 有 来 日 何 天 中 时 云 春 君 上 生 如 青 客 玉 南 已 重 见 明 路 欲 望 到 国 九 楚 雁 火 至 穷 迟 语 园 转 死 逐 亲 二 久 者 笑 物 绿 终 溪 余 孤 作 雪 流 声 行 花 空 事 酒 愁 十 古 还 东 将 游 后 随 初 亦 关 汉 两 须 留 清 家 新 情 王 所 吹 近 宫 旧 处 年 相 江 门 去 三 别 身 尽 更 似 安 乡 星 送 待 圣 罗 主 含 羽 殿 便 药 惜 力 钟 食 乘 吴 立 倚 辞 杯 尔 化 结 郎 士 剑 悠 怀 依 惊 神 平 诗 池 先 传 秦 始 识 伤 谢 里 且 帝 会 皆 忽 疏 舟 幽 乐 北 枝 言 尘 同 思 前 落 寒 自 月 长 此 金 出 谁 书 水 为 是 海 红 松 碧 庭 静 波 夕 鱼 窗 寄 寂 双 正 苍 景 合 发 醉 翠 野 西 难 飞 远 地 问 之 徐 隐 贵 寺 愿 胜 手 于 横 军 儿 琴 统 阁 迟 逐 亲 死 二 久 者 笑 物 绿

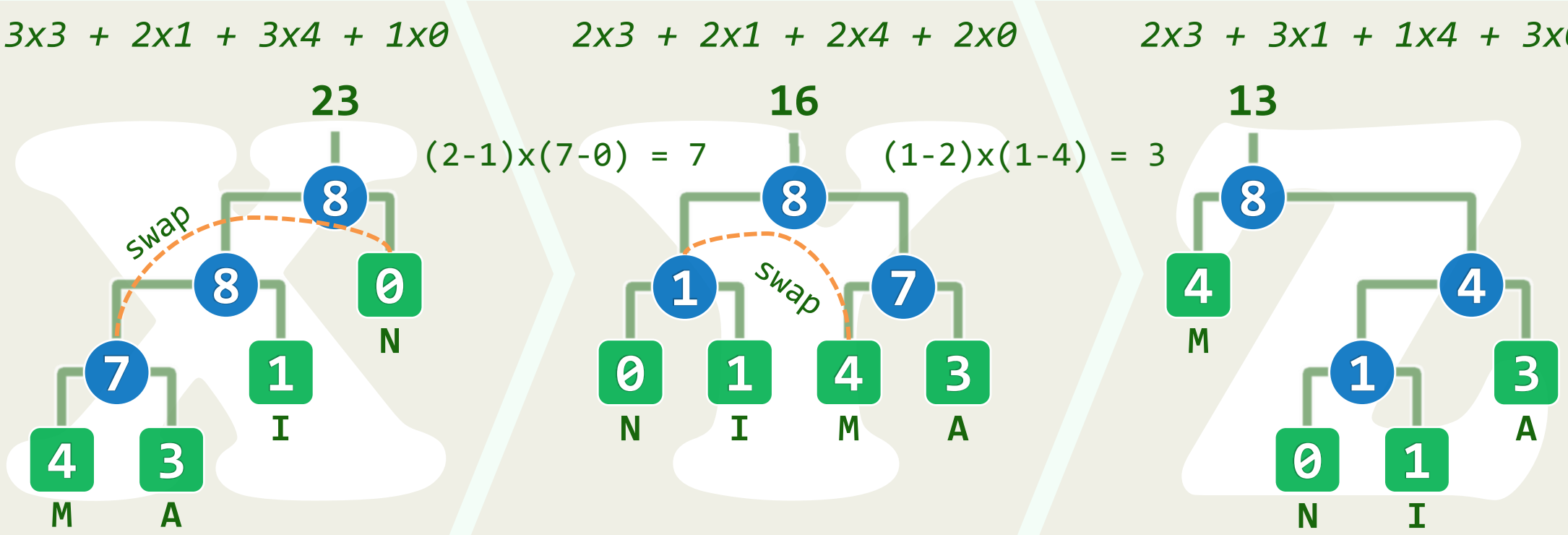
带权编码长度|weighted coding length

❖ 以字符的频数做加权，更符合实际

❖ 此时，完全树未必就是OWCT

❖ 比如，对于MAMMAMIA

A、I、M、N的频数分别为4、3、1、0



最优带权编码树|optimal weighted coding tree

- 叶子节点的权重 = 字符的频数

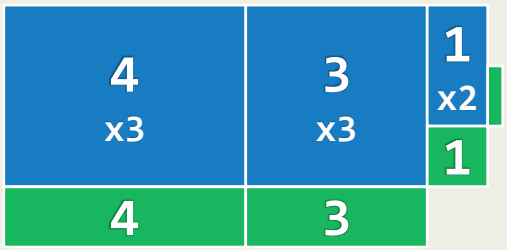
  内部节点的权重 = 两个孩子的权重之和

- 字符的WCL总和 = WCL = 内部节点的权重总和

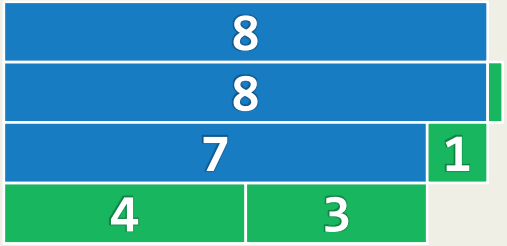
- 所谓OWCT，就是WCL最小者

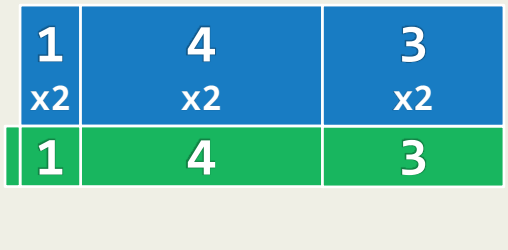
- 通过适当交换，同样可以缩短WCL

  频数高|低的字符，尽可能放在高|低处

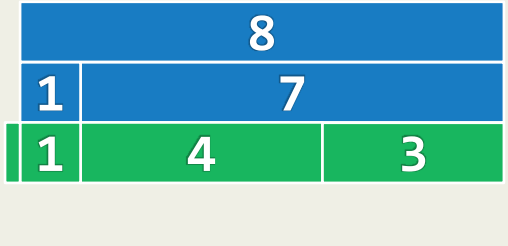


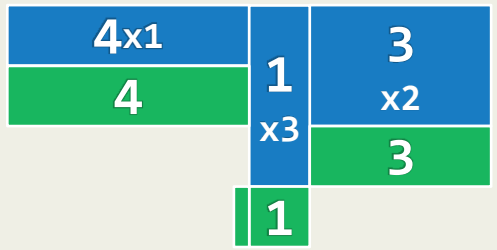
X



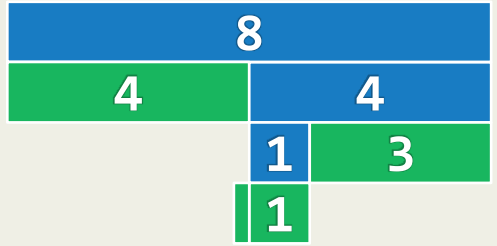


Y





Z



Huffman算法，1952

- 贪心策略：自底而上地逐个拼接：频数小|大的字符，更早|晚拼接
- 字符各自成一棵树，权重即其频数

  n棵树组成森林F

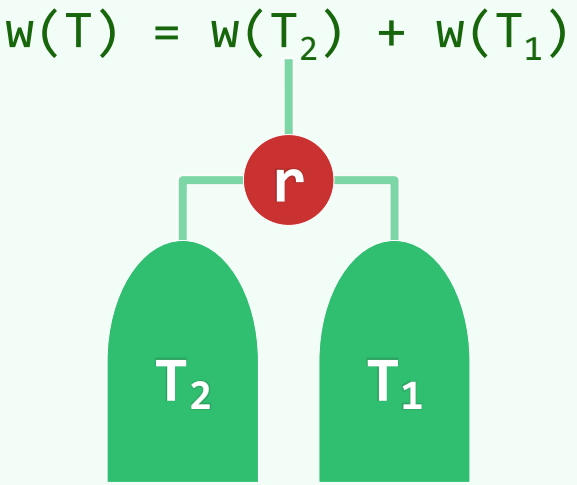
  while（F中的树不止一棵）

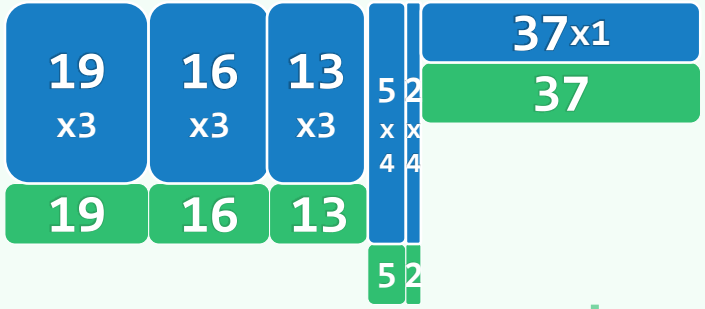
    取出频率最小的两棵树：T_1和T_2

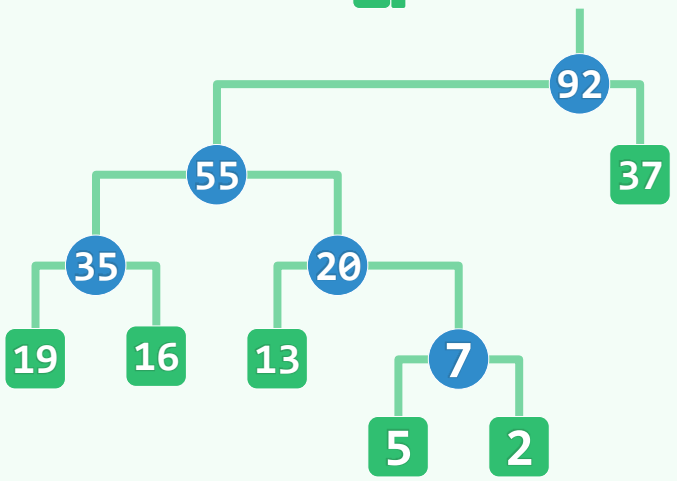
    将它们合并成一棵新树T

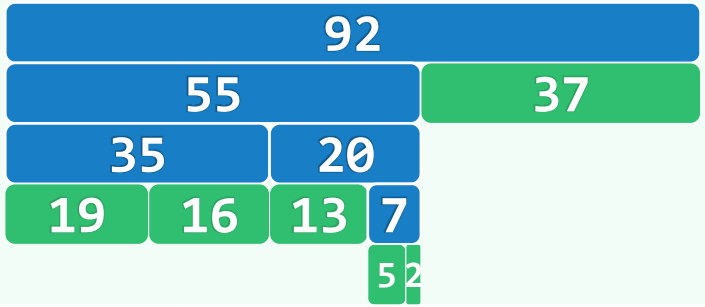
    - 以T_2、T_1为左、右子树
    - 权重取作T_2与T_1权重之和

  最终唯一剩下的那棵树，便是一棵OWCT









- 贪心策略通常并不能保证得到最优解，Huffman的算法是个例外吗？

二叉树

Huffman编码树：正确性

邓俊辉
deng@tsinghua.edu.cn

不确定中的确定

- 当我们宣称Huffman树是OWCT时

  究竟是指哪一棵？毕竟，OWCT通常都有多棵

  - 比如，深度相同的子树总是可以互换
  - 甚至，深度不同者，或许也能互换

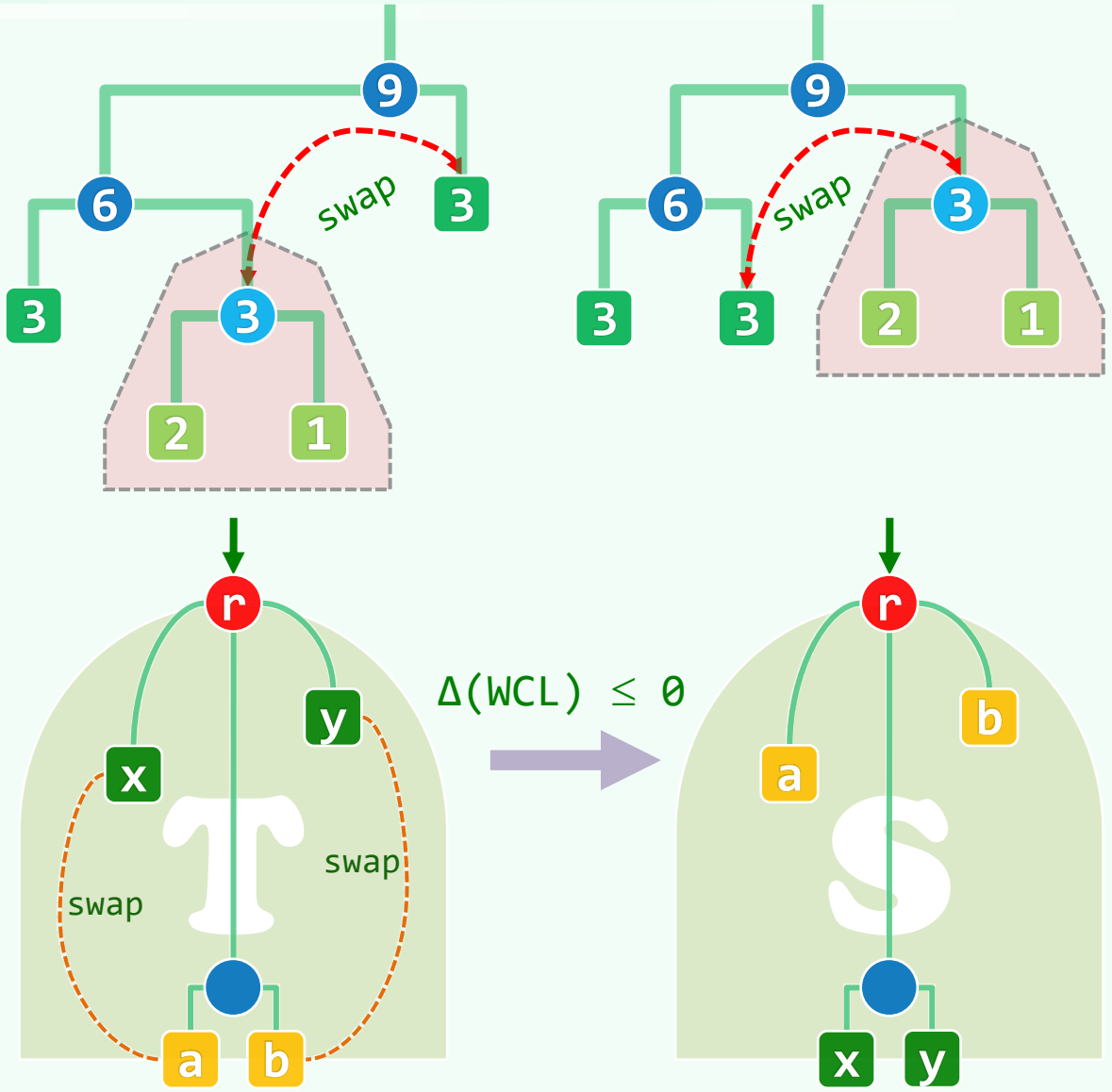
- 对某些细节，Huffman算法也未明确

- 约定：就以我们实现的版本为准

  其中最重要地：左兄弟均不小于右兄弟

- 【最小字符成对】

  频数最低的字符x和y，必在某棵OWCT中是兄弟

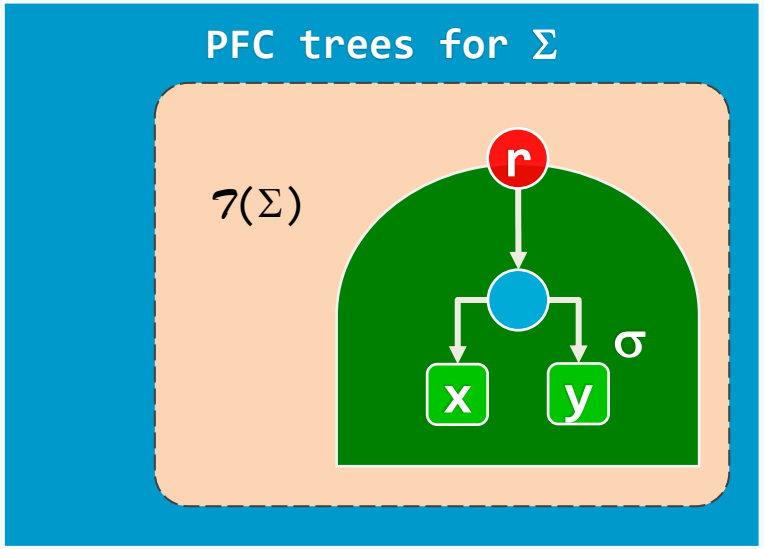


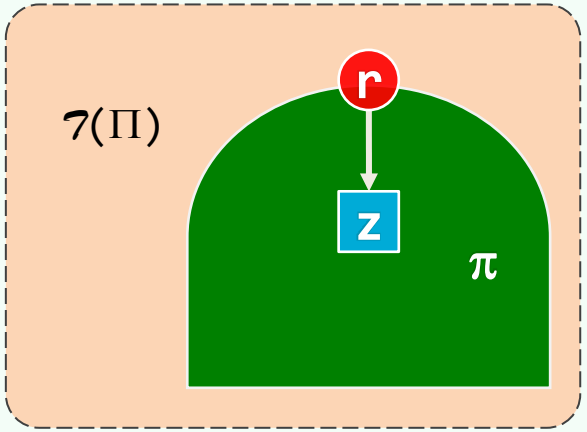
数学归纳：Huffman算法所生成的，必是一棵OWCT

- 假设$|\Sigma|<n$时均成立；考查任何$|\Sigma|=n$
- 引入字符 z ： $w(z)=w(x)+w(y)$

  并得到字符集 $\Pi=(\Sigma\backslash\{x,y\})\cup\{z\}$
- $\mathcal{T}(\Sigma)$ ： Σ所有满足“x与y成对”的编码树

  $\mathcal{T}(\Pi)$ ： Π的所有编码树
- 两个集合之间存在一一对应关系，而且
  - 若 $\sigma\in\mathcal{T}(\Sigma)$ 对应于 $\pi\in\mathcal{T}(\Pi)$
  - 则有 $WCL(\sigma)-WCL(\pi)\equiv w(x)+w(y)=w(z)$
- 差距固定，故知最优必对应于最优





二叉树

Huffman编码树：算法实现

邓俊辉
deng@tsinghua.edu.cn

超字符 + 树

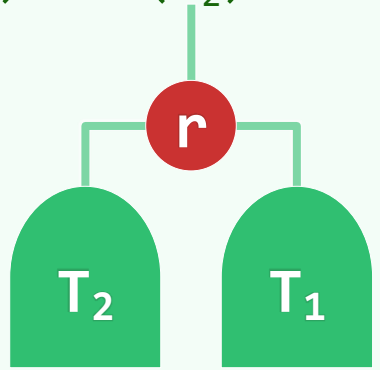
```
❖ struct HuffChar { //Huffman字符
    char ch; uint32_t weight; //字符、频数
    HuffChar( uint32_t w = 0, char c = '^' ) : weight( w ), ch( c ) {}; //构造
    HuffChar( const HuffChar& hc ) : weight( hc.weight ), ch( hc.ch ) {}; //复制
    bool operator<( const HuffChar& hc ) //比较器
       { return weight > hc.weight; } //故意大小颠倒
  }; //HuffChar

❖ using HuffTree = BinTree< HuffChar >; //Huffman树：由BinTree派生
  auto Weight = [](auto T) { return T.root()->data.weight; }; //根的频数 = 子树权重
```

构造Huffman树：贪心合并

```
❖ HuffTree huffTree( HuffForest& F ) {
    while ( 1 < F.size() ) { //每迭代一步，森林中都会减少一棵树
        HuffTree T1 = F.delMax(), T2 = F.delMax(), S; //取出权重最小的两棵树，合并成S
        S.insert( HuffChar( Weight(T2) + Weight(T1) ) ); //权重取作子树权重之和
        S.attach( T2, S.root() ); S.attach( S.root(), T1 ); //T2权重不小于T1
        F.insert( S ); //再插回森林
    } //森林中最终唯一所剩的那棵树，即Huffman编码树
    return F.delMax(); //且层次序列必然单调非增
} //huffTree
```

$w(T) = w(T_2) + w(T_1)$



编码表 ~ 散列表

```
❖ #include "HashTable.h" //用HashTable（第09章）实现
  using HuffTable = Hashtable< char, char* >; //Huffman编码表

❖ HuffTable* huffCodeTable( HuffTree tree ) { //将各字符编码，统一存入
     Hashtable table; //以散列表实现的编码表
     Stack<char> theseus; theseus.push('x');
     huffCT( theseus, table, tree.root() );
     return table;
  }; //huffCodeTable
```

导出编码表 ~ 递归遍历

```
void huffCT( Stack<char> theseus, Hashtable& table, BNP<HuffChar> v ) {
   if ( IsLeaf( v ) ) { //每遇到一个叶节点，便导出对应的编码串，并添加到编码表中
      int n = theseus.size() - 1; Stack<char> sueseht;
      for ( Rank k = 0; k < n; k++ ) sueseht.push( theseus.pop() );
      char* code = new char[n + 1]; code[n] = '\0';
      for ( Rank k = 0; k < n; k++ ) theseus.push( code[k] = sueseht.pop() );
      table.put( v->data.ch, code );
   } else { //否则，继续递归地遍历
      if ( v->lc ) { theseus.push('0'); huffCT( theseus, table, v->lc ); } //0左
      if ( v->rc ) { theseus.push('1'); huffCT( theseus, table, v->rc ); } //1右
   } //else
   theseus.pop();
} //huffCT
```

森林 = 优先级队列

❖ 运行时间决定于Huffman森林的实现方式，核心的操作是delMax()与insert()

❖ 但List不能兼顾两类操作，导致每一步耗时 $\mathcal{O}(n)$，整体 $\mathcal{O}(n^2)$

```
using HuffForest = ListPQ< HuffTree >; //Huffman森林：暂用列表实现
template <typename T> class ListPQ : public PQ<T>, public List<T> {
public: void insert( T e ) { insertLast( e ); } //新元素直接插至队尾
        T delMax() { return remove( List::getMax() ); } //删除最优先的元素
}; //ListPQ
```

❖ 改用高级数据结构，可保证每一步仅耗时 $\mathcal{O}(\log n)$，整体 $\mathcal{O}(n \cdot \log n)$：可惜，远水不解近渴

```
using HuffForest = ComplHeap< HuffTree >; //日后可改用完全二叉堆实现
using HuffForest = LeftHeap< HuffTree >; //也可改用左式堆实现
```

❖ 有意思的是，直接使用最简单的数据结构，也同样可以做到 $\mathcal{O}(n \cdot \log n)$ ...

森林 = 栈+队列：原理

- 按频数排序后

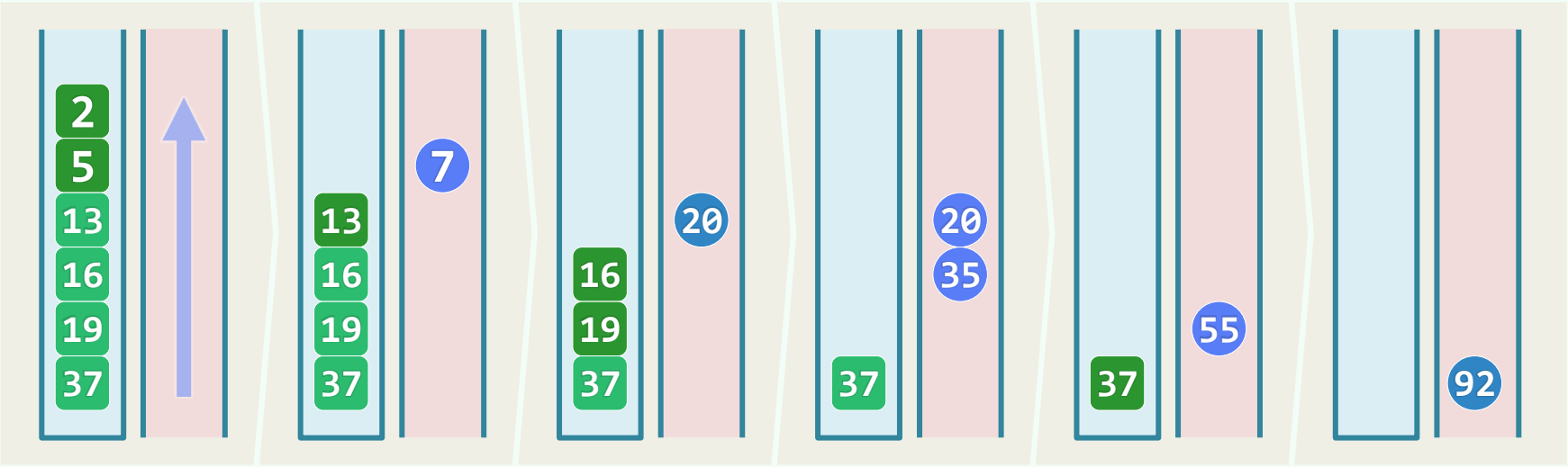
  所有字符入栈

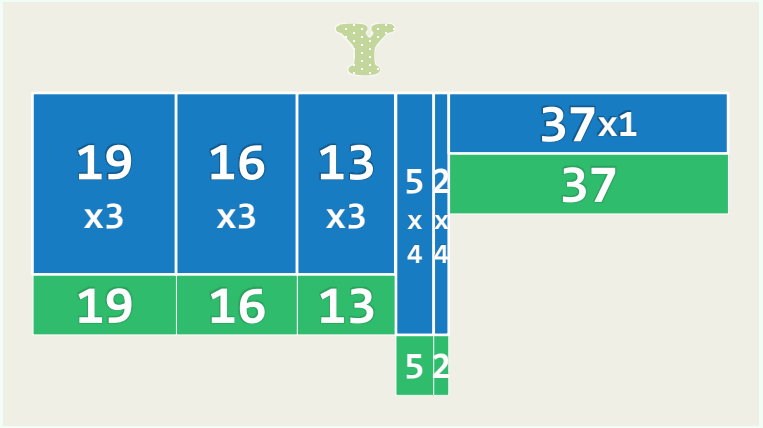
  此后只出不进

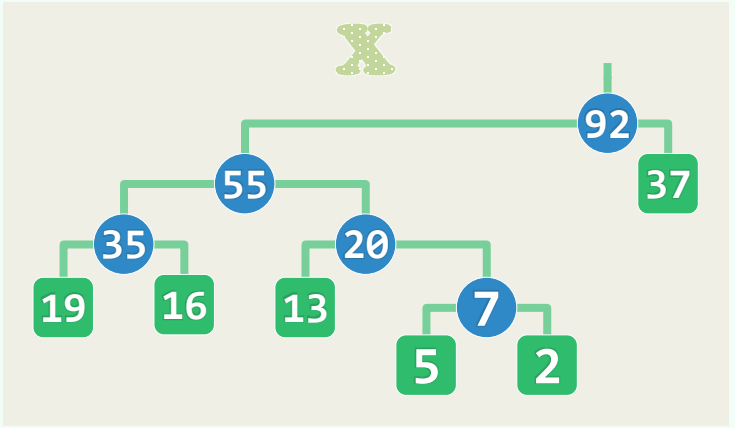
- 另一队列，初始为空

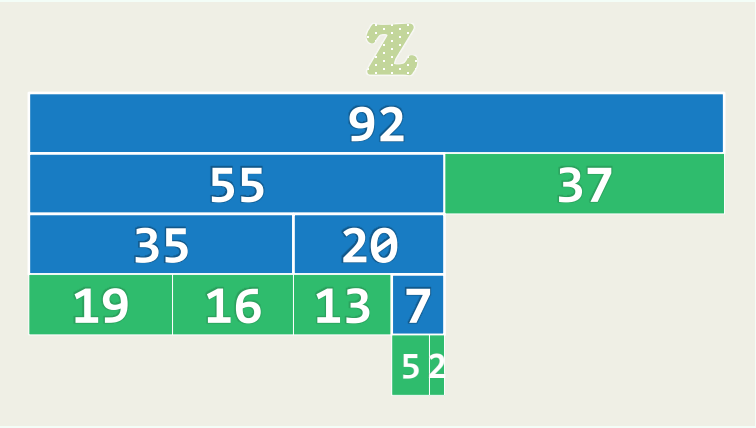
  合成的树，随即入队

- 总计 $\mathcal{O}(n \log n + \cancel{n})$









森林 ~ 栈+队列：实现

```
struct HuffForest { //用一个栈加一个队列来实现Huffman森林
   Stack<HuffTree> S; Queue<HuffTree> Q; //无论栈还是队列，全程都各自有序
   Rank size() { return S.size() + Q.size(); }
   HuffForest( uint32_t* freq, char s, char t ) { //初始化
      for ( char c = s; c < t; c++ ) //将[s,t)内的字符
         { S.push( HuffTree().insert({freq[c-s], c}) ); } //存入S，并
      S.sort(); //按优先级排序（频数逆序）
   }
   void insert( HuffTree t ) { Q.enqueue( t ); } //合并之后的树，均由队列保存
   HuffTree delMax() //优先级最大（频数最小）的树，只可能在栈顶或队首
      { return S.empty() || (!Q.empty() && (Q.front() > S.top())) ? Q.dequeue() : S.pop(); }
}; //栈与队列始终各自有序，delMax()仅是O(1)；合并而得的树总是归入队列，insert()也仅O(1)
```

05. Binary Tree

A. 树（术语和ADT）

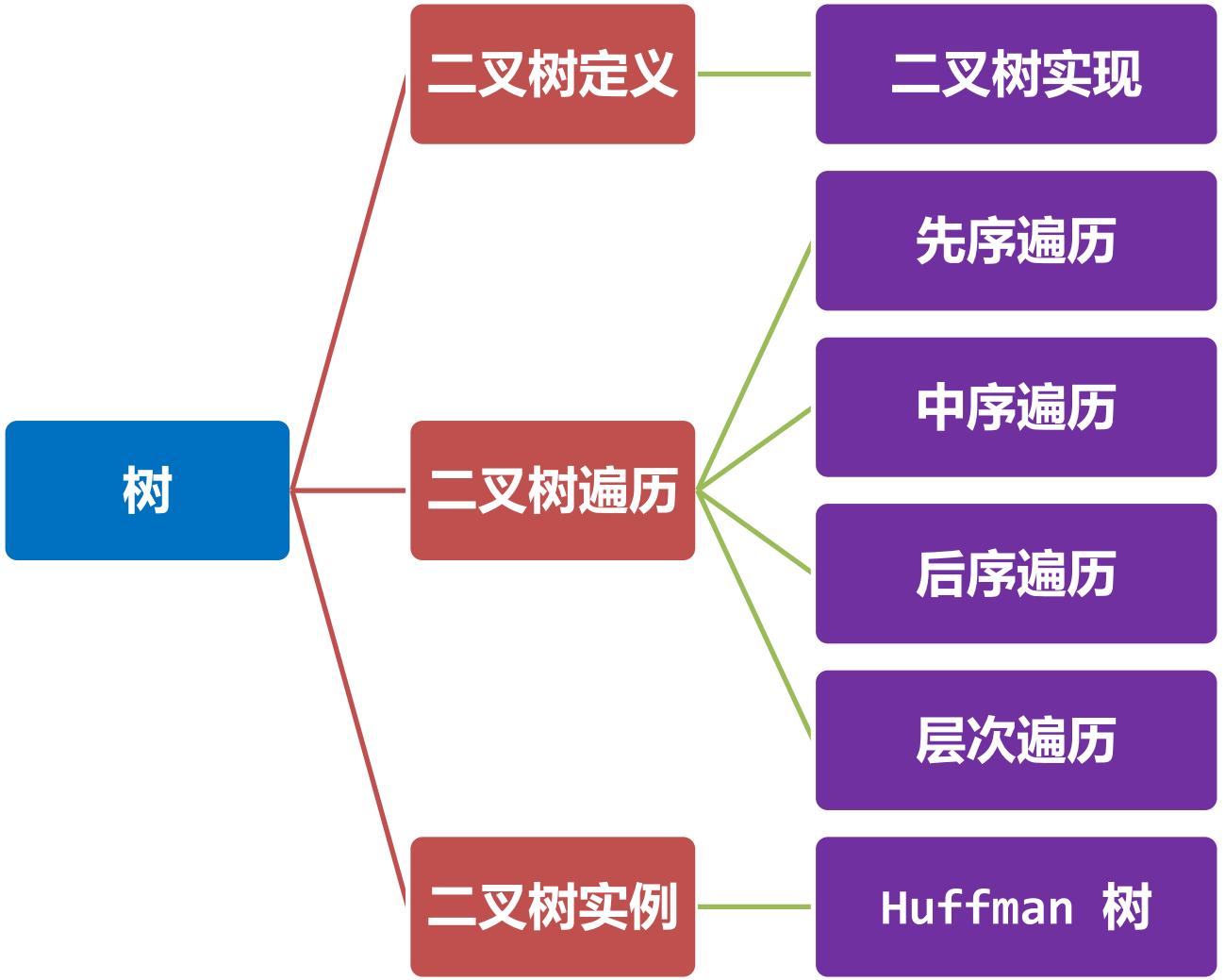
B. 二叉树（转换与实现）

C. 二叉树遍历

- 先序、中序、后序
- 层次、重构

D. Huffman 树

E. 下界与最优



05.Binary Tree

E-下界与最优

二叉树

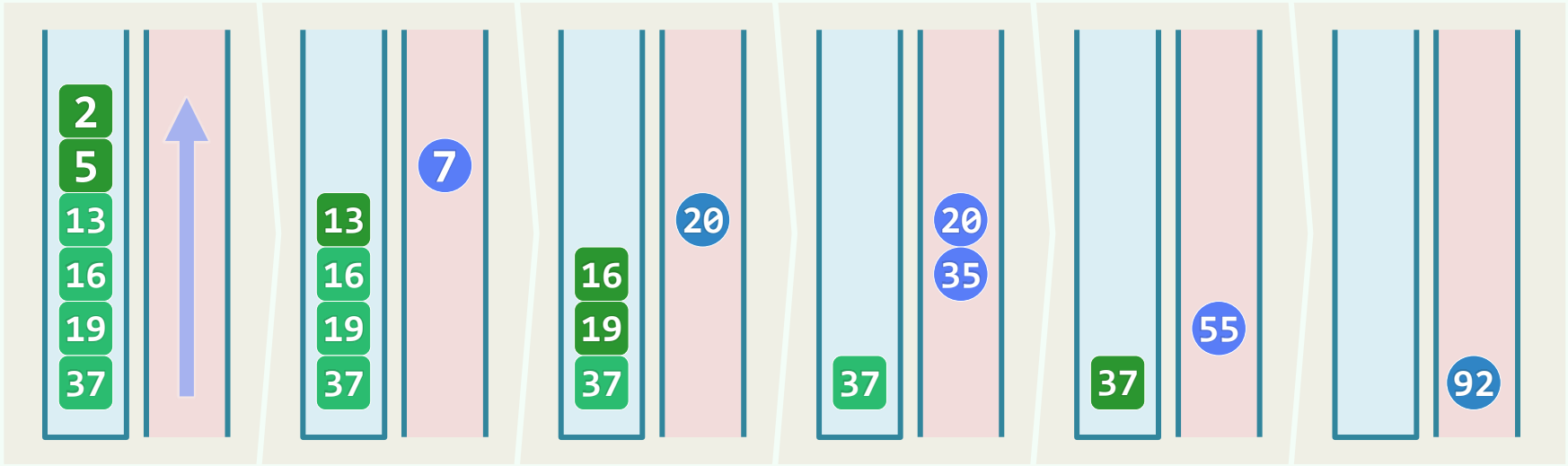
下界与最优：比较树

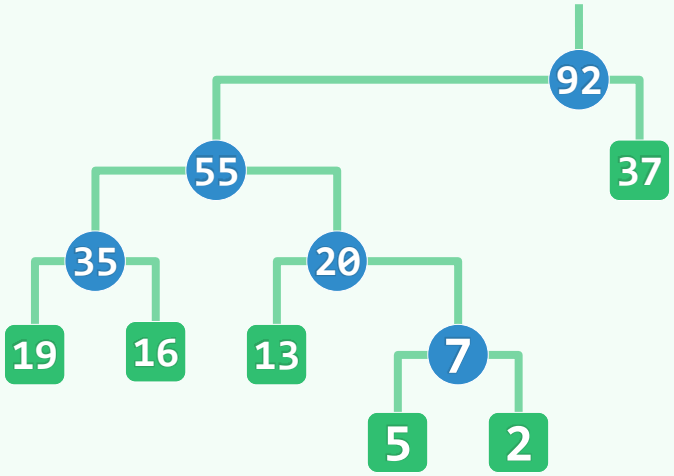
邓俊辉
deng@tsinghua.edu.cn

下界 ~ 最优

❖ 我们已经多次看到，同一问题的不同算法，复杂度可能相差悬殊

通过敏锐观察与精巧设计，往往能使算法性能有所改进





❖ 针对任何一个具体的计算问题，算法的改进不可能持续获得

- 算法改进的尽头，称作该问题的复杂度下界|lower bound
- 性能达到下界的算法，即最坏情况下最优的算法|worst-case optimal algorithm

苹果甄别

- 从4个外观相同的苹果中

  找出唯一的重量不同者

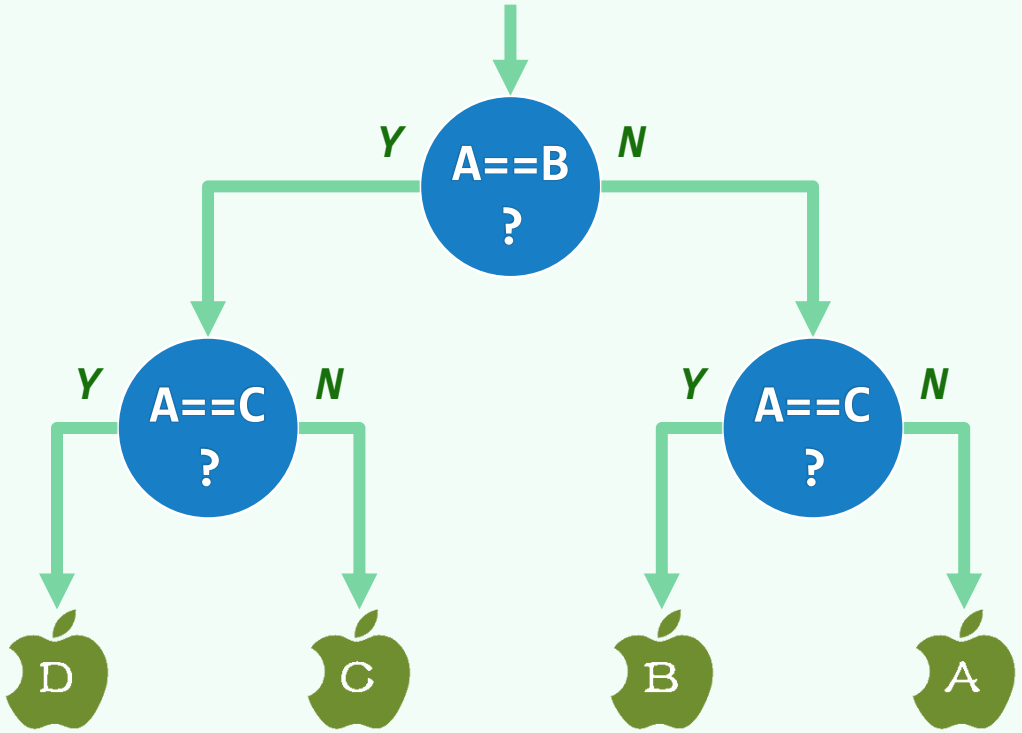
- 借助天平，只需2次称量

- 称量次数可否更少?

  2次已是下界——1次无论如何不够

- 对于更一般的问题

  如何确定下界呢...



比较树

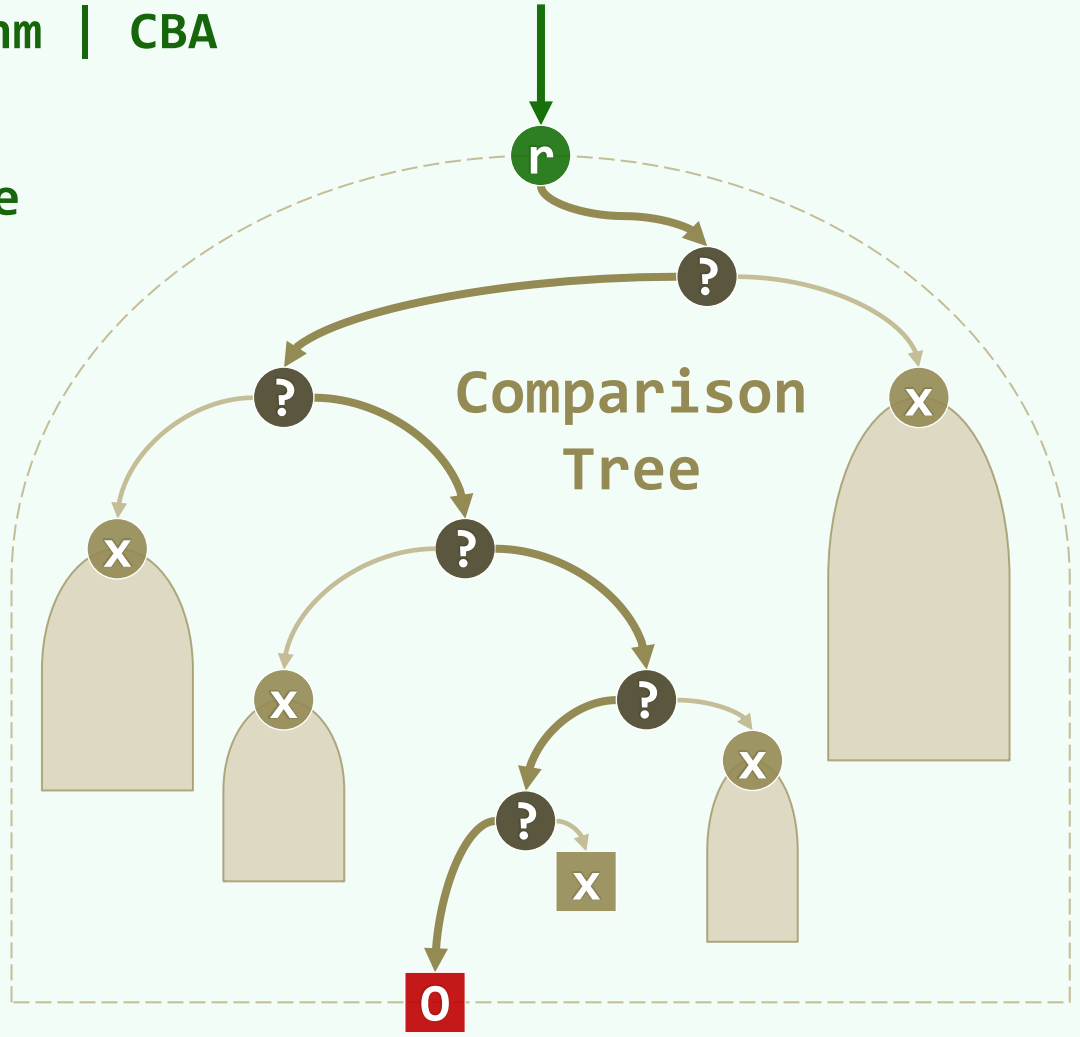
❖ 基于比较的算法 | comparison-based algorithm | CBA

❖ 每个CBA都对应于一棵比较树 | comparison tree

- 运行的输出 ~ 树中的叶节点
- 运行的过程 ~ 从根到叶节点的路径
- 消耗的时间 ~ 路径长度
- 最坏情况下消耗的时间 ~ 树的高度

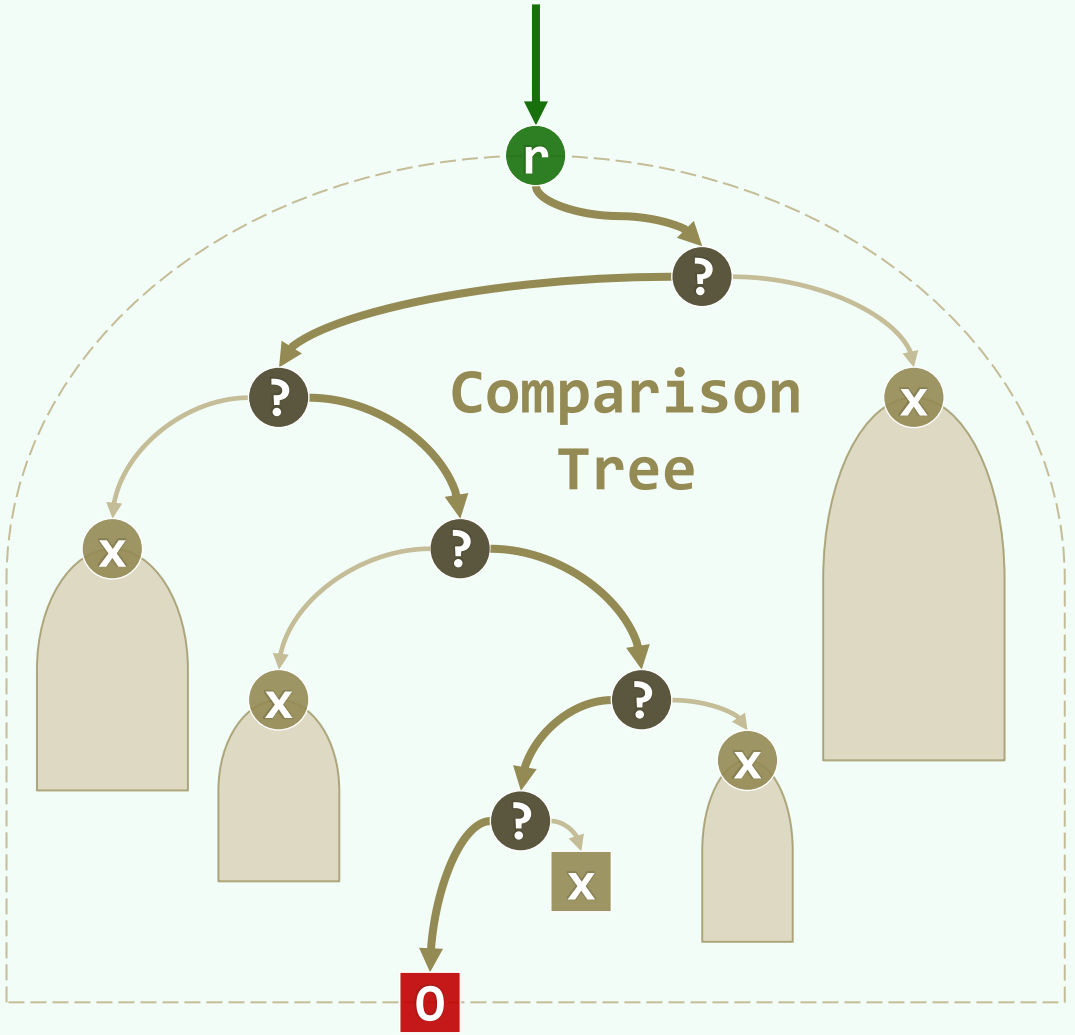
❖ n种输出 ~ 至少n个叶节点

下界 ~ 树高 = $\Omega(\log_2 n)$



实例

- 苹果甄别
  - 共有4种输出
  - 下界 $= \log_2 4 = 2$ 次称量
- n个元素的排序
  - 共有 $n!$ 种输入|输出
  - 下界 $= \Omega(\log_2 n!) = \Omega(n \log_2 n)$
  - 归并排序属于WCOA
- 每个问题的下界，都只能这样确定吗...



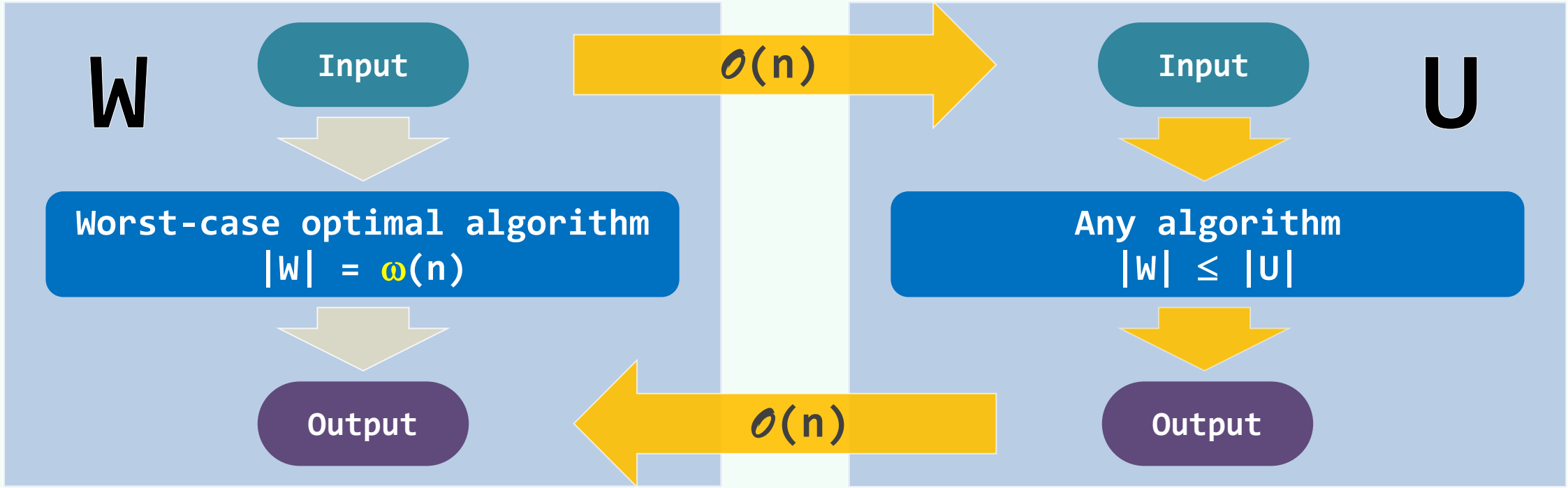
二叉树

下界与最优：线性归约

邓俊辉
deng@tsinghua.edu.cn

线性归约|linear-time reduction

❖ 只要 $W \leq_N U$，U的每个算法也对应于W的一个算法：输入转化 + U算法 + 输出转化 //曲径通幽



linear-time reduction

NP-complete/Polynomial-time reduction

P-SPACE complete/ Polynomial-time many-one reduction

SORTING $\leq_N$ HUFFMAN

- SORTING的每个输入 //n个实数

  可以直接转换为（理解为） //$O(1)$

  HUFFMAN的一个输入 //n个字符的频数

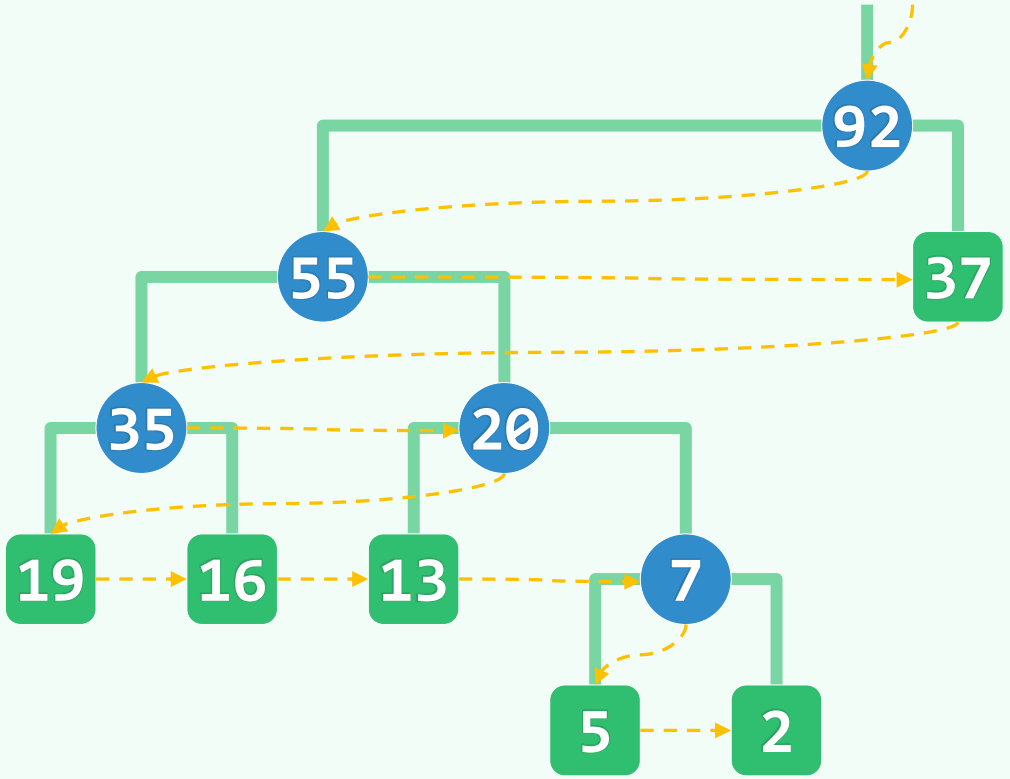
- HUFFMAN的输入 //编码树

  经层次遍历，必然得到 //高高低低，$O(n)$

  有序序列 //包含所有内部节点和叶节点

- 故有：　　SORTING $\leq_N$ HUFFMAN

  $\Omega(n\log n) = |\text{SORTING}| \leq |\text{HUFFMAN}|$



SORTING $\leq_N$ RBM

- [Red-Blue Matching] 任给平面上互不重合的红点、蓝点各n个，试用n条无交的线段联接异色点

- SORTING的每个输入 //n个实数

  可以直接转换为（理解为） //O(1)

  x正半轴上的n个红点 //O(n)预处理

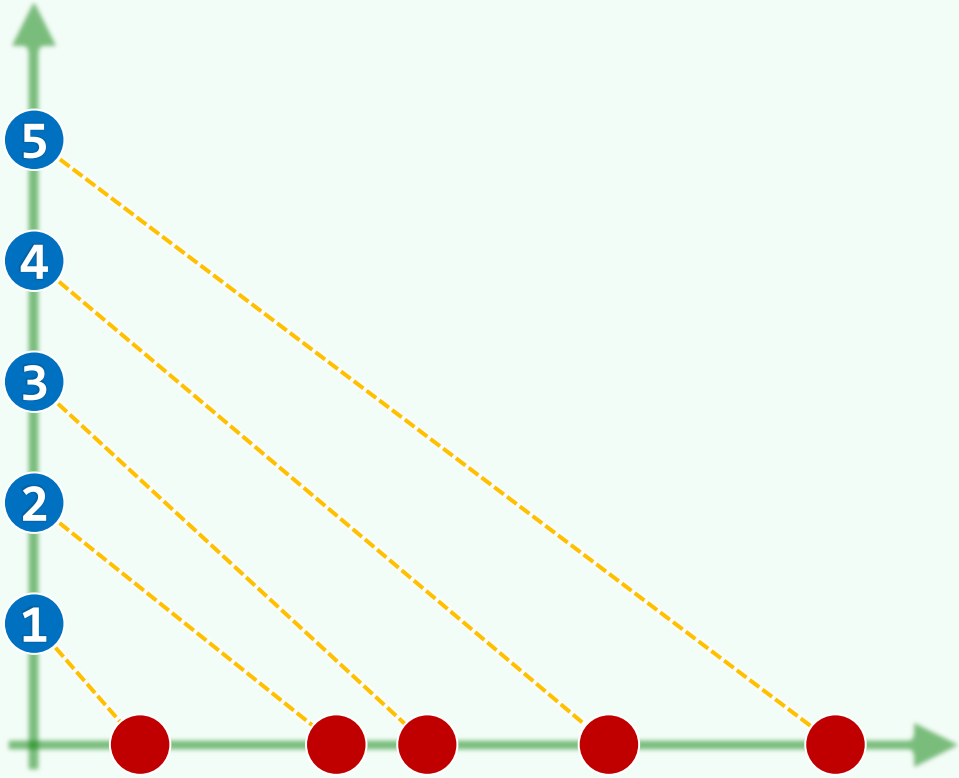
- 再加入y正半轴上有序的n个蓝点

  即得到RBM的一个输入

- RMB对应的输出

  只需一趟遍历，便可转化为 //O(n)

  SORTING的输出



$SD \leq_N DIAM$

- [Set Disjointness] 任给整数集合A和B，判断是否有公共元素

  已知：Ω(nlogn) = |SD|

- 【DIAM】直径|diameter

  在平面上任给一组点，找出相距最远的点对

- 对于SD的每个输入，将A|B中的整数

  分别映射到单位圆的上|下半圆弧 //比如CCW

  于是...

- SD的输出为true|false，当且仅当

  DIAM的输出为2——点集中存在对跖点|antipodals

